MATH FOR DATA SCIENCE

A Comprehensive Reference & Learning Guide to Advanced Mathematics, Machine Learning, and Deep Learning

Huỳnh Trung Nghĩa (NghiaHoang)

Welcome to the **Math for Data Science** blog. This resource is a curated reference and learning guide on the core mathematical foundations of Data Science, Machine Learning, and Deep Learning, authored by **Huỳnh Trung Nghĩa**.

🎯 Core Pillars

- **Calculus & Linear Algebra**: Functions, matrices, eigenvalues, gradients, and integration.
- **Probability & Statistics**: Distributions, conditional probability, Bayes' theorem, and descriptive/inferential statistics.
- **Optimization**: Convexity, duality, optimization algorithms (like gradient descent), and combinatorial optimization.

🚀 Practical Focus

Rather than dry mathematical proofs, we focus on **conceptual intuition** coupled with **step-by-step Python implementations** using libraries like `numpy`, `scipy`, `matplotlib`, and `scikit-learn`.

🗺️ Blog Map & Roadmap

The content is organized into 10 key modules, guiding you from basic arithmetic to advanced machine learning and deep learning foundations:

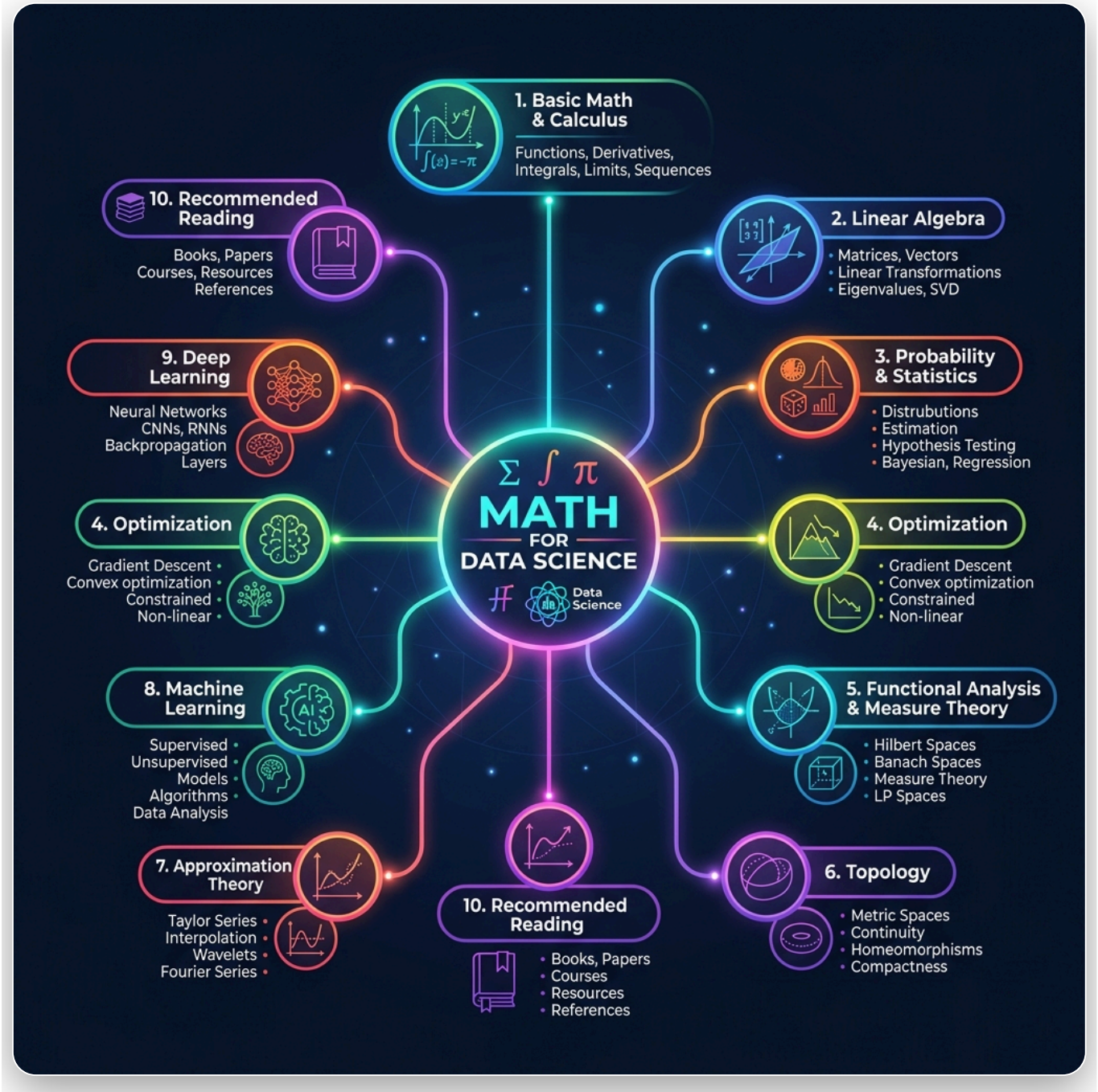


Fig. 1

Math for Data Science — Learning Roadmap

☕ Buy Me a Coffee / Support the Author

If you find these articles helpful and would like to support the ongoing creation of high-quality Data Science & AI content:

Table of Contents

Welcome to the full **Table of Contents** for *Math for Data Science*. Below is the comprehensive structure of all chapters across all modules:

📖 Basic Math and Calculus

🧮 Linear Algebra

🎲 Probability & Statistics

- Random Variables & Probability Distributions
- Convergence & Limit Theorems
- Information Theory Basics
- Statistics
- Statistical Inference
- Stochastic Processes
- Markov Chains

⚡ Optimization

- Introduction to Optimization
- Convex Sets & Functions
- Convex Optimization Problems
- Duality & KKT Conditions
- Gradient Descent & Hessian Curvature
- Optimization Algorithms
- Numerical Algorithms & Interior-Point Methods
- Geometric and Statistical Optimization
- Combinatorial Optimization & Scheduling

🌀 Functional Analysis & Measure Theory

- Functional Analysis & Operator Theory
- Measure Theory & Lebesgue Integration

🌐 Topology

- Introduction
- Topological Spaces & Metric Spaces
- Manifolds & Dimensionality Reduction
- Topological Data Analysis (TDA) & Persistent Homology
- Advanced Topological Algorithms
- Topological Surfaces & Singularities
- Interdisciplinary Applications of Topology

📈 Approximation Theory

- Approximation Theory
- Approximation and Fitting

🤖 Machine Learning

- Machine learning
- Methodology for usage
- Supervised Learning
- Unsupervised Learning
- Distributed Learning

🧠 Deep Learning

- Deep Learning Foundations

📚 Reference

- Recommended Reading & Resources

📄 Download PDF

- Download PDF Version

Introduction

Welcome to the **Basic Math and Calculus** section. This module covers the essential mathematical foundations required for data science, machine learning, and statistical analysis, structured after the textbook *Essential Math for Data Science*.

📂 Topics Covered

In this section, you will explore:

- **Number**: Understanding number systems (Natural, Whole, Integers, Rational, Irrational, Real, and Complex numbers) which form the foundation of computing and data representation.
- **Fractions**: Proportions, ratios, and fractional calculations.
- **Decimals & Percentages**: Decimals, percentage change, and performance metric conversions.
- **Order of Operations**: Following the PEMDAS rules in mathematical calculations.
- **Variables**: Using variable placeholders, solving linear and quadratic equations.
- **Functions**: Domain mapping, linear equations, slope, and intercept.
- **Curvilinear Functions**: Working with non-linear curves, parabolas, exponentials, logs, and limits.
- **Summations**: Iterative addition and using Sigma ($\sum$) notation in statistics and cost functions.
- **Exponents**: Understanding powers, square roots, and Euler's constant.
- **Logarithms**: Properties of logs, natural log, and log-transformations.
- **Limits**: Understanding limits, continuity, and formal definitions.
- **Calculus & Derivatives**: Exploring derivatives, partial derivatives, chain rule, and gradients.
- **Integration**: Learning about integrals, Riemann sums approximation, symbolic/numerical integration, and probability density functions.

🚀 Get Started

Select **Number** from the sidebar to begin your mathematical journey!

Number Sets & Systems

Understanding number classification and arithmetic rules is fundamental to data representation, indexing, and mathematical operations in Data Science.

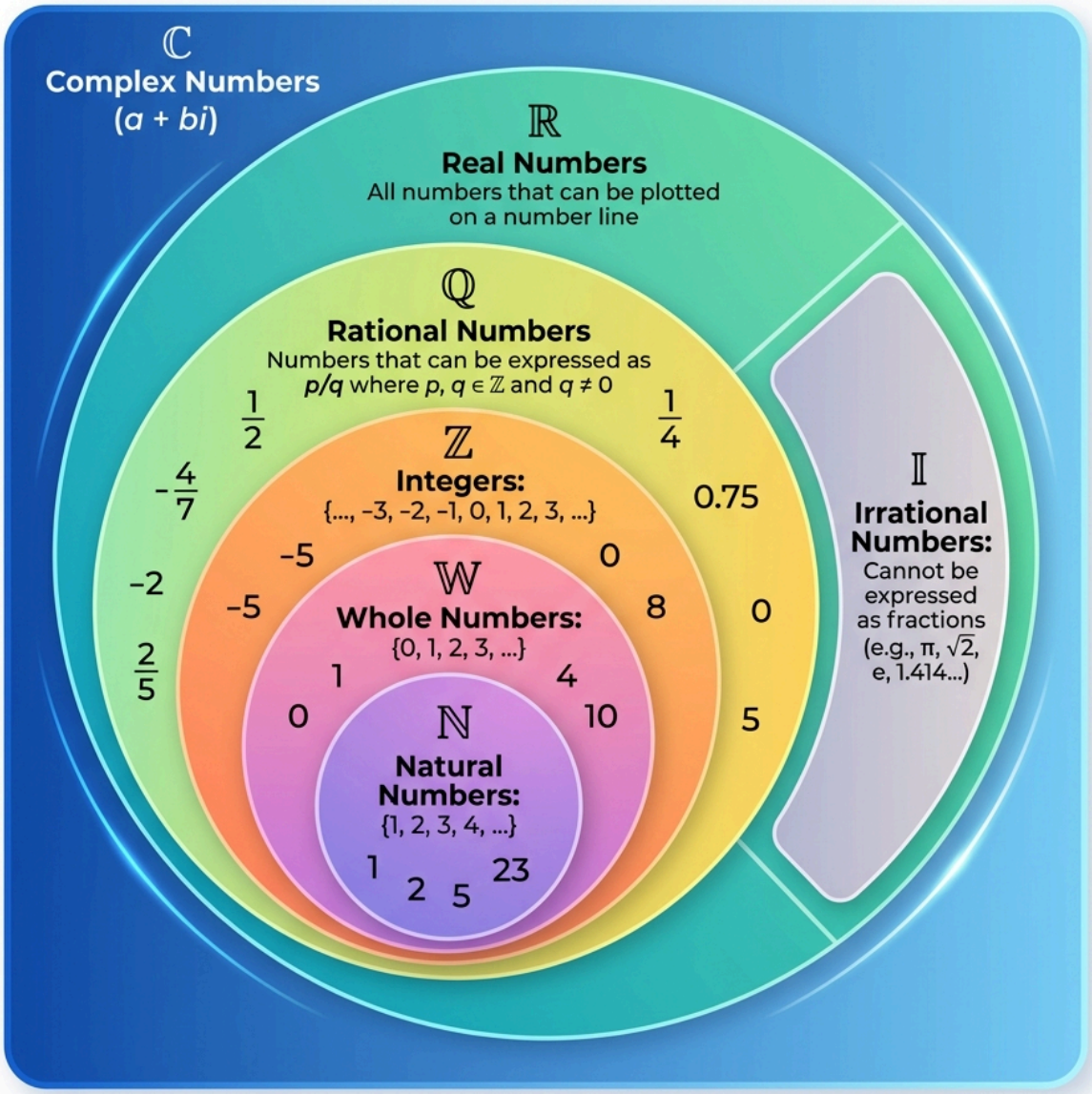


Fig. 2
Hierarchy of number sets.

1. Natural Numbers ($\mathbb{N}$)

The **natural numbers** are the positive counting numbers $1, 2, 3, \ldots$, possibly including 0 as well depending on the mathematical convention:

- Starting with 0 corresponds to the **non-negative integers**: $\{0, 1, 2, 3, \ldots\}$.
- Starting with 1 corresponds to the **positive integers**: $\{1, 2, 3, \ldots\}$.

Roles of Natural Numbers

Natural numbers serve three distinct roles in data and everyday life:

1. **Cardinal Numbers:** Used for counting quantities (e.g., *"there are six coins on the table"*).
2. **Ordinal Numbers:** Used for ordering or ranking (e.g., *"this is the third largest city in the country"*).
3. **Nominal Numbers:** Used purely as labels or identifiers, carrying no quantitative value (e.g., sports jersey numbers or user ID codes).

- **Set Notation:** $\mathbb{N} = \{1, 2, 3, 4, 5, \ldots\}$

---

2. Whole Numbers ($\mathbb{W}$)

Whole numbers consist of all **natural numbers** together with **zero (0)**.

- **Set Notation:** $\mathbb{W} = \{0, 1, 2, 3, 4, \ldots\}$
- **Data Science Note:** Zero-indexed data structures (such as Python lists, NumPy arrays, and Pandas DataFrames) rely on whole numbers for indexing positions $0, 1, 2, \ldots, n-1$.

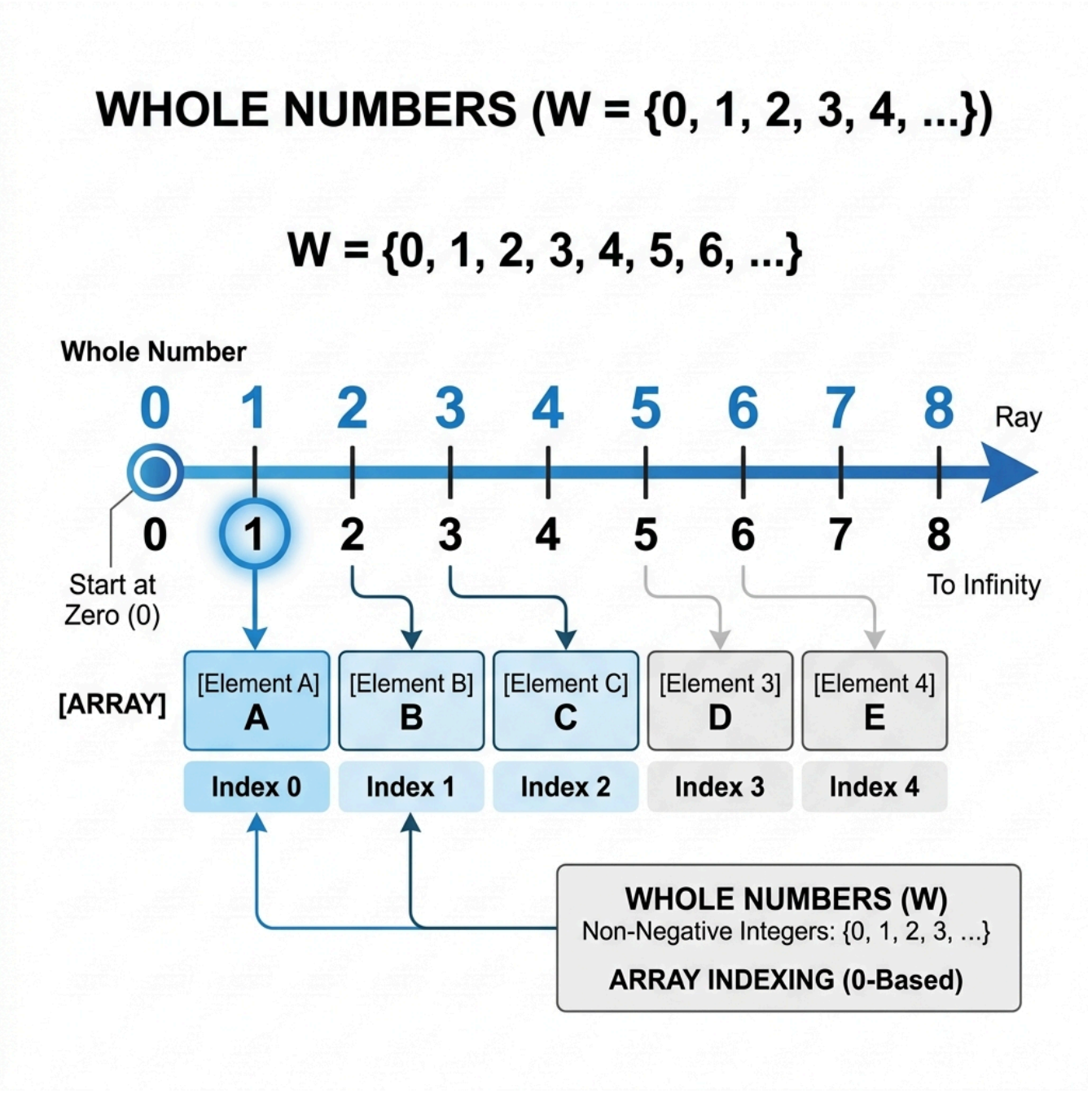


Fig. 3
Whole numbers starting at zero and their application in 0-based array indexing.

3. Integers ($\mathbb{Z}$)

Integers extend whole numbers to include negative values. In mathematics, the set of integers is denoted by the boldface letter $\mathbb{Z}$ (from the German word *Zahlen*, meaning "numbers").

- **Set Notation:** $\mathbb{Z} = \{\ldots, -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5, \ldots\}$

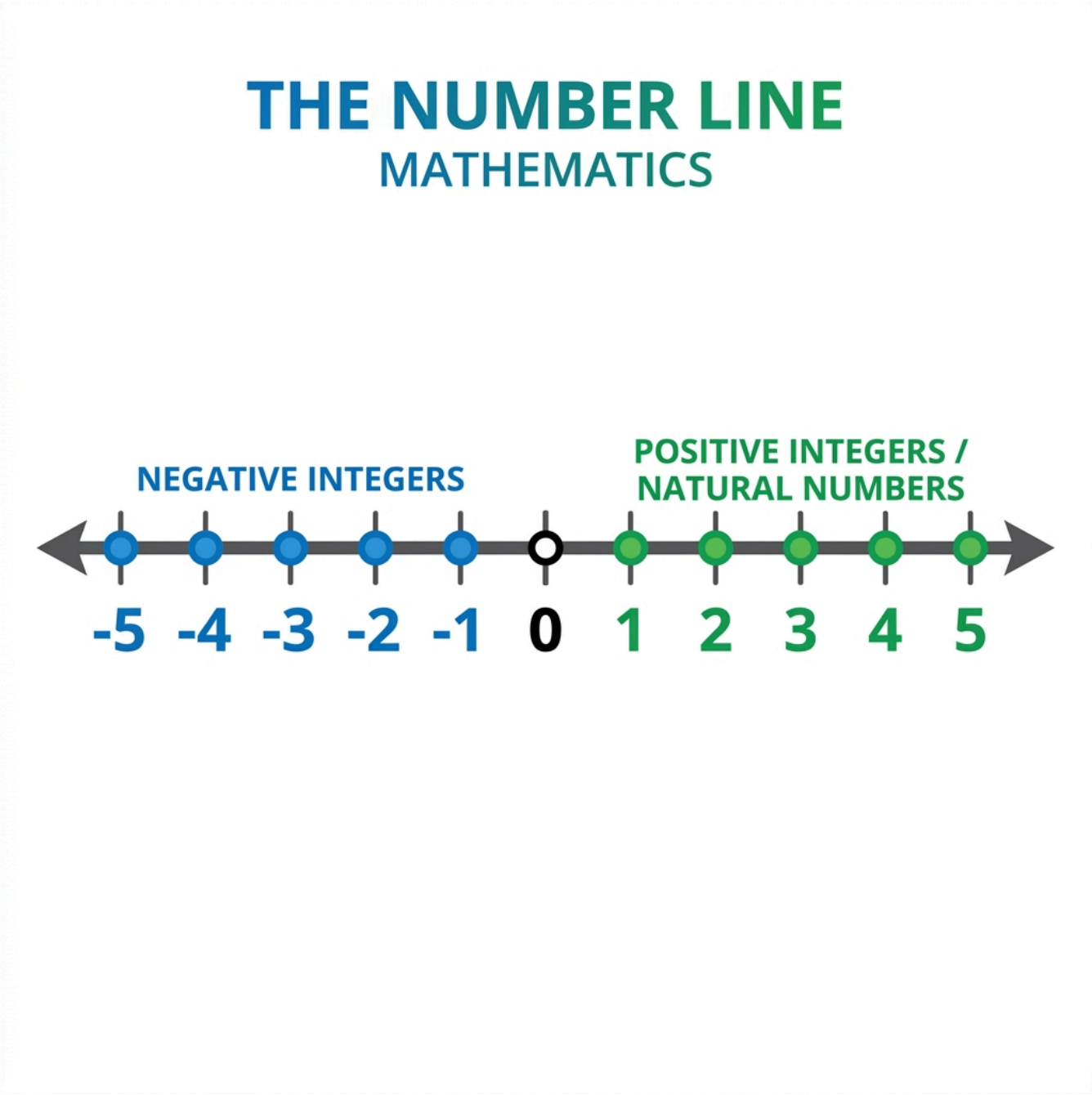


Fig. 4
Number line showing negative integers, zero, and positive integers.

4. Rational Numbers ($\mathbb{Q}$)

A **rational number** is any number that can be expressed as the quotient or fraction $\frac{p}{q}$ of two integers, where p is the numerator and $q \neq 0$ is the non-zero denominator.

The set of all rational numbers is denoted by the boldface letter $\mathbf{Q}$ or blackboard bold $\mathbb{Q}$ (for "quotient").

- **Examples:** $\frac{1}{2}$, $\frac{-3}{7}$, and every integer since any integer k can be written as $\frac{k}{1}$ (e.g., $5 = \frac{5}{1}$).

- **Decimal Expansion:** Rational numbers have decimal expansions that either terminate (e.g., $\frac{3}{4} = 0.75$) or repeat infinitely (e.g., $\frac{1}{3} = 0.3333\ldots$).

---

5. Irrational Numbers ($\mathbb{I}$)

Irrational numbers cannot be expressed as a fraction $\frac{p}{q}$ of two integers.

Geometrically, when the ratio of lengths of two line segments is an irrational number, the segments are described as **incommensurable** — meaning they share no common unit of measure, no matter how small, that can divide both lengths into exact integer multiples.

Key Characteristics & Famous Examples

Irrational numbers possess an infinite number of non-repeating decimal digits:

- **Circle Ratio (π):** $\pi = 3.14159265\ldots$ (the ratio of a circle's circumference to its diameter).
- **Euler's Number (e):** $e = 2.71828182\ldots$ (the base of the natural logarithm).
- **Golden Ratio (ϕ):** $\phi = \frac{1+\sqrt{5}}{2} \approx 1.618033\ldots$
- **Square Roots:** $\sqrt{2} \approx 1.414213\ldots$, $\sqrt{3}$, $\sqrt{5}$. In fact, all square roots of natural numbers that are not perfect squares are irrational.
- **Non-repeating Decimals:** Patterns like $2.010010001\ldots$

---

6. Real Numbers ($\mathbb{R}$)

The set of **Real Numbers** ($\mathbb{R}$) is the union of all rational and irrational numbers.

In practical Data Science and Machine Learning, almost all continuous feature values (such as prices, weights, temperatures, or probability scores) are represented as real numbers using floating-point data types (`float32` or `float64`).

---

7. Complex and Imaginary Numbers ($\mathbb{C}$)

Imaginary Numbers

Numbers that are not real are called **imaginary numbers**. Squaring an imaginary number yields a negative result.

- **Examples:** $\sqrt{-2}$, $\sqrt{-7}$, $\sqrt{-11}$.

Imaginary numbers arose historically to solve polynomial equations like $x^2 + 1 = 0$, whose solutions $x = \pm\sqrt{-1}$ cannot exist within the real number line $\mathbb{R}$. We denote $\sqrt{-1}$ with the symbol i (called **iota**), so $i^2 = -1$.

Complex Numbers

A **Complex Number** is a combination of a real number and an imaginary number written in the standard form:

$$z = a + bi \quad (\text{where } a, b \in \mathbb{R} \text{ and } i = \sqrt{-1})$$

- a is the **Real Part** ($\mathrm{Re}(z)$).
- b is the **Imaginary Part** ($\mathrm{Im}(z)$).

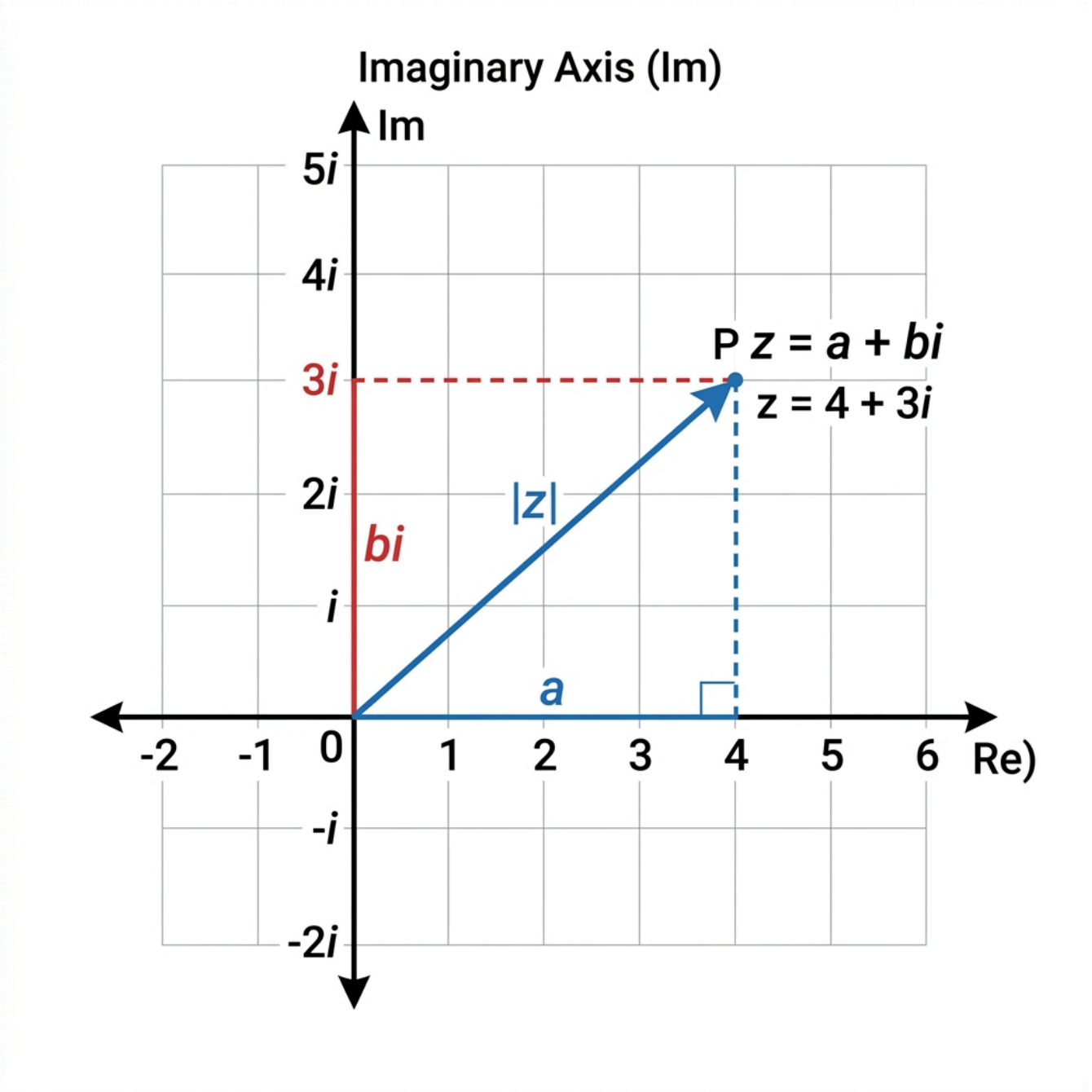


Fig. 5
Complex plane representation of a complex number.

8. Order of Operations (PEMDAS)

When evaluating mathematical expressions, we must follow the strict order of operations (**PEMDAS**) to ensure consistent results:

1. **P**arentheses `( )`
2. **E**xponents (Powers and Roots)
3. **M**ultiplication and **D**ivision (left to right)

4. **A**ddition and **S**ubtraction (left to right)

Python Verification

Python automatically enforces PEMDAS rules:

```
# Expression: 3 + 5 * 2^3 - (4 / 2)
# 1. Parentheses: (4 / 2) -> 2.0
# 2. Exponents: 2^3 -> 8
# 3. Multiplication: 5 * 8 -> 40
# 4. Addition & Subtraction: 3 + 40 - 2.0 -> 41.0
result = 3 + 5 * 2**3 - (4 / 2)
print("Result of expression:", result) # Output: 41.0
```

Exercises

Exercise 1

Evaluate the expression using PEMDAS rules:

$$12 - 3 \times (2 + 1)^2 \div 9 + 4$$

Verify your answer with Python.

Solution — Exercise 1

According to PEMDAS:

1. Parentheses: $(2 + 1) = 3$
2. Exponents: $3^2 = 9$
3. Multiplication/Division (left to right): $3 \times 9 \div 9 = 3$
4. Addition/Subtraction (left to right): $12 - 3 + 4 = 13$

```
ans = 12 - 3 * (2 + 1)**2 / 9 + 4
print("Answer:", ans) # Output: 13.0
```

Fractions

Fractions represent a part of a whole. In data science, fractions are the foundation of probabilities, proportions, and ratios (such as precision and recall).

1. What Is a Fraction?

A fraction is written as:

$$\frac{a}{b}$$

Where:

- a is the **numerator** (how many parts we have).
- b is the **denominator** (how many equal parts the whole is divided into).

2. Basic Operations

Addition & Subtraction

To add or subtract fractions, they must have a **common denominator**:

$$\frac{1}{3} + \frac{1}{2} = \frac{2}{6} + \frac{3}{6} = \frac{5}{6}$$

Multiplication

Multiply numerators together and denominators together:

$$\frac{a}{b} \times \frac{c}{d} = \frac{a \cdot c}{b \cdot d}$$

$$\frac{2}{3} \times \frac{3}{4} = \frac{6}{12} = \frac{1}{2}$$

Division

To divide by a fraction, multiply by its **reciprocal**:

$$\frac{a}{b} \div \frac{c}{d} = \frac{a}{b} \times \frac{d}{c} = \frac{a \cdot d}{b \cdot c}$$

---

3. Python Fraction Module

Python has a built-in module `fractions` to handle exact fraction arithmetic:

```python
from fractions import Fraction

# Create fractions
f1 = Fraction(1, 3)
f2 = Fraction(1, 2)

# Arithmetic operations
print("1/3 + 1/2 =", f1 + f2)
print("2/3 * 3/4 =", Fraction(2, 3) * Fraction(3, 4))
```

Exercises

Exercise 1

Calculate:

$$\frac{3}{4} \div \frac{2}{3} - \frac{1}{2}$$

Solution — Exercise 1

1. Perform division first:

$$\frac{3}{4} \times \frac{3}{2} = \frac{9}{8}$$

2. Perform subtraction:

$$\frac{9}{8} - \frac{1}{2} = \frac{9}{8} - \frac{4}{8} = \frac{5}{8}$$

Decimals & Percentages

Decimals and percentages are alternative ways of expressing fractions and proportions. In Data Science, we constantly work with probability values as decimals (e.g., 0.85) and interpret evaluation metrics as percentages (e.g., 85%).

---

1. Decimals

A decimal is a fraction whose denominator is a power of ten (10, 100, 1000, etc.), written using a decimal point.

For example:

$$\frac{7}{10} = 0.7$$

$$\frac{125}{1000} = 0.125$$

Repeating Decimals

Some fractions produce infinitely repeating decimals:

$$\frac{1}{3} = 0.3333\ldots$$

---

2. Percentages

The word **percent** means “per hundred”. To convert a decimal to a percentage, multiply by 100 and add the $\%$ sign.

$$0.85 \times 100 = 85\%$$

Percentage Change

In data analysis, we often compute the percentage change between two values (e.g. increase in website traffic):

$$\text{Percentage Change} = \frac{\text{New Value} - \text{Old Value}}{\text{Old Value}} \times 100\%$$

---

3. Python Implementations

```python
# Calculating percentage change
old_revenue = 150
new_revenue = 180

pct_change = ((new_revenue - old_revenue) / old_revenue) * 100
print(f"Revenue growth: {pct_change:.1f}%")
```

Exercises

Exercise 1

A model correctly classified 45 out of 60 test images. What is the classification accuracy in percentage?

Solution — Exercise 1

Compute the proportion:

$$\text{Accuracy} = \frac{45}{60} = 0.75$$

Convert to percentage:

$$0.75 \times 100\% = 75\%$$

Order of Operations

In mathematics and computer science, we evaluate expressions based on a strict order of operations to avoid ambiguity. The standard convention is **PEMDAS** (or **BODMAS** in some regions).

---

1. The PEMDAS Rule

When solving mathematical expressions, perform calculations in the following order:

1. **P**arentheses `()`
2. **E**xponents (Powers, Roots)
3. **M**ultiplication and **D**ivision (evaluated from left to right)
4. **A**ddition and **S**ubtraction (evaluated from left to right)

---

2. Python Implementation

Python automatically follows PEMDAS rules when calculating expressions:

```
# Expression: 3 + 5 * 2^3 - (4 / 2)
# Step-by-step:
# 1. Parentheses: (4 / 2) -> 2.0
# 2. Exponents: 2**3 -> 8
# 3. Multiplication: 5 * 8 -> 40
# 4. Add/Subtract: 3 + 40 - 2.0 -> 41.0

result = 3 + 5 * 2**3 - (4 / 2)
print("Computed result:", result)
```

Exercises

Exercise 1

Evaluate the expression manually:

$$10 - 2 \times (3 + 1)^2 \div 8 + 5$$

Verify your answer with Python.

Solution — Exercise 1

Using PEMDAS:

1. Parentheses: $(3 + 1) = 4$
2. Exponents: $4^2 = 16$
3. Multiplication/Division (left to right):
   - $2 \times 16 = 32$
   - $32 \div 8 = 4$
4. Addition/Subtraction (left to right):
   - $10 - 4 = 6$
   - $6 + 5 = 11$

Variables

In mathematics and programming, a **variable** is a named placeholder or symbol (such as x, y, or w) used to represent a value that can change or is currently unknown. In Machine Learning, variables represent our data features and model parameters.

---

1. Linear Equations

A linear equation is an equation of a straight line, typically written as:

$$ax + b = 0$$

To solve for x:

$$x = -\frac{b}{a}$$

Multi-variable Linear Equations

In Machine Learning, we often map relationships as linear combinations:

$$y = w_1x_1 + w_2x_2 + b$$

This represents a plane in 3D space, which is the foundation of Multiple Linear Regression.

---

2. Quadratic Equations

A quadratic equation is in the form:

$$ax^2 + bx + c = 0$$

The solutions (roots) can be found using the quadratic formula:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

---

3. Symbolic Algebra in Python with SymPy

We can solve algebraic equations exactly using SymPy:

```
import sympy as sp

x = sp.symbols('x')

# Solve: 2x - 6 = 0
eq1 = sp.Eq(2*x - 6, 0)
sol1 = sp.solve(eq1, x)
print("Solution for 2x - 6 = 0:", sol1)

# Solve quadratic: x^2 - 5x + 6 = 0
eq2 = sp.Eq(x**2 - 5*x + 6, 0)
sol2 = sp.solve(eq2, x)
print("Solutions for x^2 - 5x + 6 = 0:", sol2)
```

Exercises

Exercise 1

Solve the quadratic equation:

$$x^2 - 4x - 5 = 0$$

Solution — Exercise 1

Using coefficients $a = 1, b = -4, c = -5$:

$$x = \frac{-(-4) \pm \sqrt{(-4)^2 - 4(1)(-5)}}{2(1)}$$

$$x = \frac{4 \pm \sqrt{16 + 20}}{2} = \frac{4 \pm \sqrt{36}}{2}$$

$$x_1 = \frac{4 + 6}{2} = 5, \quad x_2 = \frac{4 - 6}{2} = -1$$

Functions

A **function** is a mathematical rule that takes an input (usually denoted as x) and maps it to a unique output (usually denoted as y or $f(x)$). In Data Science, we use functions to represent relationships between features (inputs) and targets (outputs).

---

1. What Is a Function?

Formally, a function f maps elements from a domain X to a codomain Y:

$$f : X \to Y$$

For every input $x \in X$, there is exactly one output $y \in Y$ such that $y = f(x)$.

---

2. Linear Functions

A **linear function** is a function that creates a straight line when graphed. Its general algebraic form is:

$$y = mx + c$$

Where:

- m is the **slope** (rate of change, or gradient). It determines the steepness of the line:

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

- c is the **y-intercept** (where the line crosses the y-axis, i.e., $f(0)$).

In Machine Learning, this equation forms the basis of **Simple Linear Regression**, where m is the weight (w) and c is the bias (b):

$$y = wx + b$$

---

3. Plotting Linear Functions in Python

We can use `matplotlib` to plot linear functions and visualize their slopes and intercepts:

```
import numpy as np
import matplotlib.pyplot as plt

# Generate x values
x = np.linspace(-10, 10, 100)

# Define two linear functions
y1 = 2 * x + 3   # slope = 2, intercept = 3
y2 = -0.5 * x - 1 # slope = -0.5, intercept = -1

# Plot
plt.figure(figsize=(8, 5))
plt.plot(x, y1, label="y = 2x + 3", color="blue")
plt.plot(x, y2, label="y = -0.5x - 1", color="red")
plt.axhline(0, color='black',linewidth=0.5)
plt.axvline(0, color='black',linewidth=0.5)
plt.grid(color='gray', linestyle='--', linewidth=0.5)
plt.legend()
plt.title("Linear Functions")
plt.savefig('../../images/linear_functions.png', dpi=150)
plt.show()
```

Exercises

Exercise 1

A line passes through the points $(1, 3)$ and $(3, 7)$.

1. Find the slope of the line.
2. Determine the equation of the line.

Solution — Exercise 1

1. Slope (m):

$$m = \frac{7-3}{3-1} = \frac{4}{2} = 2$$

2. Using the point-slope form $y - y_1 = m(x - x_1)$ with point $(1, 3)$:

$$y - 3 = 2(x - 1) \implies y = 2x + 1$$

Curvilinear Functions

A **curvilinear function** is a non-linear function whose graph forms a curve rather than a straight line. In Data Science, curvilinear functions model complex patterns, growth rates, activation thresholds (like Sigmoid), and loss curves.

---

1. Types of Curvilinear Functions

Quadratic Functions (Parabolas)

A quadratic function is a second-degree polynomial in the form:

$$y = ax^2 + bx + c$$

Its graph is a parabola. If $a > 0$, the parabola opens upwards (having a minimum). If $a < 0$, it opens downwards (having a maximum). This shape is central to squared loss functions ($MSE = \frac{1}{n}\sum(y_i - \hat{y}_i)^2$).

Exponential Functions

Exponential functions represent rapid growth or decay, in the form:

$$y = a \cdot b^x \quad \text{or} \quad y = a \cdot e^{kx}$$

Where $e \approx 2.71828$. This is used to model population growth, compounding interest, or decay rates in learning algorithms.

Logarithmic Functions

The logarithmic function is the inverse of the exponential function:

$$y = \log_b(x)$$

It grows very rapidly at first but levels off. In machine learning, log functions are used in log-transforms to scale heavily skewed features.

---

2. Limits and Curvature

Because the slope of a curvilinear function is constantly changing at every point, we cannot define a single slope.

- **Secant Line**: The average rate of change between two points (x_1, y_1) and (x_2, y_2).
- **Tangent Line**: The instantaneous rate of change at a single point, which we find using **limits** and **calculus**.

---

3. Plotting Curvilinear Functions in Python

Let's visualize a quadratic curve and the standard Logistic Sigmoid function ($S(x) = \frac{1}{1+e^{-x}}$) commonly used in Logistic Regression:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-6, 6, 200)

# Quadratic function
y_quad = x**2

# Sigmoid function
y_sigmoid = 1 / (1 + np.exp(-x))

# Plotting
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

# Parabola
ax1.plot(x, y_quad, color='blue', label=r'$y = x^2$')
ax1.set_title("Quadratic Function")
ax1.grid(True)
ax1.legend()

# Sigmoid
ax2.plot(x, y_sigmoid, color='green', label=r'$S(x) = \frac{1}{1 + e^{-x}}$')
ax2.set_title("Sigmoid Activation Function")
ax2.grid(True)
ax2.legend()

plt.savefig('../../images/curvilinear_functions.png', dpi=150)
plt.show()
```

Exercises

Exercise 1

Find the vertex (minimum point) of the quadratic function:

$$f(x) = x^2 - 6x + 9$$

Hint: The x-coordinate of the vertex of $ax^2 + bx + c$ is given by $x = -b/2a$.

Solution — Exercise 1

Using the formula $x = -b/2a$:

$$x = \frac{-(-6)}{2(1)} = 3$$

Compute $f(3)$:

$$f(3) = 3^2 - 6(3) + 9 = 9 - 18 + 9 = 0$$

The vertex (minimum) is at $(3, 0)$.

Summations

A **summation** is the operation of adding a sequence of numbers. In mathematics and data science, we represent this using the Greek capital letter **Sigma** ($\sum$).

1. What Is a Summation?

A summation is written in the following format:

$$\sum_{i=1}^{n} x_i$$

Where:

- i is the **index of summation** (start variable).

- 1 is the **lower limit** (starting point).
- n is the **upper limit** (ending point).
- x_i represents the term to be added at index i.

This expands to:

$$\sum_{i=1}^{n} x_i = x_1 + x_2 + x_3 + \cdots + x_n$$

Example:

$$\sum_{i=1}^{4} i = 1 + 2 + 3 + 4 = 10$$

---

2. Summations in Data Science

We use summations to define key algorithms and performance metrics.

Mean:

$$\bar{x} = \frac{\sum_{i=1}^{n} x_i}{n}$$

Mean Squared Error (MSE Loss):

$$MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

---

3. Python Implementation

You can compute summations in Python using loops or NumPy's `sum()` function:

```python
import numpy as np

# Data points
x = [2, 4, 6, 8]

# 1. Summation using built-in sum()
total = sum(x)
print("Sum of elements:", total)

# 2. Summation using NumPy
np_total = np.sum(x)
print("NumPy sum:", np_total)
```

Exercises

Exercise 1

Evaluate the summation:

$$\sum_{i=2}^{5}(2i-1)$$

Solution — Exercise 1

Expand the expression for $i = 2, 3, 4, 5$:

- For $i = 2$: $2(2) - 1 = 3$
- For $i = 3$: $2(3) - 1 = 5$
- For $i = 4$: $2(4) - 1 = 7$
- For $i = 5$: $2(5) - 1 = 9$

Sum the values:

$$3 + 5 + 7 + 9 = 24$$

Exponents

An **exponent** (or power) represents repeated multiplication of a number by itself. In Data Science, exponential functions model growth rates, decay rates, and activation functions.

---

1. What Is an Exponent?

An exponent is written as:

$$x^n$$

Where:

- x is the **base**.
- n is the **exponent** (or power).

This represents x multiplied by itself n times:

$$3^4 = 3 \times 3 \times 3 \times 3 = 81$$

Rules of Exponents

Exponents follow specific algebraic properties:

- **Product Rule**: $x^a \cdot x^b = x^{a+b}$
- **Quotient Rule**: $\frac{x^a}{x^b} = x^{a-b}$
- **Power of a Power Rule**: $(x^a)^b = x^{a \cdot b}$
- **Negative Exponent Rule**: $x^{-n} = \frac{1}{x^n}$
- **Fractional Exponents (Roots)**: $x^{\frac{1}{n}} = \sqrt[n]{x}$

---

2. Euler's Number (e)

Of special importance in calculus and data science is **Euler's number** $e \approx 2.71828$. It is an irrational number that represents the base of natural growth:

$$y = e^x$$

---

3. Python Implementation

```python
import math

# Exponential calculations: 3^4
print("3^4 =", 3**4)

# Euler's number calculation: e^2
print("e^2 =", math.exp(2))
```

Exercises

Exercise 1

Simplify the expression:

$$\frac{x^5 \cdot x^2}{x^3}$$

Solution — Exercise 1

Apply the rules of exponents:

1. Product Rule: $x^5 \cdot x^2 = x^{5+2} = x^7$
2. Quotient Rule: $\frac{x^7}{x^3} = x^{7-3} = x^4$

Logarithms

A **logarithm** is the mathematical operation that is the inverse of exponentiation. Logarithms are widely used in Data Science to scale skewed data (log-transform) and calculate errors in classification loss functions.

---

1. What Is a Logarithm?

The logarithm answers the question: *To what power must we raise the base to get a certain number?*

If:

$$b^y = x$$

Then:

$$\log_b(x) = y$$

For example:

$$\log_2(8) = 3 \quad \text{because} \quad 2^3 = 8$$

Intuition: The “Zero-Counting” Operation

To understand logarithms intuitively, think of the **base-10 logarithm** ($\log_{10}$) as a tool that **counts the number of zeros** in a number (when written as a power of 10):

- 10 has **1** zero $\implies \log_{10}(10) = 1$
- 100 has **2** zeros $\implies \log_{10}(100) = 2$
- $1,000$ has **3** zeros $\implies \log_{10}(1,000) = 3$
- $1,000,000$ (1 million) has **6** zeros $\implies \log_{10}(1,000,000) = 6$

Thus, the logarithm tells us the “scale” or “order of magnitude” of a number. If a number falls between 100 and $1,000$ (e.g., 500), its logarithm will be a decimal between 2 and 3 ($\log_{10}(500) \approx 2.699$).

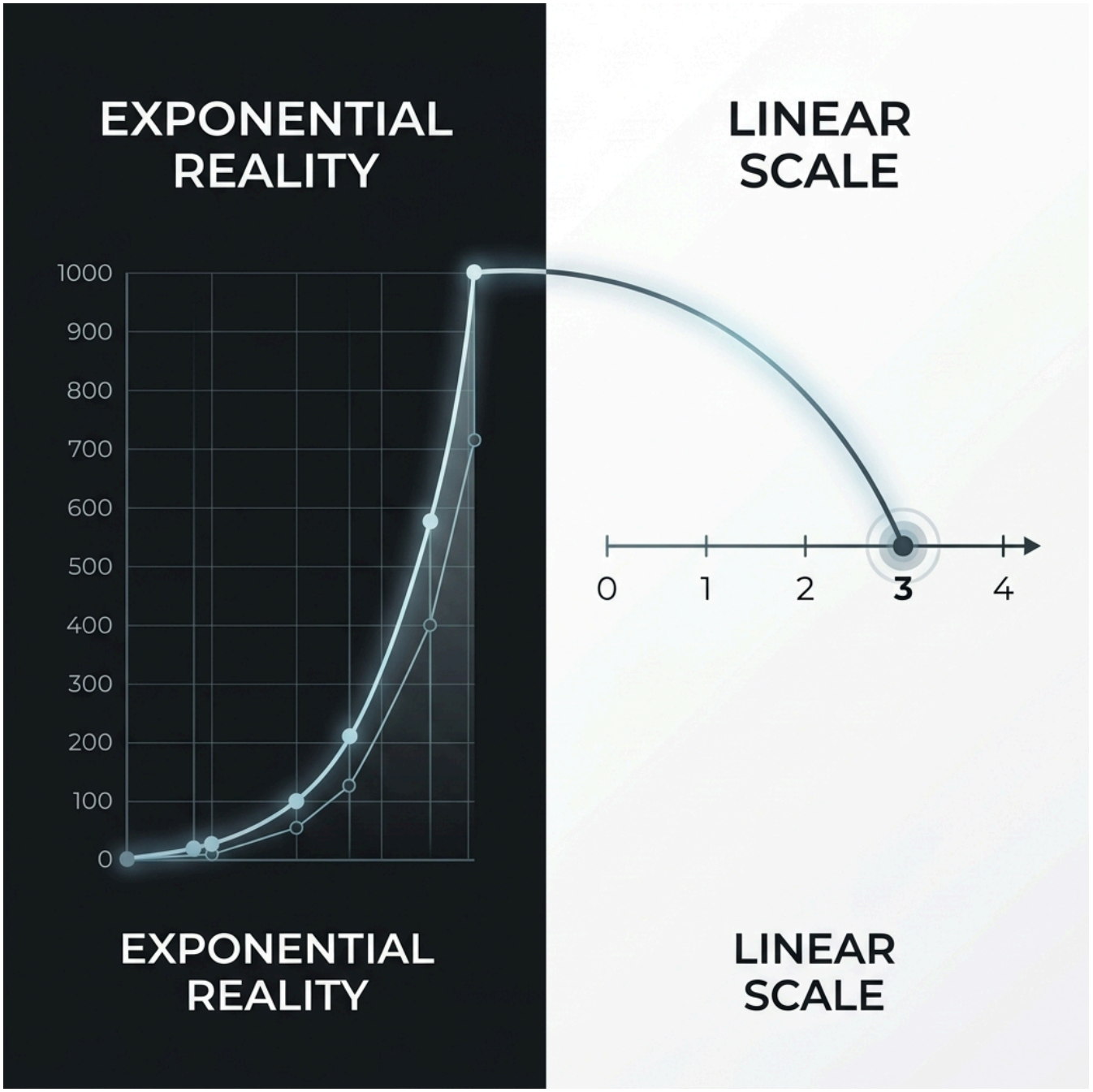


Fig. 6

Visualizing how the logarithm maps an “exponential reality” onto a manageable “linear scale”.

Why Do We Use Logarithms? Real-World Applications

In nature, human senses and physical phenomena span vast scales of magnitude. Linearly graphing numbers from 1 to $10,000,000,000$ on a single piece of paper is impossible. Logarithms compress these huge numbers into human-manageable scales.

Here is how logarithms are used across various scientific domains and real-world phenomena:

1. Earthquakes (The Richter Scale)

The Richter scale quantifies the energy released by earthquakes on a base-10 logarithmic scale:

$$\text{Magnitude } M = \log_{10}(A) - \log_{10}(A_0)$$

Where A is the maximum amplitude recorded by a seismograph.

- **Seismic Wave Amplitude:** Because of the $\log_{10}$ scale, each whole unit increase of $+1.0$ on the Richter scale means the ground shaking amplitude is **10 times greater**.
  - A magnitude 6.0 earthquake has **10 times** the amplitude of a 5.0 earthquake.
  - A magnitude 7.0 earthquake has $10 \times 10 = \mathbf{100 \text{ times}}$ the amplitude of a 5.0 earthquake.
- **Radiated Energy (E):** The actual energy released scales exponentially as $10^{1.5 \times M}$ (or $\approx 31.6^M$):

$$\frac{E_2}{E_1} = 10^{1.5 \times (M_2 - M_1)}$$

  - An increase of $+1.0$ in magnitude releases $10^{1.5} \approx \mathbf{31.6 \text{ times}}$ more energy.
  - An increase of $+2.0$ in magnitude (e.g., from 5.0 to 7.0) releases $10^{1.5 \times 2} = 10^3 = \mathbf{1,000 \text{ times}}$ more energy!

2. Sound Intensity (Decibel Scale - dB)

Human ears can process sound power ranging from a quiet whisper (10^{-12} W/m^2) to a jet engine takeoff (10^2 W/m^2) — a difference of **100,000,000,000,000 times** (10^{14}).

- We use decibels: $L = 10 \cdot \log_{10}\left(\frac{I}{I_0}\right)$
- Every addition of **10 dB** represents a **10-fold increase** in sound intensity.
- A 60 dB sound (normal conversation) is 1,000 times more intense than a 30 dB sound (whisper).

3. Acidity and Chemistry (pH Scale)

The pH scale measures hydrogen ion concentration $[H^+]$ in a liquid: $\text{pH} = -\log_{10}[H^+]$.

- A pH drop of 1 unit means the solution is **10 times more acidic**.
- Battery acid ($\text{pH} \approx 1$) is **100,000 times more acidic** than coffee ($\text{pH} \approx 5$).

4. Atmospheric Pressure, Hurricanes & Altitude

Atmospheric pressure decreases exponentially with increasing altitude ($P = P_0 e^{-h/H}$).

- Meteorologists use log-scale calculations to estimate altitude from pressure readings or track pressure drops during **hurricanes and tropical storms**.

- In hurricane tracking, central barometric pressure drops exponentially as wind speeds increase.

5. Stellar Brightness (Astronomy - Magnitude Scale)

Astronomers rank star brightness using a reverse logarithmic scale where a difference of 5 magnitudes corresponds to a factor of exactly **100 in light intensity** (each magnitude step is a factor of $100^{1/5} \approx 2.512$).

6. Viral Growth & Data Science (Log-Transform)

- **Epidemics & Social Trends (Logarithmic Scaling):** Viral phenomena—such as the spread of infectious diseases (e.g., COVID-19) or viral social media videos—begin with **exponential growth** (e.g., $y = 2^t$ or $y = e^{kt}$). On a standard linear axis, these curves shoot up almost vertically, making early trends impossible to analyze.

  By plotting data on a **logarithmic scale** ($\log(y)$ vs. t), exponential growth curves transform into **straight lines**:

  $$\ln(y) = \ln(e^{kt}) = kt$$

  The slope of this line represents the constant growth rate k. A bending of the line downward indicates that the growth rate is flattening out (turning point), making log plots essential for epidemiologists and growth analysts.

- **Handling Skewed Data in Machine Learning (Log Feature Scaling):** Real-world features like income levels, housing prices, or website traffic usually exhibit extreme **right-skewness** (heavy-tailed distributions) with values ranging over several orders of magnitude (e.g., $10,000 to $1,000,000,000).

  Applying a log transformation (such as $y = \log_{10}(x)$ or $y = \ln(1 + x)$) compresses these extreme scales:

  - $\$10,000 \implies \log_{10}(10,000) = 4$
  - $\$100,000 \implies \log_{10}(100,000) = 5$
  - $\$1,000,000,000 \implies \log_{10}(1,000,000,000) = 9$

  Why this matters in ML:

  1. It transforms highly skewed distributions into bell-shaped (Gaussian-like) distributions, satisfying key assumptions of algorithms like Linear Regression.
  2. It stabilizes feature variance and prevents extreme outliers from producing massive gradients, allowing gradient descent to converge faster and more reliably.

---

2. Euler's Number (e) and the Natural Logarithm ($\ln$)

What is Euler's Number (e)?

Euler's number, denoted as e, is a mathematical constant approximately equal to 2.71828. It is the base of natural growth and decay.

The value of e is defined as the limit of the compound interest formula as the number of compounding periods approaches infinity:

$$e = \lim_{n \to \infty} \left(1 + \frac{1}{n}\right)^n \approx 2.71828$$

The **Natural Logarithm ($\ln(x)$)** is simply a logarithm with base e:

$$\ln(x) = \log_e(x)$$

Because e and $\ln$ are inverse functions, they cancel each other out:

$$\ln(e^x) = x \quad \text{and} \quad e^{\ln(x)} = x$$

Real-World Application: Continuous Compound Interest

In finance, if an investment of principal P grows at an annual interest rate r compounded n times a year for t years, the final amount A is:

$$A = P\left(1 + \frac{r}{n}\right)^{nt}$$

If the interest is compounded **continuously** (meaning $n \to \infty$, compounding every microsecond), the formula simplifies using Euler's number e (often called the **PERT formula**):

$$A = Pe^{rt}$$

Deriving the "Rule of 72" in Investing

The **Rule of 72** is a quick shortcut used to estimate how many years (t) it takes to double your money at a given annual interest rate (r expressed as a percentage, e.g., $r\%$).

$$\text{Years to double} \approx \frac{72}{\text{Interest Rate}}$$

We can mathematically derive this rule using the natural logarithm:

1. To double our money ($A = 2P$), we substitute this into the continuous compound interest formula:

$$2P = Pe^{rt} \implies 2 = e^{rt}$$

2. Take the natural logarithm ($\ln$) of both sides to get rid of e:

$$\ln(2) = \ln(e^{rt})$$

$$\ln(2) = rt$$

3. Since $\ln(2) \approx 0.693$, we get:

$$rt \approx 0.693 \implies t \approx \frac{0.693}{r}$$

4. If we express the interest rate r as a percentage (e.g., $R = r \times 100$, so $R = 8$ for 8%), the formula becomes:

$$t \approx \frac{69.3}{R}$$

5. While 69.3 is the exact mathematical numerator for continuous compounding, 72 is used in finance because it is highly divisible by many common interest rates (such as 2, 3, 4, 6, 8, 9, 12), making mental math extremely easy. For example, at an 8% interest rate, money will double in approximately $72/8 = 9$ years (the exact value is about 8.66 years).

---

3. Common Properties of Logarithms

Logarithms possess unique mathematical properties that simplify complex operations:

- **Product Rule**: $\log_b(x \cdot y) = \log_b(x) + \log_b(y)$
- **Quotient Rule**: $\log_b(\frac{x}{y}) = \log_b(x) - \log_b(y)$
- **Power Rule**: $\log_b(x^k) = k \cdot \log_b(x)$

Two Crucial Bases:

1. **Natural Logarithm ($\ln(x)$)**: Logarithm with base $e \approx 2.71828$.
2. **Common Logarithm ($\log_{10}(x)$)**: Logarithm with base 10.

---

4. Python Implementation

We can compute logarithms in Python using the `math` and `numpy` libraries:

```python
import math
import numpy as np

# Natural logarithm: ln(10)
print("ln(10) =", math.log(10)) # math.log defaults to base e

# Base 10 logarithm
print("log10(100) =", math.log10(100))

# NumPy log-transform on an array
data = np.array([1, 10, 100, 1000])
log_data = np.log10(data)
print("Log-transformed data:", log_data)
```

Exercises

Exercise 1

Solve for x:

$$\log_3(x) = 4$$

Exercise 2

Expand the expression using logarithm properties:

$$\ln\left(\frac{x^3}{y}\right)$$

Solution — Exercise 1

Convert to exponential form:

$$x = 3^4 = 81$$

Solution — Exercise 2

Apply the quotient and power rules:

$$\ln\left(\frac{x^3}{y}\right) = \ln(x^3) - \ln(y) = 3\ln(x) - \ln(y)$$

Limits

In mathematics, a **limit** is the foundational building block of Calculus. It describes what value a function *approaches* as its input variable gets closer and closer to a specific point, regardless of whether the function is actually defined at that exact point.

---

1. Why Do We Need Limits in Calculus?

Limits serve as the mathematical "bridge" that allows us to handle division by zero and infinitesimally small quantities, paving the way for the creation of Calculus.

The $\frac{0}{0}$ Indeterminate Form

In classical algebra and geometry, division by zero is strictly prohibited. However, the two foundational problems that led to the development of Calculus directly encounter this issue:

- **Instantaneous Velocity (Leads to Derivatives):** If you travel $100\ \text{km}$ in $2\ \text{hours}$, your average velocity is $v = \frac{\Delta s}{\Delta t} = \frac{100}{2} = 50\ \text{km/h}$. But what if you want to know your exact speed at a single instantaneous second (when $\Delta t = 0$)? The classical formula yields $\frac{\Delta s}{\Delta t} = \frac{0}{0}$, which is algebraically undefined.
- **Tangent Line of a Curve:** To find the slope of a line passing through two points, we use $m = \frac{y_2 - y_1}{x_2 - x_1}$. But a tangent line only touches a curve at a single point. When the second point merges with the first point, the distance becomes 0, leading to a slope of $m = \frac{0}{0}$.

Limits resolve this by allowing us to calculate behavior *infinitely close* to 0 without ever evaluating at 0 directly:

$$\lim_{\Delta t \to 0} \frac{\Delta s}{\Delta t}$$

Note

Distinguishing $\frac{a}{0}$ vs. $\frac{0}{0}$ (The Key to Calculus):

- **The $\frac{a}{0}$ Form (where $a \neq 0$, e.g., $\frac{1}{x}$):** If we let x approach 0 on the number line:
  - Approaching from the left ($-1 \to 0$): We divide by extremely small negative values (e.g., $\frac{1}{-0.000001} = -1,000,000$), so the value shoots to **negative infinity ($-\infty$)**.
  - Approaching from the right ($1 \to 0$): We divide by extremely small positive values (e.g., $\frac{1}{0.000001} = 1,000,000$), so the value shoots to **positive infinity ($+\infty$)**.

  Because the two sides head in opposite directions to different infinities ($\pm\infty$), the limit **does not exist**, and we observe a **vertical asymptote**.
- **The $\frac{0}{0}$ Form (Indeterminate Form):** Unlike the $\frac{a}{0}$ case, here the numerator is also shrinking to zero, which "counters" the division-by-zero explosion. The result depends entirely on how fast the numerator and denominator approach zero relative to each other:
  - If $f(x) = \frac{x}{x}$, the ratio is always 1 for any $x \neq 0$. The limit is 1.
  - If $g(x) = \frac{2x}{x}$, the ratio is always 2 for any $x \neq 0$. The limit is 2.
  - If $h(x) = \frac{x^2}{x}$, the ratio simplifies to x. As $x \to 0$, the limit is 0.

  Since $\frac{0}{0}$ can resolve to $1, 2, 0$, or any other real number, it is called **indeterminate** (undetermined) and is the only form capable of representing rates of change and derivatives.

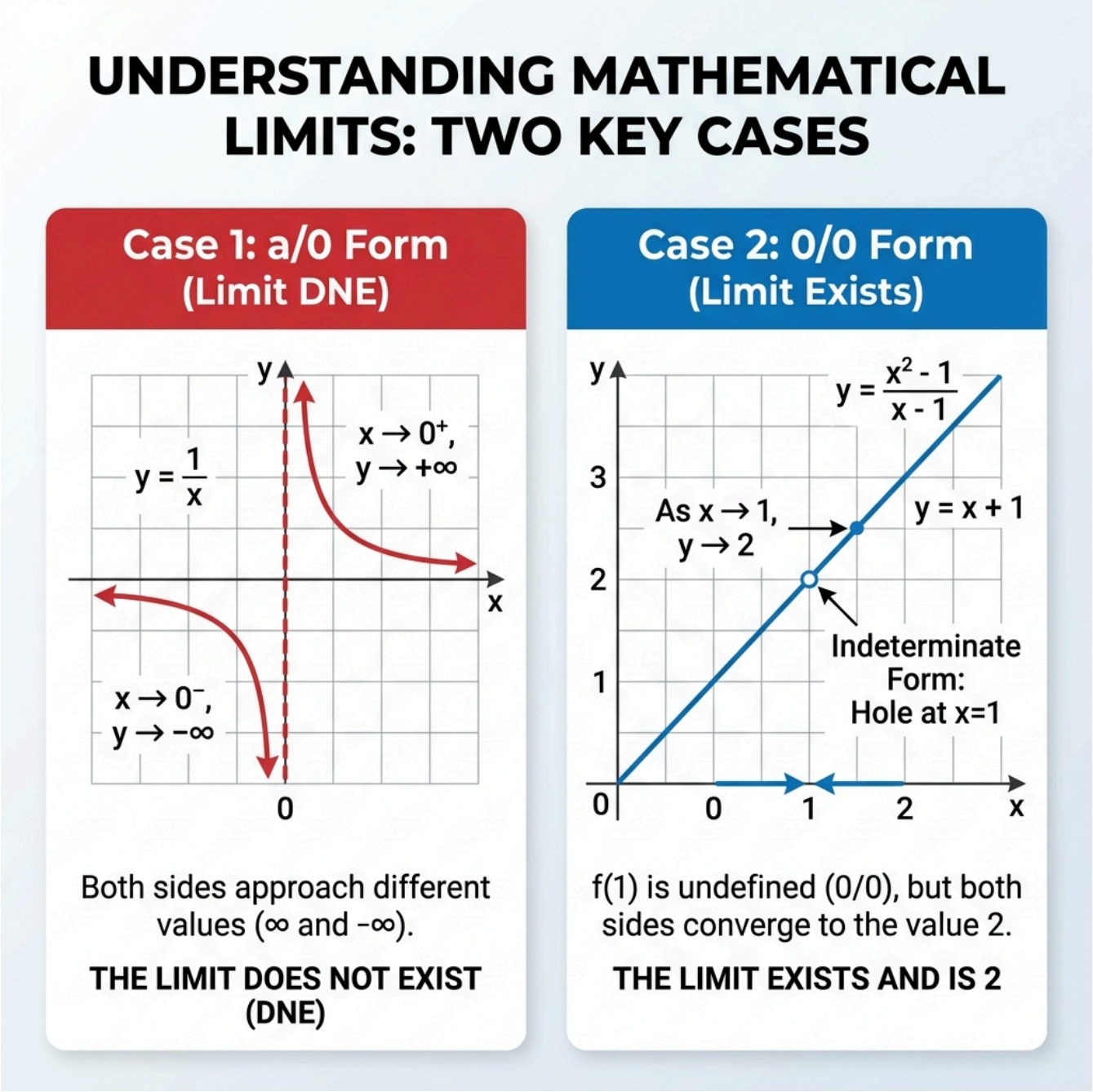


Fig. 7

Comparison between a/0 (Limit does not exist) and 0/0 (Limit exists) forms.

The Foundation of Derivatives and Integrals

Both core operations of Calculus are defined entirely through limits:

- **Derivatives:** The limit of the rate of change as the interval h approaches 0:

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

- **Integrals:** The limit of Riemann sums as we divide the area under a curve into an infinite number of infinitely thin rectangles ($n \to \infty$):

$$\int_a^b f(x)dx = \lim_{n\to\infty} \sum_{i=1}^{n} f(x_i^*)\Delta x$$

The Tangent Line Problem: A Conceptual Preview

The most famous geometric illustration of why we need limits is the **Tangent Line Problem**:

- **Secant Line:** In algebra, we can easily find the slope between two separate points on a curve. This is called a *secant line*.
- **The Tangent Obstacle:** If we want to find the slope at exactly *one* single point (the *tangent line*), the points merge, the distance becomes zero, and our slope formula results in an undefined division by zero ($\frac{0}{0}$).
- **The Limit Solution:** By taking the limit as the distance between the two points approaches zero ($\Delta x \to 0$), the secant line gradually rotates and aligns perfectly with the tangent line. This slope is what we call the **derivative**.

*(We will explore the complete mathematical definition of the derivative and its applications in the next chapter: **Calculus & Derivatives**).*

---

2. The Intuition of a Limit

Consider the function:

$$f(x) = \frac{x^2 - 1}{x - 1}$$

If we try to evaluate this function at $x = 1$, we get a division by zero:

$$f(1) = \frac{1^2 - 1}{1 - 1} = \frac{0}{0} \quad \text{(Undefined)}$$

However, we can look at the values of $f(x)$ as we choose inputs extremely close to 1:

x (approaching from left)	$f(x)$	x (approaching from right)	$f(x)$
0.9	1.9	1.1	2.1
0.99	1.99	1.01	2.01
0.9999	1.9999	1.0001	2.0001

Even though $f(1)$ is undefined, the output values of $f(x)$ are heading directly toward 2 as x gets closer to 1. Mathematically, we express this as:

$$\lim_{x \to 1} \frac{x^2 - 1}{x - 1} = 2$$

Using algebra, we can simplify the expression:

$$\lim_{x \to 1} \frac{(x-1)(x+1)}{x-1} = \lim_{x \to 1} (x+1) = 1 + 1 = 2$$

> **Note**
>
> **L'Hôpital's Rule Preview:** When encountering indeterminate forms like $\frac{0}{0}$ or $\frac{\pm\infty}{\pm\infty}$, instead of algebraic simplification, we can use **L'Hôpital's Rule** (which utilizes derivatives):
>
> $$\lim_{x \to a} \frac{f(x)}{g(x)} = \lim_{x \to a} \frac{f'(x)}{g'(x)}$$

---

3. Formal Definition of a Limit (ϵ-δ)

To make limits mathematically rigorous, the Karl Weierstrass formal ϵ-δ **(epsilon-delta) definition** of a limit is used:

$$\lim_{x \to a} f(x) = L$$

For every real number $\epsilon > 0$, there exists a real number $\delta > 0$ such that:

$$\text{if } 0 < |x - a| < \delta, \quad \text{then } |f(x) - L| < \epsilon$$

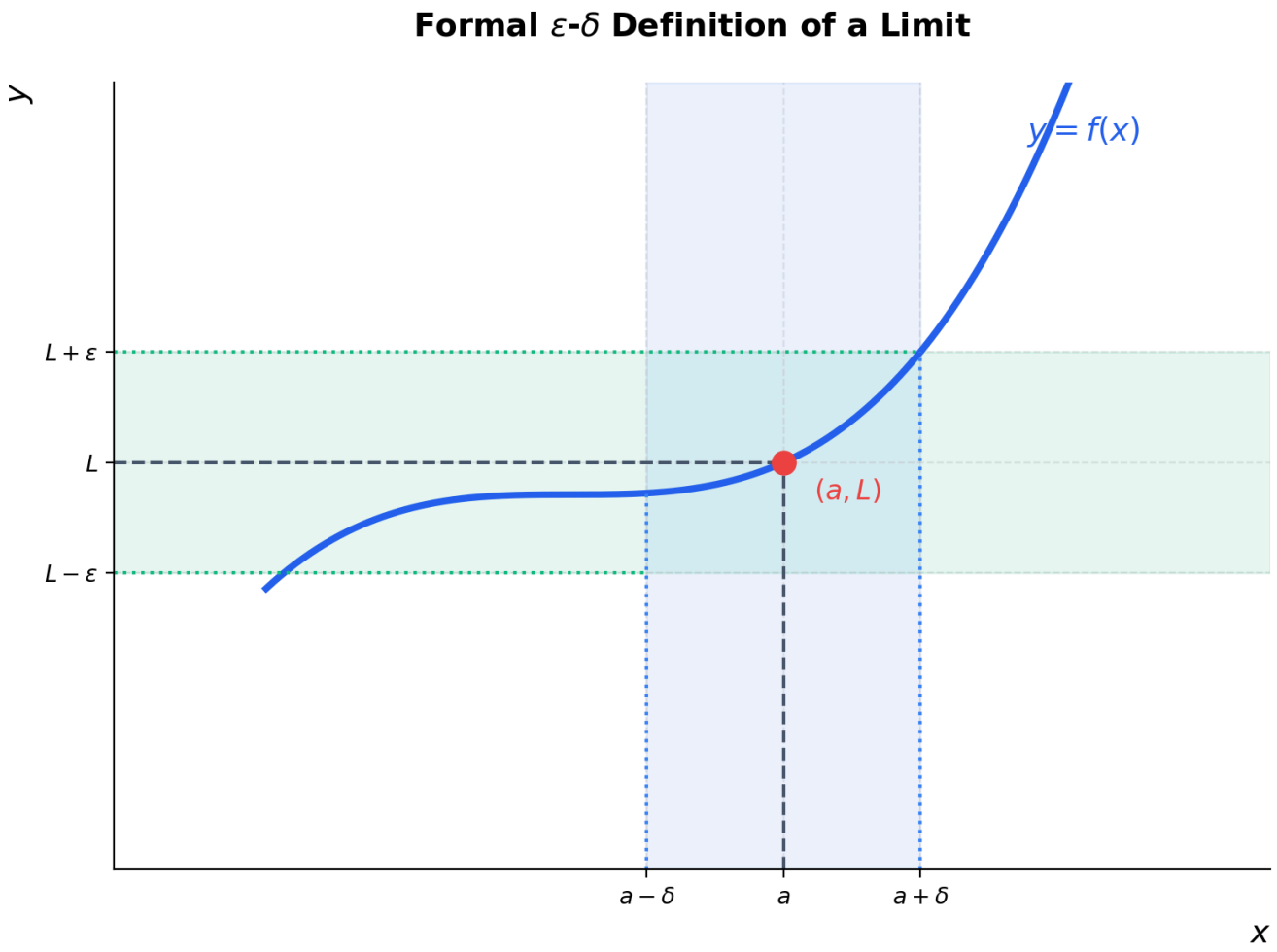


Fig. 8

Geometrical illustration of the epsilon-delta limit definition.

Understanding the Epsilon-Delta Definition

- ϵ **(Epsilon)** represents the target window of error on the vertical y-axis (output space).
- δ **(Delta)** represents the search window on the horizontal x-axis (input space).
- The definition states: If you want the output $f(x)$ to be within an extremely small distance ϵ from L, there must exist a matching boundary size δ such that any input x within that distance of a (excluding a itself) lands inside the target output window.

---

4. One-Sided Limits and Existence

A limit only exists if the function approaches the same value from both directions.

- **Left-Hand Limit:** The value $f(x)$ approaches as x approaches a from values less than a:

$$\lim_{x \to a^-} f(x)$$

- **Right-Hand Limit:** The value $f(x)$ approaches as x approaches a from values greater than a:

$$\lim_{x \to a^+} f(x)$$

Existence Criterion

The limit $\lim_{x \to a} f(x)$ exists if and only if both one-sided limits exist and are equal:

$$\lim_{x \to a^-} f(x) = \lim_{x \to a^+} f(x) = L$$

If the left-hand and right-hand limits are not equal (such as at a step discontinuity), the limit **does not exist (DNE)**.

---

5. Basic Properties of Limits

Given that $\lim_{x \to a} f(x) = L$ and $\lim_{x \to a} g(x) = M$, the following algebraic properties hold:

- **Sum Rule:** $\lim_{x \to a}[f(x) + g(x)] = L + M$
- **Product Rule:** $\lim_{x \to a}[f(x) \cdot g(x)] = L \cdot M$
- **Quotient Rule:** $\lim_{x \to a} \frac{f(x)}{g(x)} = \frac{L}{M}$ (provided $M \neq 0$)
- **Constant Multiple Rule:** $\lim_{x \to a}[c \cdot f(x)] = c \cdot L$ (where c is a constant)
- **Power Rule:** $\lim_{x \to a}[f(x)]^n = L^n$

---

6. Continuity

In data science, we prioritize **continuous functions**. Intuitively, a function is continuous if you can draw its graph without lifting your pen from the paper.

Formally, a function $f(x)$ is **continuous at a point** $x = a$ if and only if three conditions are met:

1. $f(a)$ is defined (the point exists in the domain).
2. $\lim_{x \to a} f(x)$ exists.
3. The limit equals the function's value:

$$\lim_{x \to a} f(x) = f(a)$$

If any of these conditions fail, the function has a discontinuity at $x = a$.

- **Machine Learning Connection:** Continuity (along with smoothness and differentiability) is a critical requirement for gradient-based optimization algorithms (like Gradient Descent in Neural Networks). If a Loss Function is discontinuous, gradients cannot be computed, preventing the model from updating its weights properly.

---

7. Symbolic Limit Calculations in Python

We can calculate symbolic limits in Python using the `sympy` library:

```
import sympy as sp

x = sp.symbols('x')

# 1. Undefined algebraic expression limit
f1 = (x**2 - 1) / (x - 1)
limit1 = sp.limit(f1, x, 1)
print("Limit of (x^2 - 1)/(x - 1) as x->1 :", limit1)  # 2

# 2. Limit at infinity
f2 = 1 / x
limit2 = sp.limit(f2, x, sp.oo)
print("Limit of 1/x as x->oo :", limit2)  # 0
```

Calculus & Derivatives

Calculus is the mathematical framework that allows us to understand change. In Machine Learning, calculus provides the engine (such as derivatives and the chain rule) that lets models evaluate and update their parameters.

Here we cover the essential calculus tools used to analyze functions of single and multiple variables.

---

1. What Is a Derivative?

The **derivative** of a function measures how the output value changes relative to a change in its input. Geometrically, the derivative is the **slope** of the tangent line to the graph of the function at a given point.

The Tangent Line Problem: Transition from Secant to Tangent

To understand how derivatives work, we look at the transition from average rate of change to instantaneous rate of change:

1. **Slope of a Secant Line (Average Rate of Change):** Suppose we want to find the slope of a curve between two distinct points $P(x, f(x))$ and $Q(x + \Delta x, f(x + \Delta x))$. Connecting these two points gives a **secant line**. The slope of this secant line represents the average rate of change:

$$m_{\text{secant}} = \frac{\Delta y}{\Delta x} = \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

2. **The Tangent Obstacle (Instantaneous Speed):** If we want to know the slope at exactly *one* single point P (the **tangent line**), we cannot simply set the second point Q directly on top of P. Doing so makes the distance $\Delta x = 0$, causing the slope formula to collapse into the undefined division:

$$m = \frac{0}{0}$$

3. **The Limit Solution:** Instead of forcing Q to merge with P immediately, we use a **limit** to slide Q along the curve *infinitely close* to P. As the distance Δx approaches 0 ($\Delta x \to 0$), the secant line rotates and converges into the tangent line at point P.

The slope of this tangent line is the **derivative** of the function at x, defined as:

$$m_{\text{tangent}} = f'(x) = \lim_{\Delta x \to 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

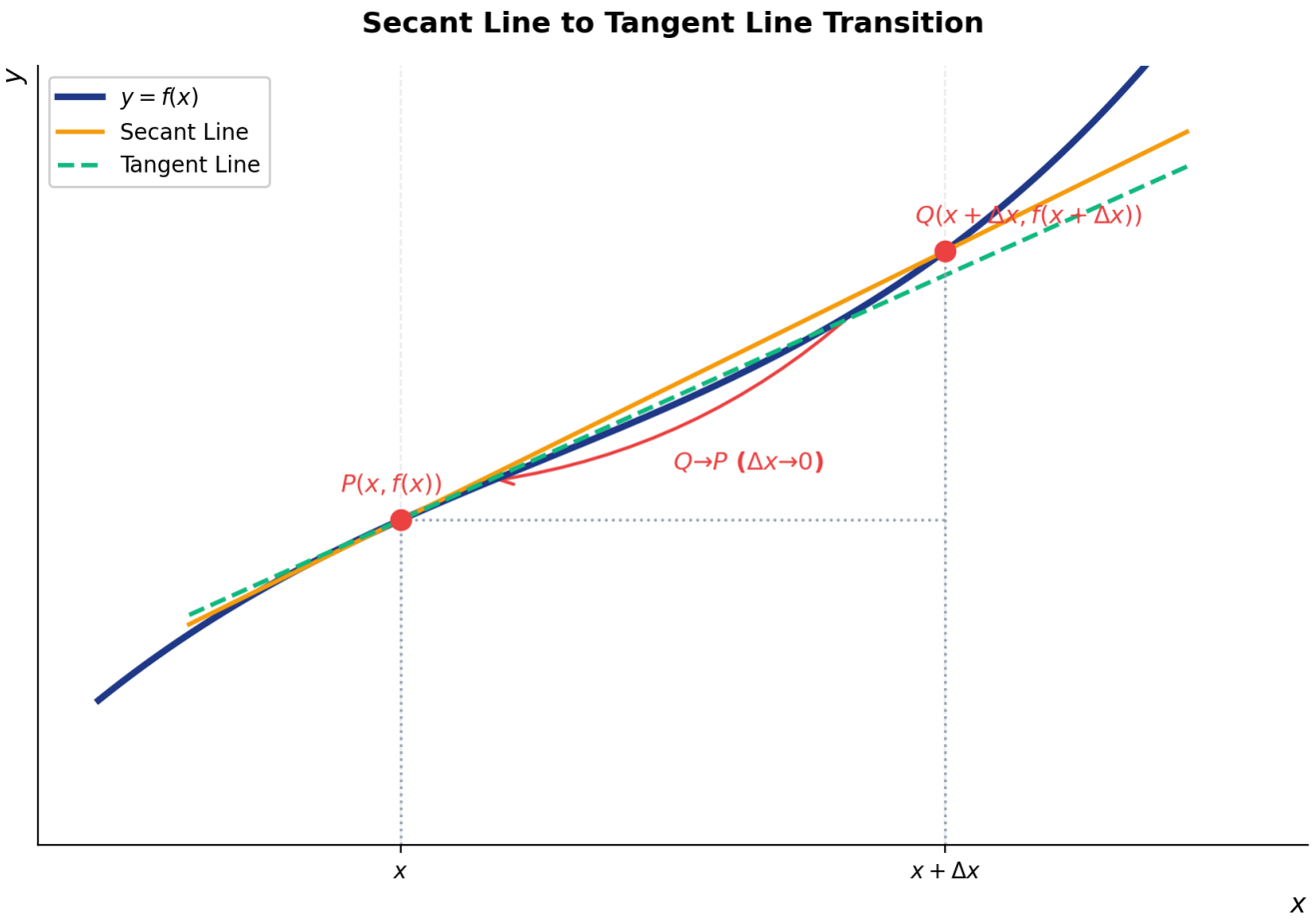


Fig. 9

Geometrical transition of a secant line into a tangent line as $\Delta x \to 0$

.

---

2. Basic Rules of Differentiation

Calculating derivatives using limits can be tedious. Instead, we use standard differentiation rules:

- **Power Rule**: $\frac{d}{dx}(x^n) = nx^{n-1}$
- **Constant Rule**: $\frac{d}{dx}(c) = 0$
- **Sum Rule**: $\frac{d}{dx}(f + g) = f' + g'$
- **Product Rule**: $\frac{d}{dx}(f \cdot g) = f'g + fg'$
- **Quotient Rule**: $\frac{d}{dx}(\frac{f}{g}) = \frac{f'g - fg'}{g^2}$

---

3. The Chain Rule

The **Chain Rule** is a formula for calculating the derivative of the composition of two or more functions: $f(g(x))$.

If $y = f(u)$ and $u = g(x)$, then:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

Why it matters in Deep Learning

In neural networks, the loss (error) is a nested composition of functions across layers. The Chain Rule is the mathematical backbone of **Backpropagation**, allowing the network to calculate how changes in early weights affect the final output error.

---

4. Partial Derivatives & Gradients

When a function has multiple variables, e.g., $f(x, y)$, we use **partial derivatives**. A partial derivative measures the rate of change with respect to *one* variable while keeping all other variables constant.

- Partial derivative with respect to x: $\frac{\partial f}{\partial x}$
- Partial derivative with respect to y: $\frac{\partial f}{\partial y}$

The Gradient Vector

The **gradient** is a vector containing all the partial derivatives of a multivariable function. It is denoted by the symbol ∇ (nabla):

$$\nabla f(x, y) = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

Geometrical Meaning: The gradient vector points in the direction of the **steepest ascent** (greatest increase) of the function at that point.

- **Machine Learning Note:** Since our goal in Machine Learning is usually to *minimize* the Loss Function (find the lowest point of error), we move in the **opposite** direction of the gradient ($-\nabla f$). This core optimization technique is called **Gradient Descent**:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \alpha \nabla f(\mathbf{w})$$

(where α is the learning rate that controls the size of each step).

---

5. Python Implementation (SymPy)

We can compute exact symbolic derivatives and gradients easily in Python:

```python
import sympy as sp

x, y = sp.symbols('x y')
f = x**2 * y + 3*x * sp.sin(y)

# 1. Single derivative
df_dx = sp.diff(f, x)
print("Partial derivative w.r.t x :", df_dx)

# 2. Derivative w.r.t y
df_dy = sp.diff(f, y)
print("Partial derivative w.r.t y :", df_dy)
```

Exercises

Exercise 1

Find the derivative of $f(x) = 5x^3 - 2x^2 + 7$.

Exercise 2

Apply the Chain Rule to find the derivative of:

$$f(x) = (3x^2 + 1)^4$$

Exercise 3

Find the gradient vector $\nabla f(x, y)$ of:

$$f(x, y) = 3x^2y^3 + 2y$$

Solution — Exercise 1

Using the Power and Sum rules:

$$f'(x) = 5(3x^2) - 2(2x) + 0 = 15x^2 - 4x$$

Solution — Exercise 2

Let $u = 3x^2 + 1$ (inside function) and $y = u^4$ (outside function).

- $\frac{dy}{du} = 4u^3$
- $\frac{du}{dx} = 6x$

Applying the Chain Rule:

$$\frac{dy}{dx} = 4(3x^2 + 1)^3 \cdot 6x = 24x(3x^2 + 1)^3$$

Solution — Exercise 3

Compute partial derivatives:

- $\frac{\partial f}{\partial x} = 6xy^3$
- $\frac{\partial f}{\partial y} = 9x^2y^2 + 2$

The gradient vector is:

$$\nabla f(x, y) = \begin{bmatrix} 6xy^3 \\ 9x^2y^2 + 2 \end{bmatrix}$$

Integration

Integration is one of the two fundamental operations of calculus (alongside differentiation). While a **derivative** tells us the *rate of change* of a function, an **integral** tells us the *accumulated area* under a curve.

In data science, integration is essential for:

- Computing probabilities from probability density functions (PDFs)
- Understanding the Normal Distribution's CDF
- Calculating expected values in statistics

What Is an Integral?

Imagine you are driving a car. Your speedometer shows your *speed* at any given moment (the derivative). If you want to know **how far you've traveled**, you need to *accumulate* that speed over time — that's integration.

Formally, the **definite integral** of a function $f(x)$ from a to b is written as:

$$\int_a^b f(x)\,dx$$

This represents the **area** between the curve $f(x)$ and the x-axis, from $x = a$ to $x = b$.

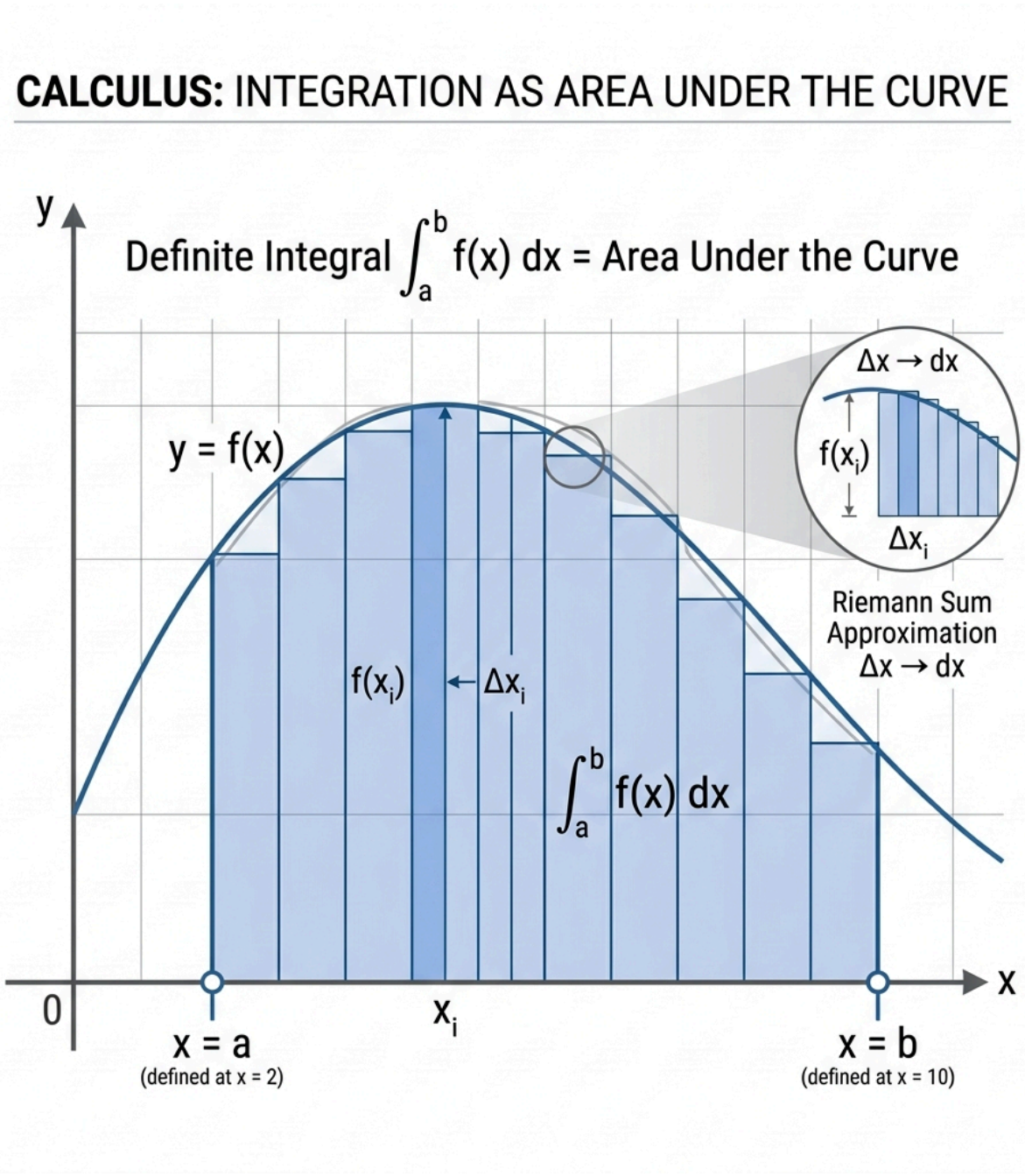


Fig. 10

Definite integration as area under a curve.

Note

If $f(x)$ is negative in some region, the area there is counted as **negative**. The integral gives the *net signed area*.

Riemann Sums — Approximating the Area

Before numerical integration libraries existed, mathematicians approximated the area under a curve by slicing the region into n thin vertical rectangles and summing their areas. This fundamental concept is known as a **Riemann Sum**.

Core Intuition

Suppose we want to find the area under a function $f(x)$ over the interval $[a, b]$.

We divide the interval $[a, b]$ into n equal sub-intervals, each with width:

$$\Delta x = \frac{b - a}{n}$$

For each sub-interval i, we select a sample point x_i^* and construct a rectangle of height $f(x_i^*)$ and width Δx.

The total approximated area is the sum of these rectangle areas:

$$\text{Area} \approx \sum_{i=1}^{n} f(x_i^*) \cdot \Delta x$$

As the number of slices approaches infinity ($n \to \infty$), the width $\Delta x \to 0$, and the Riemann Sum converges to the exact **Definite Integral**:

$$\int_a^b f(x)\, dx = \lim_{n \to \infty} \sum_{i=1}^{n} f(x_i^*) \cdot \Delta x$$

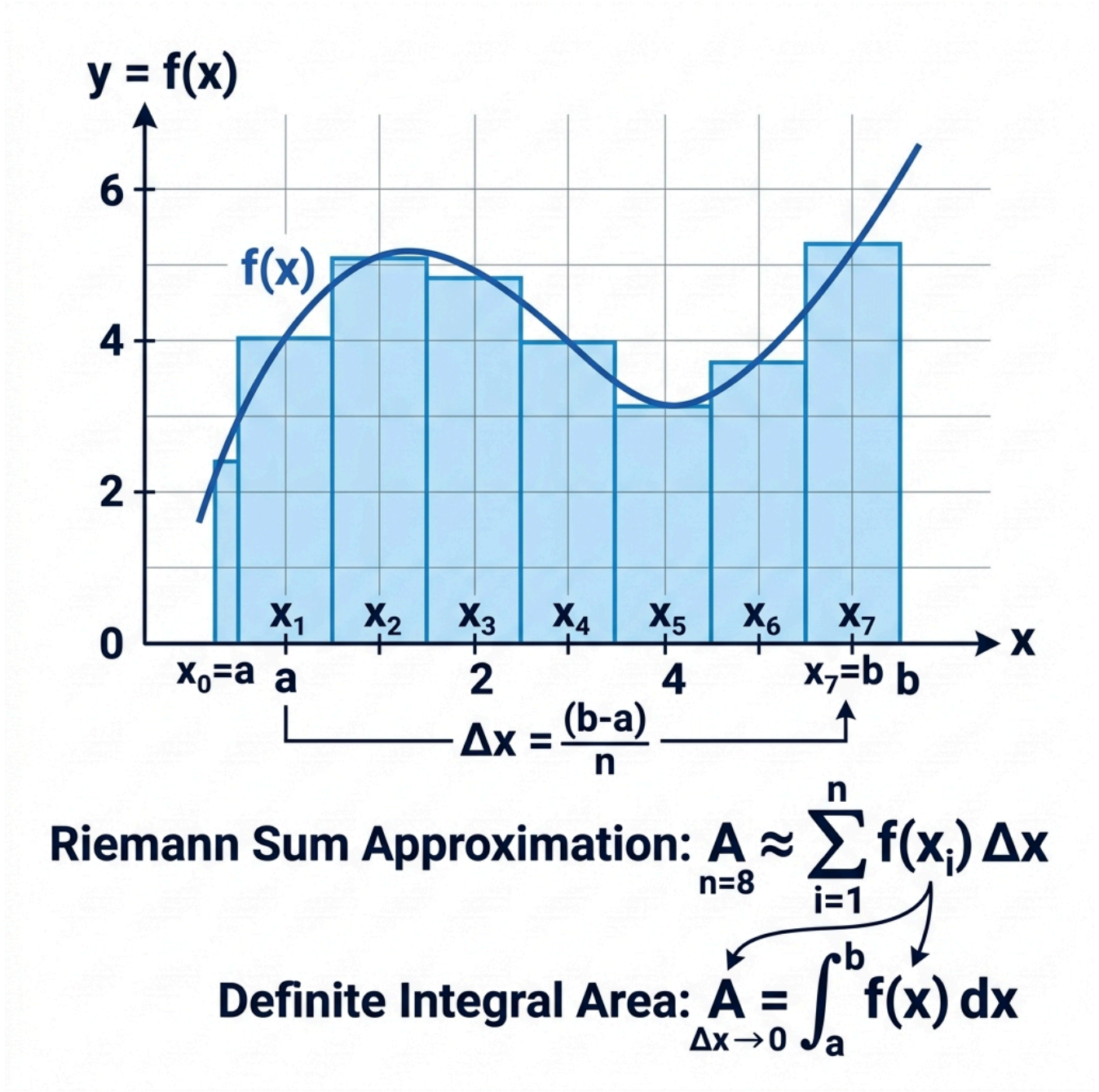


Fig. 11

Riemann sum approximation converging to definite integral area.

Three Types of Riemann Sums

Depending on where we choose the sample point x_i^* within each sub-interval, we get three common types of Riemann Sums:

Type	Sample Point Selection (x_i^*)	Behavior for Increasing Functions
Left Riemann Sum	Left endpoint of each slice	Under-estimates area
Right Riemann Sum	Right endpoint of each slice	Over-estimates area
Midpoint Riemann Sum	Midpoint of each slice	Most balanced & accurate approximation

---

Python Implementation

```python
import numpy as np
import matplotlib.pyplot as plt

# Define the function f(x) = x^2
def f(x):
    return x ** 2

# Parameters
a = 0       # lower bound
b = 1       # upper bound
n = 10      # number of rectangles

# Width of each rectangle
dx = (b - a) / n

# Generate x positions for each rectangle
x_left = np.linspace(a, b - dx, n)    # left edges
x_right = np.linspace(a + dx, b, n)   # right edges
x_mid = x_left + dx / 2               # midpoints

# Compute Riemann Sums
left_sum  = np.sum(f(x_left)  * dx)
right_sum = np.sum(f(x_right) * dx)
mid_sum   = np.sum(f(x_mid)   * dx)

print(f"Left Riemann Sum  (n={n}): {left_sum:.6f}")
print(f"Right Riemann Sum (n={n}): {right_sum:.6f}")
print(f"Midpoint Sum      (n={n}): {mid_sum:.6f}")
print(f"Exact answer             : {1/3:.6f}")   # ∫x² dx from 0 to 1 = 1/3
```

Output:

```
Left Riemann Sum  (n=10): 0.285000
Right Riemann Sum (n=10): 0.385000
Midpoint Sum      (n=10): 0.332500
Exact answer             : 0.333333
```

Visualizing the Rectangles

```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return x ** 2

a, b, n = 0, 1, 10
dx = (b - a) / n
x_left = np.linspace(a, b - dx, n)

# Smooth curve
x_curve = np.linspace(a, b, 300)

fig, ax = plt.subplots(figsize=(8, 5))

# Draw rectangles (Left Riemann Sum)
ax.bar(x_left, f(x_left), width=dx, align='edge',
       alpha=0.4, color='steelblue', edgecolor='black', label='Rectangles (Left Sum)')

# Draw the actual curve
ax.plot(x_curve, f(x_curve), 'r-', linewidth=2.5, label=r'$f(x) = x^2$')

ax.set_xlabel('x', fontsize=12)
ax.set_ylabel('f(x)', fontsize=12)
ax.set_title(f'Left Riemann Sum with n={n} rectangles', fontsize=14)
ax.legend()
plt.tight_layout()
plt.savefig('../../images/riemann_sum.png', dpi=150)
plt.show()
```

Increasing Accuracy with More Rectangles

```
import numpy as np

def f(x):
    return x ** 2

a, b = 0, 1
exact = 1 / 3   # true value of ∫x² dx from 0 to 1

print(f"{'n':>8} | {'Left Sum':>12} | {'Right Sum':>12} | {'Midpoint':>12} | {'Error (mid)':>12}")
print("-" * 65)

for n in [5, 10, 50, 100, 1000, 10000]:
    dx = (b - a) / n
    x_left  = np.linspace(a, b - dx, n)
    x_right = np.linspace(a + dx, b, n)
    x_mid   = x_left + dx / 2

    left  = np.sum(f(x_left)  * dx)
    right = np.sum(f(x_right) * dx)
    mid   = np.sum(f(x_mid)   * dx)
    error = abs(mid - exact)

    print(f"{n:>8} | {left:>12.8f} | {right:>12.8f} | {mid:>12.8f} | {error:>12.2e}")
```

Output:

```
       n |     Left Sum |    Right Sum |     Midpoint |  Error (mid)
-------------------------------------------------------------------
       5 | 0.24000000   | 0.44000000   | 0.33500000   |  1.67e-03
      10 | 0.28500000   | 0.38500000   | 0.33250000   |  8.33e-04
      50 | 0.32340000   | 0.34340000   | 0.33330000   |  3.33e-05
     100 | 0.32835000   | 0.33835000   | 0.33332500   |  8.33e-06
    1000 | 0.33283350   | 0.33383350   | 0.33333333   |  8.33e-09
   10000 | 0.33328334   | 0.33338334   | 0.33333333   |  8.33e-11
```

The midpoint rule converges the fastest — with $n = 1000$ it's already accurate to 8 decimal places.

Computing Integrals with SciPy

Instead of approximating manually, we can use `scipy.integrate.quad` for accurate numerical integration.

Basic Usage of `quad`

```python
from scipy.integrate import quad

# Define the function
def f(x):
    return x ** 2

# Integrate f(x) from 0 to 1
result, error = quad(f, 0, 1)

print(f"Integral result : {result:.10f}")
print(f"Estimated error : {error:.2e}")
print(f"Exact value     : {1/3:.10f}")
```

Output:

```
Integral result : 0.3333333333
Estimated error : 3.70e-15
Exact value     : 0.3333333333
```

Integrating Complex Functions

```python
from scipy.integrate import quad
import numpy as np

# Example 1: ∫ sin(x) dx from 0 to π
result, _ = quad(np.sin, 0, np.pi)
print(f"∫ sin(x) dx from 0 to π = {result:.6f}")   # Expected: 2.0

# Example 2: ∫ e^(-x²) dx from -∞ to +∞  (Gaussian integral)
result, _ = quad(lambda x: np.exp(-x**2), -np.inf, np.inf)
print(f"∫ e^(-x²) dx from -∞ to +∞ = {result:.6f}")  # Expected: √π ≈ 1.7725
print(f"√π = {np.sqrt(np.pi):.6f}")
```

Output:

```
∫ sin(x) dx from 0 to π = 2.000000
∫ e^(-x²) dx from -∞ to +∞ = 1.772454
√π = 1.772454
```

Symbolic Integration with SymPy

SymPy can compute integrals **symbolically** (exact answers, not approximations).

Indefinite Integrals (Antiderivatives)

```python
from sympy import symbols, integrate, sin, exp, oo, sqrt, pi

x = symbols('x')

# Indefinite integral of x²
expr = x**2
antideriv = integrate(expr, x)
print(f"∫ x² dx = {antideriv} + C")

# Indefinite integral of sin(x)
print(f"∫ sin(x) dx = {integrate(sin(x), x)} + C")
```

Output:

```
∫ x² dx = x**3/3 + C
∫ sin(x) dx = -cos(x) + C
```

Definite Integrals

```python
from sympy import symbols, integrate, sin, Rational

x = symbols('x')

# Definite integral of x² from 0 to 1
result = integrate(x**2, (x, 0, 1))
print(f"∫₀¹ x² dx = {result}")            # 1/3

# Definite integral of sin(x) from 0 to π
result2 = integrate(sin(x), (x, 0, Rational(355, 113)))   # ≈ π
print(f"∫₀^π sin(x) dx ≈ {float(result2):.6f}")
```

Real-World Application: Probability from a PDF

In statistics, many probability distributions are defined by a **Probability Density Function (PDF)**. The probability that a value falls between a and b is:

$$P(a \leq X \leq b) = \int_a^b f(x)\,dx$$

Example: Normal Distribution

The PDF of the standard Normal distribution is:

$$f(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$$

Let's compute the probability that a value falls within **1 standard deviation** of the mean (i.e., between -1 and 1):

```
from scipy.integrate import quad
from scipy.stats import norm
import numpy as np

# Standard normal PDF
def normal_pdf(x, mu=0, sigma=1):
    return (1 / (sigma * np.sqrt(2 * np.pi))) * np.exp(-0.5 * ((x - mu) / sigma) ** 2)

# Compute P(-1 ≤ X ≤ 1) by integration
prob_1sigma, _ = quad(normal_pdf, -1, 1)
print(f"P(-1 ≤ X ≤ 1) = {prob_1sigma:.4f} ({prob_1sigma*100:.2f}%)")

# P(-2 ≤ X ≤ 2)
prob_2sigma, _ = quad(normal_pdf, -2, 2)
print(f"P(-2 ≤ X ≤ 2) = {prob_2sigma:.4f} ({prob_2sigma*100:.2f}%)")

# P(-3 ≤ X ≤ 3)
prob_3sigma, _ = quad(normal_pdf, -3, 3)
print(f"P(-3 ≤ X ≤ 3) = {prob_3sigma:.4f} ({prob_3sigma*100:.2f}%)")

# Verify using scipy's built-in CDF
print("\n--- Verification using scipy.stats.norm.cdf ---")
print(f"P(-1 ≤ X ≤ 1) = {norm.cdf(1) - norm.cdf(-1):.4f}")
print(f"P(-2 ≤ X ≤ 2) = {norm.cdf(2) - norm.cdf(-2):.4f}")
print(f"P(-3 ≤ X ≤ 3) = {norm.cdf(3) - norm.cdf(-3):.4f}")
```

Output:

```
P(-1 ≤ X ≤ 1) = 0.6827 (68.27%)
P(-2 ≤ X ≤ 2) = 0.9545 (95.45%)
P(-3 ≤ X ≤ 3) = 0.9973 (99.73%)

--- Verification using scipy.stats.norm.cdf ---
P(-1 ≤ X ≤ 1) = 0.6827
P(-2 ≤ X ≤ 2) = 0.9545
P(-3 ≤ X ≤ 3) = 0.9973
```

This confirms the famous **68-95-99.7 rule** of the Normal Distribution.

Exercises

Exercise 1

Use a Riemann Sum (with $n = 100$) to approximate:

$$\int_0^2 (3x^2 + 2x + 1)\, dx$$

Then verify your answer using `scipy.integrate.quad`.

Hint: The exact answer is 14.

Exercise 2

Using `sympy.integrate`, compute the following indefinite integrals:

1. $\int (4x^3 - 2x)\, dx$
2. $\int e^{2x}\, dx$
3. $\int \frac{1}{x}\, dx$

Exercise 3

A machine produces parts whose weights follow a Normal distribution with mean $\mu = 100g$ and standard deviation $\sigma = 5g$.

What is the probability that a randomly selected part weighs **between 90g and 110g**?

Use `scipy.integrate.quad` with a normal PDF to answer this.

Solution — Exercise 1

```python
import numpy as np
from scipy.integrate import quad

def f(x):
    return 3*x**2 + 2*x + 1

# Riemann Sum (Midpoint, n=100)
a, b, n = 0, 2, 100
dx = (b - a) / n
x_mid = np.linspace(a + dx/2, b - dx/2, n)
riemann = np.sum(f(x_mid) * dx)
print(f"Riemann Sum (n=100): {riemann:.6f}")

# Exact via scipy
exact, _ = quad(f, 0, 2)
print(f"scipy.quad result : {exact:.6f}")
print(f"True answer       : 14")
```

Linear Algebra

Linear Algebra is the mathematical language of Data Science and Machine Learning. It allows us to represent, manipulate, and compute on large datasets efficiently using vectors, matrices, and tensor operations.

This module is organized into six specialized chapters:

1. **Vector Spaces & Matrix Rank:** Explores vector space foundations, linear independence, basis, dimension, and rank deficiency (multicollinearity).
2. **Determinants:** Explores matrix determinants, permutation-based definitions, properties under row operations, and invertibility conditions.
3. **Norms & Orthogonality:** Introduces vector norms (L_1, L_2, L_∞), orthogonality, Gram-Schmidt process, and ML regularization (Lasso & Ridge).
4. **Eigenvalues, Eigenvectors & PCA:** Covers matrix diagonalization, eigendecomposition, and Principal Component Analysis (PCA) for dimensionality reduction.
5. **SVD & Matrix Factorizations:** Explores Singular Value Decomposition (SVD), LU & QR Factorizations, and numerically stable Least Squares.
6. **Linear Transformations & Neural Networks:** Explains geometric matrix transformations and how Dense layers in Neural Networks operate.

Quick Primer: Vectors & Matrices

Vectors & Data Points

A **vector** $x \in \mathbb{R}^d$ represents a data point with d features:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

Feature Matrices

A **matrix** $A \in \mathbb{R}^{m \times n}$ represents a dataset with m samples and n features:

$$A = \begin{bmatrix} A_{11} & A_{12} & \dots & A_{1n} \\ A_{21} & A_{22} & \dots & A_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ A_{m1} & A_{m2} & \dots & A_{mn} \end{bmatrix}$$

Basic Python NumPy Operations

```python
import numpy as np

# Create matrices
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Matrix Multiplication (Dot product)
C = A @ B
print("A @ B =\n", C)

# Transpose & Determinant
print("\nTranspose of A =\n", A.T)
print(f"Determinant of A: {np.linalg.det(A):.4f}")
```

Now, let's dive into the first chapter: Vector Spaces & Matrix Rank!

Vector Spaces & Matrix Rank

Vector spaces and matrix rank form the foundation of linear algebra. Understanding these concepts helps Data Scientists detect feature redundancy and solve linear systems efficiently.

---

1. Vector Spaces & Subspaces

A **Vector Space** V is a set of vectors closed under vector addition and scalar multiplication.

A **Subspace** $S \subseteq V$ is a subset of a vector space that is itself a vector space. It must satisfy three rules:

1. Contains the zero vector $\vec{0} \in S$.
2. **Closed under addition:** If $u, v \in S$, then $u + v \in S$.
3. **Closed under scalar multiplication:** If $v \in S$ and $c \in \mathbb{R}$, then $cv \in S$.

---

2. Linear Independence, Span & Basis

Linear Combination & Span

- **Linear Combination:** A vector v is a linear combination of $\{v_1, \ldots, v_k\}$ if:

$$v = c_1v_1 + c_2v_2 + \cdots + c_kv_k \quad (c_i \in \mathbb{R})$$

- **Span:** The set of all possible linear combinations of vectors $\{v_1, \ldots, v_k\}$:

$$\text{Span}(v_1, \ldots, v_k) = \{c_1v_1 + \cdots + c_kv_k \mid c_i \in \mathbb{R}\}$$

Linear Independence

A set of vectors $\{v_1, \ldots, v_k\}$ is **linearly independent** if no vector in the set can be written as a linear combination of the others. Mathematically, the only solution to:

$$c_1v_1 + c_2v_2 + \cdots + c_kv_k = \vec{0}$$

is $c_1 = c_2 = \cdots = c_k = 0$. If non-zero constants c_i exist, the vectors are **linearly dependent**.

Basis & Dimension

- **Basis:** A minimal set of linearly independent vectors that spans a vector space V.
- **Dimension ($\dim(V)$):** The number of vectors in any basis for V. For example, $\mathbb{R}^3$ has dimension 3.

---

3. Matrix Rank & Multicollinearity

The **Rank** of a matrix $A \in \mathbb{R}^{m \times n}$ (denoted $\text{rank}(A)$) is the maximum number of linearly independent column vectors (which equals the maximum number of linearly independent row vectors).

- **Full Rank:** $\text{rank}(A) = \min(m, n)$.
- **Rank Deficient:** $\text{rank}(A) < \min(m, n)$, meaning at least one column/row is a linear combination of others.

Data Science Insight: Multicollinearity in Regression

In linear regression ($y = Xw$), if two features are perfectly correlated (e.g., house size in sq ft and sq meters), the feature matrix X becomes rank deficient.

Consequently, the matrix $(X^T X)$ is non-invertible (singular), causing Ordinary Least Squares (OLS) estimation $w = (X^T X)^{-1} X^T y$ to fail or yield wildly unstable weights!

```
import numpy as np

# Feature matrix X with a redundant column (Col 3 = 2 * Col 1)
X = np.array([
    [1, 2, 2],
    [2, 5, 4],
    [3, 1, 6]
])

# Compute rank
rank = np.linalg.matrix_rank(X)
print("Matrix Rank:", rank)  # Output: 2 (Rank deficient because Col 3 is dependent!)

# Attempting to invert (X^T X) will result in numerical instability or high condition number
XTX = X.T @ X
print("Condition Number of X^T X:", np.linalg.cond(XTX))  # Very large number!
```

Determinants

Given a square matrix, is there an easy way to know when it is invertible? Answering this fundamental question is the main goal of this chapter. The **determinant** of a square matrix is a single scalar value that determines whether the matrix is invertible (non-singular) and measures how the matrix scales areas or volumes under linear transformations.

---

1. Simple Examples (Small Cases)

For small matrices, the determinant formula is straightforward:

- 1×1 **Matrix:** If $M = [m]$, then $M^{-1} = [1/m]$. Thus, M is invertible if and only if:

$$\det(M) = m \neq 0$$

- 2×2 **Matrix:** If $M = \begin{bmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{bmatrix}$, its inverse is:

$$M^{-1} = \frac{1}{m_{11}m_{22} - m_{12}m_{21}} \begin{bmatrix} m_{22} & -m_{12} \\ -m_{21} & m_{11} \end{bmatrix}$$

  Thus, M is invertible if and only if the denominator is non-zero. This quantity is defined as the determinant:

$$\det(M) = m_{11}m_{22} - m_{12}m_{21}$$

- 3×3 **Matrix:** For $M = \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix}$, the determinant is:

$$\det(M) = m_{11}m_{22}m_{33} - m_{11}m_{23}m_{32} + m_{12}m_{23}m_{31} - m_{12}m_{21}m_{33} + m_{13}m_{21}m_{32} - m_{13}m_{22}m_{31}$$

---

2. Permutations and the General Definition

To generalize the determinant formula to $n \times n$ matrices, we use the mathematics of **permutations**.

What is a Permutation?

A permutation σ of the set $\{1, 2, \ldots, n\}$ is a bijective mapping of the set onto itself (a shuffle).

- There are $n!$ possible permutations of n distinct objects.
- Every permutation can be built by successively swapping pairs of objects (transpositions).
- **Parity (Sign of Permutation):** A permutation is **even** if it requires an even number of swaps to be built from the identity permutation $[1, 2, \ldots, n]$, and **odd** if it requires an odd number of swaps.

We define the sign function $\operatorname{sgn}(\sigma)$ as:

$$\operatorname{sgn}(\sigma) = \begin{cases} 1 & \text{if } \sigma \text{ is even} \\ -1 & \text{if } \sigma \text{ is odd} \end{cases}$$

General Definition of the Determinant

The determinant of an $n \times n$ matrix $M = (m_{ij})$ is defined by the Leibniz formula:

$$\det(M) = \sum_{\sigma} \operatorname{sgn}(\sigma) m_{1\sigma(1)} m_{2\sigma(2)} \cdots m_{n\sigma(n)}$$

The sum is taken over all $n!$ possible permutations σ of the set $\{1, \ldots, n\}$.

---

3. Properties under Row and Column Operations

We can compute the determinant of larger matrices efficiently by analyzing how elementary row operations affect the determinant:

1. **Row Swap (E_{ij}):** Swapping two rows of a matrix flips the sign of the determinant:

$$\det(E_{ij}M) = -\det(M)$$

- *Corollary:* If a matrix M has two identical rows, then swapping them changes the sign of the determinant but leaves the matrix unchanged ($\det(M) = -\det(M)$), which means $\det(M) = 0$.

2. **Row Scaling ($R_i(\lambda)$):** Multiplying a single row by a scalar λ scales the determinant by λ:

$$\det(R_i(\lambda)M) = \lambda \det(M)$$

3. **Row Addition ($S_{ij}(\mu)$):** Adding a scalar multiple of one row to another row leaves the determinant unchanged:

$$\det(S_{ij}(\mu)M) = \det(M)$$

Transpose Invariance

An important theorem states that the transpose of a matrix has the same determinant as the original matrix:

$$\det(M^T) = \det(M)$$

Because of this property, **every property of determinants regarding rows also applies to columns** (e.g., swapping two columns flips the sign of the determinant).

---

4. Expansion by Minors (Cofactor Expansion)

For matrices larger than 2×2, we can recursively calculate the determinant by expanding along any row or column using **minors** and **cofactors**.

- **Minor (M_{ij}):** The determinant of the submatrix obtained by deleting row i and column j from M.
- **Cofactor (C_{ij}):** The minor multiplied by a grid sign factor:

$$\text{cofactor}(m_{ij}) = C_{ij} = (-1)^{i+j}M_{ij}$$

Expansion Formula

Expanding along the i-th row:

$$\det(M) = \sum_{j=1}^{n} m_{ij}C_{ij} = \sum_{j=1}^{n}(-1)^{i+j}m_{ij}M_{ij}$$

Example: 3×3 Expansion by Minors

Let's compute the determinant of:

$$M = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Expanding along the first row:

$$\det(M) = 1 \det \begin{bmatrix} 5 & 6 \\ 8 & 9 \end{bmatrix} - 2 \det \begin{bmatrix} 4 & 6 \\ 7 & 9 \end{bmatrix} + 3 \det \begin{bmatrix} 4 & 5 \\ 7 & 8 \end{bmatrix}$$

$$\det(M) = 1(5 \cdot 9 - 8 \cdot 6) - 2(4 \cdot 9 - 7 \cdot 6) + 3(4 \cdot 8 - 7 \cdot 5)$$

$$\det(M) = 1(-3) - 2(-6) + 3(-3) = -3 + 12 - 9 = 0$$

Since $\det(M) = 0$, M is not invertible.

Simplification Using Row Operations

We can apply row operations to introduce zeros into a row or column before performing cofactor expansion to simplify calculations:

$$N = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 0 & 0 \\ 7 & 8 & 9 \end{bmatrix}$$

Since the second row has many zeros, we can swap Row 1 and Row 2 (which multiplies the determinant by -1):

$$\det(N) = -\det \begin{bmatrix} 4 & 0 & 0 \\ 1 & 2 & 3 \\ 7 & 8 & 9 \end{bmatrix} = -4 \det \begin{bmatrix} 2 & 3 \\ 8 & 9 \end{bmatrix} = -4(2 \cdot 9 - 8 \cdot 3) = 24$$

---

5. Determinants of Products, Diagonals, and Inverses

- **Diagonal Matrices:** The determinant of a diagonal matrix is the product of its diagonal entries.

$$\det(\text{diag}(d_1, \ldots, d_n)) = d_1 d_2 \cdots d_n \implies \det(I) = 1$$

- **Product Rule:** For any two square matrices M and N:

$$\det(MN) = \det(M)\det(N)$$

- **Inverse Rule:**

$$\det(M^{-1}) = \frac{1}{\det(M)}$$

---

6. The Adjoint Matrix and Inverse Formula

For any square matrix M, we can define the **Cofactor Matrix** C where each entry is $C_{ij} = \text{cofactor}(m_{ij})$. The transpose of the cofactor matrix is called the **Adjoint (or Adjuggate) Matrix**, denoted as $\text{adj}(M)$:

$$\text{adj}(M) = C^T$$

Important Relation

The product of a matrix and its adjoint always yields a diagonal matrix of determinants:

$$M\,\text{adj}(M) = \det(M)I$$

This gives us an explicit analytical formula for the inverse matrix:

$$M^{-1} = \frac{1}{\det(M)}\text{adj}(M)$$

---

7. Geometric Application: Volume of a Parallelepiped

In three-dimensional space, the absolute value of the determinant of a 3×3 matrix represents the **volume of the parallelepiped** spanned by its column vectors u, v, and w:

$$\text{Volume} = |\det([u \quad v \quad w])|$$

This matches the scalar triple product $|u \cdot (v \times w)|$ learned in multivariate calculus.

---

8. Python Implementation (NumPy & SymPy)

We can compute determinants, transposes, inverses, and adjoint matrices in Python:

```python
import numpy as np
import sympy as sp

# 1. Numerical calculations with NumPy
M_num = np.array([
    [3, -1, -1],
    [1,  2,  0],
    [0,  1,  1]
])
det_num = np.linalg.det(M_num)
print("Numerical Determinant (NumPy):", round(det_num, 4))  # 6.0
print("Numerical Inverse (NumPy):\n", np.linalg.inv(M_num))

# 2. Symbolic and Adjoint calculations with SymPy
a, b, c, d = sp.symbols('a b c d')
M_sym = sp.Matrix([[a, b], [c, d]])

# Compute symbolic determinant, inverse, and adjoint
print("\nSymbolic Determinant:", M_sym.det())
print("Symbolic Adjoint (adj M):\n", M_sym.adjugate())
print("Symbolic Inverse (M^-1):\n", M_sym.inv())
```

Norms & Orthogonality

Vector norms and orthogonality are foundational to geometry, optimization, and Machine Learning regularization techniques.

---

1. Vector Norms (L_1, L_2, L_∞)

A **Norm** $\|\cdot\|$ measures the length or magnitude of a vector $x \in \mathbb{R}^n$. It satisfies three conditions: non-negativity ($\|x\| \geq 0$), absolute scalability ($\|cx\| = |c|\|x\|$), and triangle inequality ($\|x + y\| \leq \|x\| + \|y\|$).

1.1 L_1 Norm (Manhattan / Taxicab Norm)

The sum of the absolute values of the vector components:

$$\|x\|_1 = \sum_{i=1}^{n} |x_i|$$

1.2 L_2 Norm (Euclidean Norm)

The standard geometric distance from the origin:

$$\|x\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2} = \sqrt{x^T x}$$

1.3 L_∞ Norm (Max Norm)

The maximum absolute component value:

$$\|x\|_\infty = \max_{1 \le i \le n} |x_i|$$

---

2. Orthogonality & Gram-Schmidt Process

2.1 Orthogonal Vectors

Two vectors u, v are **orthogonal** ($u \perp v$) if their inner product (dot product) is zero:

$$u^T v = \sum_{i=1}^{n} u_i v_i = 0$$

An **Orthonormal Set** of vectors is mutually orthogonal ($q_i^T q_j = 0$ for $i \neq j$) and normalized ($\|q_i\|_2 = 1$).

2.2 Gram-Schmidt Process

The **Gram-Schmidt process** converts a set of linearly independent vectors $\{v_1, v_2, \ldots, v_k\}$ into an orthonormal basis $\{q_1, q_2, \ldots, q_k\}$:

1. $u_1 = v_1 \implies q_1 = \frac{u_1}{\|u_1\|}$
2. $u_2 = v_2 - (v_2^T q_1) q_1 \implies q_2 = \frac{u_2}{\|u_2\|}$
3. $u_k = v_k - \sum_{j=1}^{k-1} (v_k^T q_j) q_j \implies q_k = \frac{u_k}{\|u_k\|}$

---

3. Machine Learning Application: Regularization (Lasso & Ridge)

In Machine Learning, when a model has too many parameters, it risks **overfitting** (memorizing noise in the training data). To prevent this, we add a norm penalty on the model weight vector w to the loss function:

$$\text{Loss}_{\text{regularized}} = \text{Loss}_{\text{original}} + \lambda \cdot \text{Penalty}(w)$$

Ridge Regression (L_2 Regularization)

- **Penalty:** $\lambda \|w\|_2^2 = \lambda \sum w_i^2$
- **Effect:** Penalizes large weights smoothly, shrinking parameters towards zero without making them exactly zero.

Lasso Regression (L_1 Regularization)

- **Penalty:** $\lambda \|w\|_1 = \lambda \sum |w_i|$
- **Effect:** Encourages **sparsity** (drives irrelevant feature weights strictly to 0.0), functioning as an automated feature selection tool!

```python
import numpy as np

# Model weights
w = np.array([3.0, -4.0, 0.0])

# Compute norms
l1_norm = np.linalg.norm(w, ord=1)
l2_norm = np.linalg.norm(w, ord=2)

print(f"Weights w: {w}")
print(f"L1 Norm (Lasso Penalty): {l1_norm:.2f}")  # 3 + 4 = 7.0
print(f"L2 Norm (Ridge Penalty): {l2_norm:.2f}")  # sqrt(9 + 16) = 5.0
```

Eigenvalues, Eigenvectors & PCA

Matrix Diagonalization, Eigendecomposition, and Principal Component Analysis (PCA) are central to dimensionality reduction and feature extraction in Machine Learning.

---

1. Eigenvalues & Eigenvectors

When a square matrix $A \in \mathbb{R}^{n \times n}$ multiplies a vector x, it generally changes both the vector's length and direction. However, for certain special vectors, the transformation only **scales** the vector without changing its direction.

$$Av = \lambda v$$

- **Eigenvector (v):** A non-zero vector whose direction remains unchanged after transformation by A.
- **Eigenvalue (λ):** The scaling factor corresponding to eigenvector v.

Finding Eigenvalues & Eigenvectors

1. **Solve Characteristic Polynomial:** $\det(A - \lambda I) = 0$ to find eigenvalues λ_i.
2. **Solve Linear System:** $(A - \lambda_i I)v_i = 0$ to find corresponding eigenvectors v_i.

---

2. Matrix Diagonalization

A square matrix A is **diagonalizable** if it can be written as:

$$A = PDP^{-1}$$

Where:

- D is a **diagonal matrix** containing the eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_n$.
- P is an invertible matrix whose columns are the corresponding eigenvectors $v_1, v_2, \ldots, v_n$.

Computing Matrix Powers (A^k)

Diagonalization dramatically simplifies matrix exponentiation:

$$A^k = (PDP^{-1})^k = PD^kP^{-1}$$

Since D is diagonal, D^k is computed instantaneously by raising each diagonal element to power k.

---

3. Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is one of the most fundamental Machine Learning algorithms for dimensionality reduction, feature extraction, and data visualization.

Given a dataset with many correlated features, PCA transforms the data into a new coordinate system of orthogonal axes called **Principal Components (PCs)** using Eigendecomposition or Singular Value Decomposition (SVD) on the centered/standardized Covariance Matrix:

- **First Principal Component (PC1):** The axis along which the data has the **maximum variance** (spread).
- **Second Principal Component (PC2):** The axis orthogonal (perpendicular) to PC1 that captures the second highest variance.
- **Subsequent Components (PC_k):** Orthogonal to all previous components, capturing maximum remaining variance.

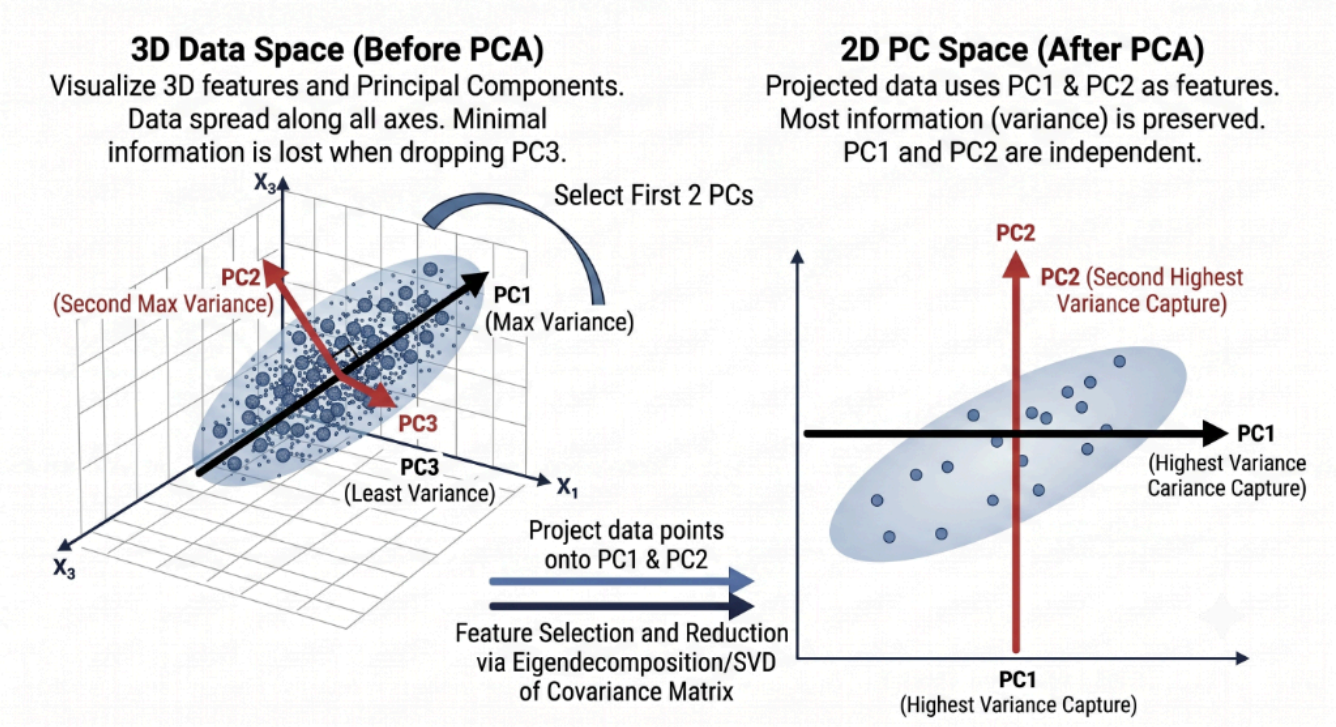


Fig. 12

Principal Component Analysis (PCA) diagram showing 3D original space reduced to 2D PC space.

4. Python Implementation of PCA

Here is how to perform PCA step-by-step using NumPy:

```
import numpy as np

# 1. Generate correlated 2D dataset (100 samples, 2 features)
np.random.seed(42)
X = np.random.multivariate_normal([0, 0], [[3, 2.2], [2.2, 2]], 100)

# 2. Step 1: Center the data (subtract mean)
X_centered = X - np.mean(X, axis=0)

# 3. Step 2: Compute Covariance Matrix
cov_matrix = np.cov(X_centered.T)

# 4. Step 3: Compute Eigenvalues & Eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

# 5. Step 4: Sort eigenvectors by eigenvalues in descending order
sorted_idx = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[sorted_idx]
eigenvectors = eigenvectors[:, sorted_idx]

print("Principal Component 1 (PC1) Vector:", eigenvectors[:, 0])
print(f"Variance explained by PC1: {eigenvalues[0] / np.sum(eigenvalues) * 100:.2f}%")

# 6. Step 5: Project 2D data onto 1D (Dimensionality Reduction)
X_1D = X_centered @ eigenvectors[:, 0]
print("Shape of reduced data:", X_1D.shape) # Reduced from (100, 2) to (100,)
```

SVD & Matrix Factorizations

Matrix factorizations decompose complex matrices into product forms that are easier to analyze, invert, or compute.

---

1. Singular Value Decomposition (SVD)

1.1 Mathematical Definition

While Eigendecomposition $A = PDP^{-1}$ only works for square matrices, **Singular Value Decomposition (SVD)** applies to **any** $m \times n$ matrix A:

$$A = U\Sigma V^T$$

Where:

- U is an $m \times m$ **orthogonal matrix** containing left-singular vectors (eigenvectors of AA^T).
- Σ is an $m \times n$ **diagonal matrix** containing singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$ (square roots of eigenvalues of A^TA).
- V^T is the transpose of an $n \times n$ **orthogonal matrix** V containing right-singular vectors (eigenvectors of A^TA).

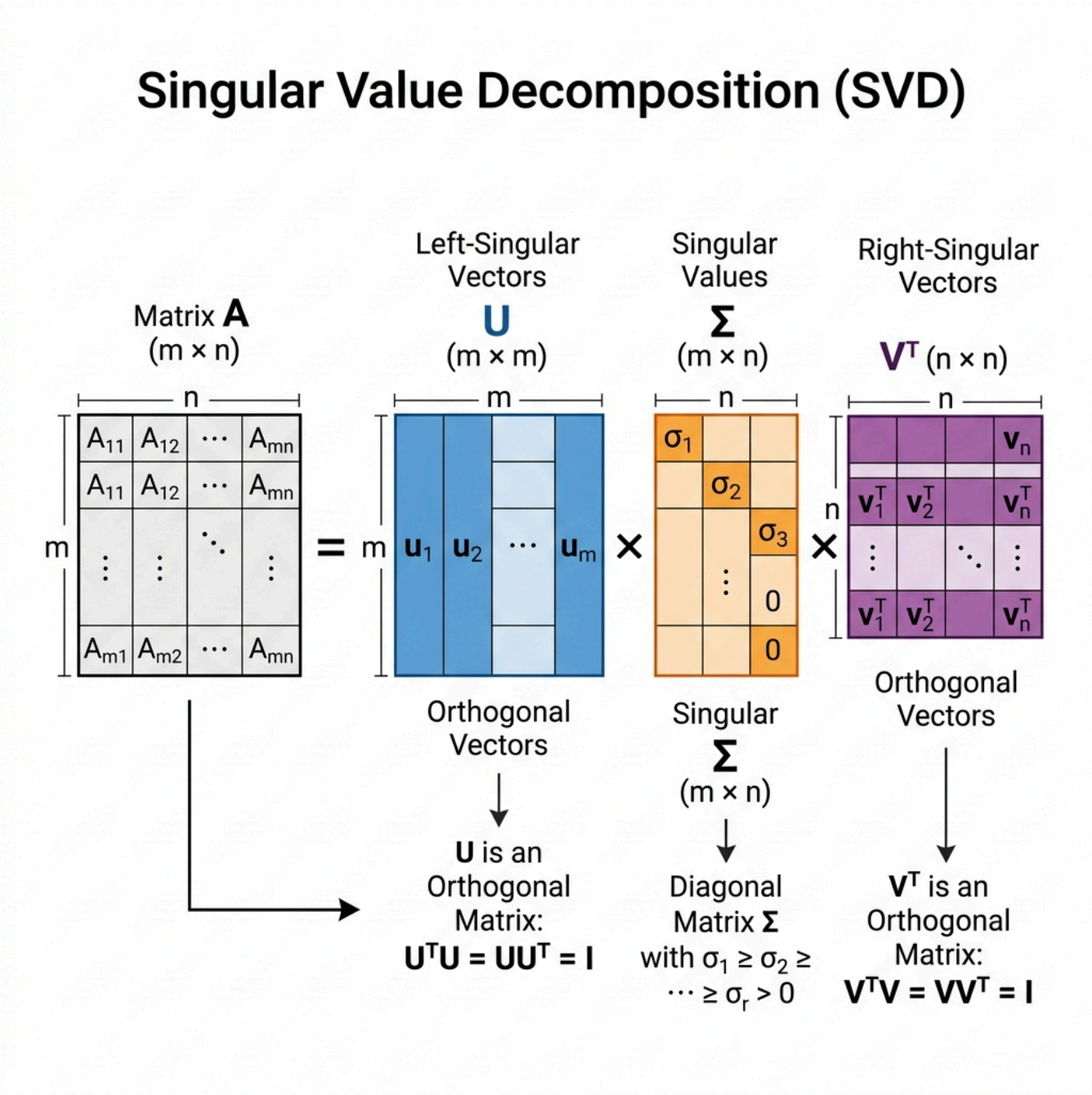


Fig. 13

Singular value decomposition of a matrix.

1.2 Truncated SVD & Applications

By keeping only the top k largest singular values (Σ_k), we get the optimal rank-k approximation of A (Eckart–Young–Mirsky Theorem):

$$A \approx U_k \Sigma_k V_k^T$$

Applications in AI & Data Science:

1. **Recommender Systems (Matrix Factorization):** Decomposing user-item rating matrices (e.g., Collaborative Filtering on Netflix/Spotify).
2. **Natural Language Processing:** Truncated SVD on term-document matrices (Latent Semantic Analysis - LSA).
3. **Image Compression:** Storing high-resolution images using only a fraction of singular values.

```python
import numpy as np

# Sample 5x4 matrix
A = np.array([
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12],
    [13, 14, 15, 16],
    [17, 18, 19, 20]
])

# Full SVD
U, s, Vt = np.linalg.svd(A)

print("U shape:", U.shape)     # (5, 5)
print("Singular values:", s)   # Singular values
print("Vt shape:", Vt.shape)   # (4, 4)

# Truncated SVD (Rank 2 Approximation)
k = 2
S_k = np.diag(s[:k])
A_reconstructed = U[:, :k] @ S_k @ Vt[:k, :]
print("\nReconstructed Matrix (Rank 2 Approximation):\n", np.round(A_reconstructed, 2))
```

---

2. LU Decomposition

LU Decomposition factors a square matrix A into a lower triangular matrix L and an upper triangular matrix U:

$$A = LU$$

Why use LU? Solving $Ax = b$ via standard Gaussian elimination takes $O(n^3)$ operations. Once A is factored into LU, solving for multiple target vectors b takes only $O(n^2)$ via forward and backward substitution!

---

3. QR Decomposition & Least Squares

QR Decomposition factors an $m \times n$ matrix A into an orthogonal matrix Q ($Q^TQ = I$) and an upper triangular matrix R:

$$A = QR$$

Application: Numerically Stable Least Squares

In linear regression ($Xw = y$), computing $(X^T X)^{-1}$ directly can be numerically unstable if $X^T X$ has a high condition number. Using QR decomposition ($X = QR$):

$$w = R^{-1} Q^T y$$

This avoids explicitly computing matrix inverses, vastly improving numerical precision!

```
import numpy as np

X = np.array([[1, 1], [1, 2], [1, 3]])
y = np.array([6, 5, 7])

# QR Decomposition of Feature Matrix X
Q, R = np.linalg.qr(X)

# Solve w = R^-1 @ Q.T @ y
w = np.linalg.inv(R) @ Q.T @ y
print("Weights via QR Decomposition:", w)
```

Linear Transformations & Neural Networks

Matrix operations can be geometrically viewed as transformations of vector spaces. Understanding linear maps sheds light on how Neural Networks process and reshape data.

---

1. Matrix as a Geometric Transformation

A matrix $A \in \mathbb{R}^{m \times n}$ acts as a function $T : \mathbb{R}^n \to \mathbb{R}^m$ defined by:

$$T(x) = Ax$$

Linear transformations satisfy two fundamental properties for any vectors u, v and scalar c:

1. **Additivity:** $T(u + v) = T(u) + T(v)$
2. **Homogeneity:** $T(cv) = cT(v)$

Common 2D Geometric Transformations

- **Scaling:** $\begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$ (resizes vectors along axes).
- **Rotation:** $\begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$ (rotates vectors counter-clockwise by angle θ).

- **Shearing:** $\begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$ (slants shapes horizontally).

```
import numpy as np

# Rotation matrix for theta = 90 degrees (pi/2)
theta = np.pi / 2
R = np.array([
    [np.cos(theta), -np.sin(theta)],
    [np.sin(theta),  np.cos(theta)]
])

v = np.array([1.0, 0.0]) # Vector pointing right along X-axis
v_rotated = R @ v

print("Original vector:", v)
print("Rotated vector (90 deg counter-clockwise):", np.round(v_rotated, 2)) # Points up along Y-
axis!
```

---

2. Deep Learning: Dense Layers as Linear Maps

In Deep Learning, a single fully connected (Dense) layer transforms an input feature vector $x \in \mathbb{R}^n$ into an output representation $h \in \mathbb{R}^m$:

$$h = \sigma(Wx + b)$$

Where:

- $W \in \mathbb{R}^{m \times n}$ is the Weight Matrix (Linear Transformation mapping feature space).
- $b \in \mathbb{R}^m$ is the Bias Vector (Translation / Shift).
- $\sigma(\cdot)$ is a non-linear Activation Function (e.g., ReLU, Sigmoid, Tanh).

Why Non-Linearity Matters

If we stack 100 neural network layers without non-linear activations (σ), the entire network reduces to a single combined linear transformation:

$$y = W_{100}W_{99}\dots W_1 x + b' = W_{\text{combined}}x + b'$$

No matter how deep the network is, a linear transformation can only draw straight decision boundaries (hyperplanes). Non-linear activation functions bend and warp the feature space, allowing Deep Learning models to learn highly complex, non-linear patterns!

Probability

Probability is the mathematical language of uncertainty. In data science and machine learning, we use probability to build models under uncertainty, estimate parameters, evaluate classifiers, and construct complex systems like Bayesian networks.

This module is divided into four main chapters:

1. **Foundations of Probability:** Covers the core axioms, conditional probability, and Bayes' Theorem (the backbone of Bayesian Inference).
2. **Random Variables & Probability Distributions:** Introduces discrete and continuous random variables, expectations, variance, and common distributions (Normal, Binomial, Poisson).
3. **Convergence & Limit Theorems:** Explores what happens when we collect a lot of data, featuring the Law of Large Numbers (LLN) and the Central Limit Theorem (CLT).
4. **Information Theory Basics:** Connects probability to information theory, explaining Entropy, Cross-Entropy, and KL Divergence—concepts foundational to Deep Learning loss functions.

A Quick Primer: Probability vs. Odds

Before diving into the chapters, it is crucial to distinguish between **Probability** and **Odds**, two concepts often confused in casual language but mathematically distinct.

- **Probability (P)**: The ratio of the target event's occurrence to all possible outcomes. It is always bounded between 0 and 1.

$$P = \frac{\text{Successes}}{\text{Total Outcomes}}$$

- **Odds (O)**: The ratio of the probability of the event occurring (P) to the probability of the event *not* occurring ($1 - P$). Its value can range from 0 to ∞.

$$O = \frac{P}{1 - P}$$

Feature	Probability (P)	Odds (O)
Definition	Likelihood of event over total outcomes	Likelihood of event over non-event
Range	$[0, 1]$ or $[0\%, 100\%]$	$[0, \infty)$
Example (Rolling a 4 on a 6-sided die)	$\frac{1}{6} \approx 0.1667$ (16.67%)	$\frac{1/6}{5/6} = \frac{1}{5}$ $(1:5 \text{ or } 0.2)$

> **Note**
>
> In machine learning, **Odds** are highly important in **Logistic Regression**. The model outputs log-odds (logit): $\ln(\frac{P}{1-P}) = wx + b$.

Now, let's move on to the first chapter: Foundations of Probability!

Foundations of Probability

Probability is the mathematical language used to quantify uncertainty. In Data Science and Machine Learning, we rarely have perfect data or complete information about the real world. Therefore, probability helps us make the most reasonable predictions based on available data.

This chapter introduces the core concepts of probability, moving from intuition to rigorous mathematical definitions.

1. Sample Space and Events

Imagine you flip a coin. The outcome can be Heads (H) or Tails (T).

- **Experiment:** An action with an uncertain outcome (e.g., flipping a coin).
- **Sample Space (Ω):** The set of all possible outcomes of an experiment.
  - *Example (Coin):* $\Omega = \{H, T\}$
  - *Example (Die):* $\Omega = \{1, 2, 3, 4, 5, 6\}$
- **Event (E):** A subset of the sample space. It is the set of outcomes we are interested in.
  - *Example:* Let E be the event "rolling an even number". Then $E = \{2, 4, 6\}$.

In Python, we can simulate simple events using sets:

```
# Sample space when rolling a single die
Omega = {1, 2, 3, 4, 5, 6}

# Event E: rolling an even number
E = {2, 4, 6}

# Does the event occur if the roll result is 4?
roll_result = 4
is_event_E = roll_result in E
print(f"Did Event E occur? {is_event_E}") # True
```

2. Probability Measure and Kolmogorov's Axioms

From an intuitive (Frequentist) perspective, the probability of an event is the proportion of times that event occurs if we repeat the experiment a large number of times.

$$P(E) = \frac{\text{Number of favorable outcomes for } E}{\text{Total number of possible outcomes in } \Omega}$$

(This formula applies when the outcomes in the sample space are equally likely).

However, to establish a rigorous mathematical foundation, mathematician Andrey Kolmogorov introduced 3 axioms for the probability measure P:

1. **Non-negativity:** The probability of any event cannot be negative.

$$P(E) \geq 0$$

2. **Normalization:** The probability of the entire sample space is always 1 (it is certain that *some* outcome in the sample space will occur).

$$P(\Omega) = 1$$

3. **Additivity:** If two events A and B are mutually exclusive (they cannot occur simultaneously, meaning $A \cap B = \emptyset$), then the probability of A or B occurring is the sum of their individual probabilities.

$$P(A \cup B) = P(A) + P(B)$$

From these 3 axioms, we can derive all other probability rules, for example: $P(E^c) = 1 - P(E)$ (where E^c is the complement of E).

3. Conditional Probability and Independence

Conditional Probability

In reality, gaining new information can change the likelihood of an event occurring. Conditional probability $P(A|B)$ is the probability of event A occurring **given that** event B has already occurred.

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \quad (\text{for } P(B) > 0)$$

Intuition: The original sample space Ω is now restricted to just B. We examine what proportion the intersection of A and B takes up within this new space B.

Independence

Two events A and B are considered **independent** if the occurrence of B provides no additional information about whether A will occur (and vice versa).

Mathematically:

$$P(A|B) = P(A)$$

Which is equivalent to:

$$P(A \cap B) = P(A) \times P(B)$$

4. Bayes' Theorem and Law of Total Probability

Bayes' Theorem

From the conditional probability formula, we have $P(A \cap B) = P(A|B)P(B)$ and $P(B \cap A) = P(B|A)P(A)$. Since $P(A \cap B) = P(B \cap A)$, we can derive the famous Bayes' Theorem:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

In Machine Learning, this theorem is often understood from the perspective of updating beliefs (Bayesian Inference):

- $P(A)$ **(Prior):** Our initial belief about hypothesis A before seeing the data.
- $P(B|A)$ **(Likelihood):** The probability of observing the data B if hypothesis A is true.
- $P(B)$ **(Evidence):** The overall probability of observing the data B.
- $P(A|B)$ **(Posterior):** Our updated belief about hypothesis A after observing the data B.

Simple Explanation of Bayes Theorem!

Imagine you are building a basic Spam Filter. You receive an email containing the word **"Free"**. What is the probability that this email is Spam?

Bayes' Theorem states that you cannot just look at the word "Free" and jump to a conclusion. You need to combine 3 pieces of information:

1. **Prior:** Generally, what percentage of emails sent to you are Spam? (e.g., 20%).
2. **Likelihood:** Among the emails that are *definitely Spam*, what percentage contain the word "Free"? (e.g., Scammers use this word a lot $\rightarrow$ 80%).
3. **Evidence:** Overall, in *all emails* (both real and Spam), how often does the word "Free" appear? (e.g., Legitimate companies also send promotional emails like "Free shipping" $\rightarrow$ 25%).

Plugging it into the formula:

$$P(\text{Spam}|\text{"Free"}) = \frac{80\% \times 20\%}{25\%} = 64\%$$

Conclusion: Even though the word "Free" appears frequently in spam (80%), because the overall volume of spam is relatively low (20%) and normal emails also use this word, the actual probability that the email is spam is only 64% (not certain enough to delete it outright). This is how Bayes helps AI (and humans) reason logically and avoid hasty conclusions!

Another Example: The Two Bowls

Let's consider a classic probability puzzle: Imagine you have two bowls in front of you. You close your eyes, pick a bowl at random, and then draw a single marble from it.

- **Bowl 1** contains 4 marbles: **3 Red** and **1 Blue**.
- **Bowl 2** contains 4 marbles: **1 Red** and **3 Blue**.

You open your eyes and see that the marble you picked is **Red**. What is the probability that you picked from **Bowl 1**?

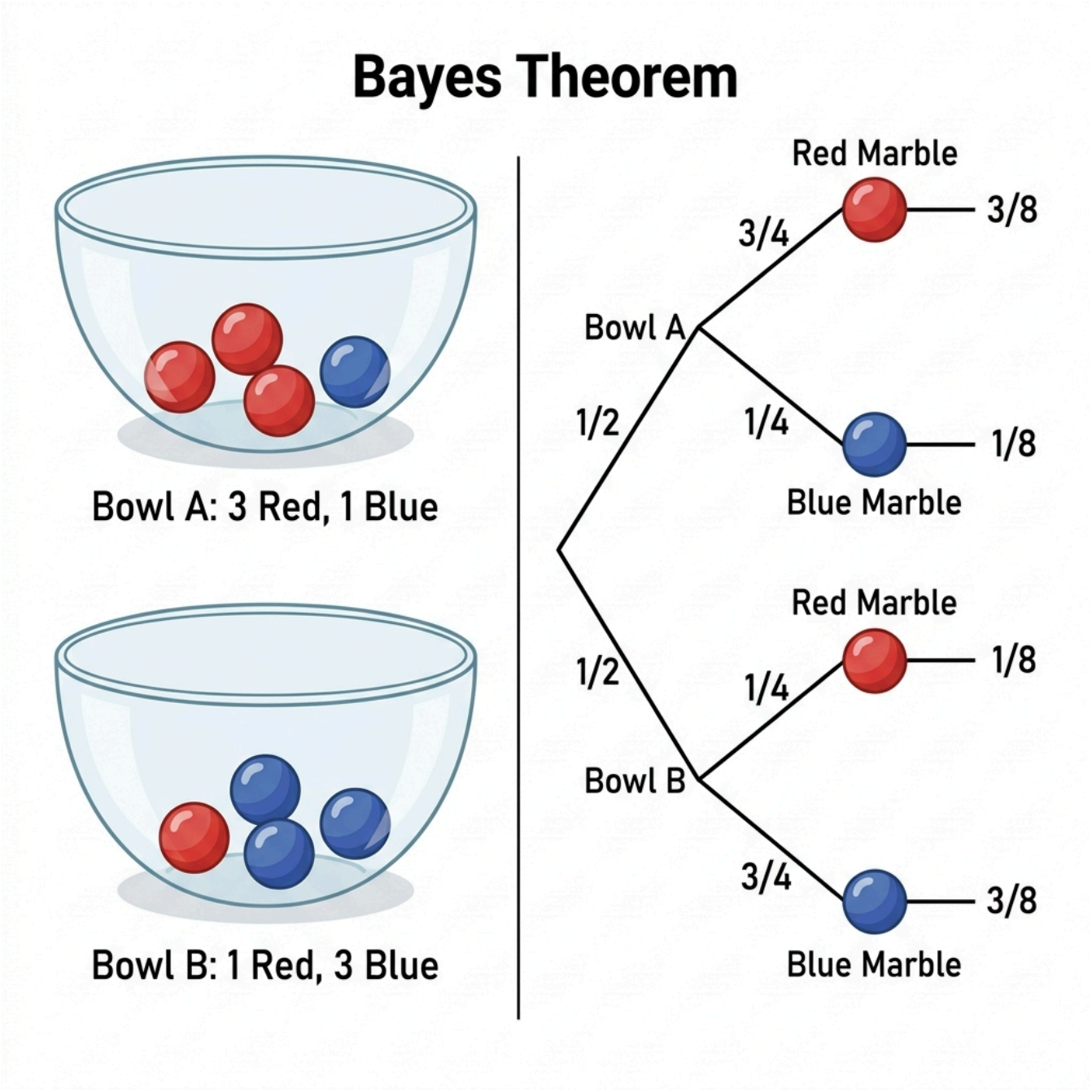


Fig. 14

AI-generated Bayes' Theorem Infographic Diagram for the Two Bags Problem.

Let's break this down using Bayes' Theorem:

- **Prior ($P(\text{Bowl } 1)$):** Before looking at the marble, you picked a bowl at random. Since there are two bowls, the probability is 0.5 (or 50%).
- **Likelihood ($P(\text{Red}|\text{Bowl } 1)$):** If you were definitely drawing from Bowl 1, what is the chance of getting a Red marble? There are 3 Red out of 4, so it's $3/4 = 0.75$.
- **Likelihood for the other bowl ($P(\text{Red}|\text{Bowl } 2)$):** If you were drawing from Bowl 2, the chance is $1/4 = 0.25$.
- **Evidence ($P(\text{Red})$):** What is the total probability of drawing a Red marble overall? We use the Law of Total Probability (explained in the next section):

$$
\begin{aligned}
P(\text{Red}) &= P(\text{Red}|\text{Bowl 1}) \times P(\text{Bowl 1}) + P(\text{Red}|\text{Bowl 2}) \times P(\text{Bowl 2}) \\
&= 0.75 \times 0.5 + 0.25 \times 0.5 \\
&= 0.375 + 0.125 \\
&= 0.5
\end{aligned}
$$

Now, applying Bayes' formula:

$$
P(\text{Bowl 1}|\text{Red}) = \frac{P(\text{Red}|\text{Bowl 1}) \times P(\text{Bowl 1})}{P(\text{Red})} = \frac{0.75 \times 0.5}{0.5} = 0.75
$$

Conclusion: Given that you drew a Red marble, there is a **75% chance** that you drew it from Bowl 1. This perfectly matches our intuition because Bowl 1 has way more red marbles!

Law of Total Probability

To calculate the denominator $P(B)$ in Bayes' Theorem, we often use the law of total probability. If the sample space Ω is partitioned into mutually exclusive events $A_1, A_2, \ldots, A_n$ (meaning they cover the entire sample space and do not overlap), then:

$$
P(B) = \sum_{i=1}^{n} P(B|A_i)P(A_i)
$$

Python Example: Medical Diagnosis

Suppose there is a rare disease that only 1% of the population has ($P(D) = 0.01$). A test has an accuracy of 99% for people with the disease ($P(+|D) = 0.99$) and a false positive rate of 5% for healthy people ($P(+|D^c) = 0.05$). If a person tests positive (+), what is the probability they actually have the disease ($P(D|+)$)?

```python
# Probability of having the disease (Prior) and not having it
p_disease = 0.01
p_no_disease = 1 - p_disease

# Probability of a positive test (Likelihood)
p_pos_given_disease = 0.99
p_pos_given_no_disease = 0.05

# Law of total probability: Probability of getting a positive test (Evidence)
p_pos = (p_pos_given_disease * p_disease) + (p_pos_given_no_disease * p_no_disease)

# Bayes' Theorem: Probability of disease given a positive test (Posterior)
p_disease_given_pos = (p_pos_given_disease * p_disease) / p_pos

print(f"Probability of disease given positive test: {p_disease_given_pos:.2%}")
# Output: Probability of disease given positive test: 16.67%
```

Are you surprised? Even though the test seems very accurate (99%), because the disease is so rare (1%), most positive results are actually false positives! This is the power of Bayes' Theorem in correcting human intuition.

Random Variables & Probability Distributions

While the sample space Ω contains raw outcomes (e.g., "Heads", "Tails"), we often want to work with numbers for easier mathematical computation. **Random Variables (RV)** serve as this bridge.

A random variable, often denoted as X, Y, Z, is a mathematical function that maps outcomes from the sample space to a real number: $X : \Omega \to \mathbb{R}$.

1. Discrete vs. Continuous Random Variables

Discrete Random Variables

Discrete random variables take on countable values (e.g., the number of Heads in 3 coin flips, the number of customers entering a store).

- **Probability Mass Function (PMF):** Describes the probability that the random variable X takes on a specific value x, denoted as $P(X = x)$ or $p(x)$.

  $$P(X = x) \geq 0 \quad \forall x$$

  The sum of probabilities for all possible values must equal 1:

  $$\sum_{x} P(X = x) = 1$$

Continuous Random Variables

Continuous random variables take on values within a continuous range (e.g., a person's height, waiting time for a bus).

- **Probability Density Function (PDF):** Unlike a PMF, the probability that a continuous variable X takes on a *specific, exact* value is always 0 (e.g., $P(X = 170.0000\ldots) = 0$). Instead, the PDF $f(x)$ describes the probability density, allowing us to find the probability that X falls within a range $[a, b]$ by integrating:

  $$P(a \leq X \leq b) = \int_{a}^{b} f(x)dx$$

  The total area under the PDF curve must equal 1:

$$\int_{-\infty}^{\infty} f(x)dx = 1$$

Cumulative Distribution Function (CDF)

The CDF, denoted as $F(x)$, is the probability that the random variable X takes on a value less than or equal to x. It applies to both discrete and continuous variables:

$$F(x) = P(X \leq x)$$

---

2. Expectation, Variance & Moments

Expected Value (μ or $E[X]$)

The expected value is the average value we expect X to take if we run the experiment many times. It acts as the "center of mass" of the distribution.

- **Discrete Expected Value:**

$$E[X] = \sum_{x} x \cdot P(X = x)$$

- **Continuous Expected Value:**

$$E[X] = \int_{-\infty}^{\infty} x \cdot f(x)dx$$

Variance (σ^2 or $\mathrm{Var}(X)$)

Variance measures the spread or dispersion of the data around the expected value.

$$\mathrm{Var}(X) = E[(X - \mu)^2] = E[X^2] - (E[X])^2$$

Standard Deviation (σ) is the square root of the variance, which brings the unit back to the same scale as X.

Moments

- 1st Moment: Expected Value (Mean).
- 2nd Central Moment: Variance.
- 3rd Moment (Skewness): Measures the asymmetry of the distribution.
- 4th Moment (Kurtosis): Measures the “peakedness” and the heaviness of its tails.

3. Joint Distributions and Correlation

When working with multiple variables (e.g., having many features in Machine Learning), we use Joint Distributions.

- **Marginal Distribution:** The probability of one variable when we ignore (integrate/sum out) the other variables.
- **Covariance ($\mathrm{Cov}(X, Y)$):** Measures the tendency of two variables to change together.

$$\mathrm{Cov}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)]$$

- **Pearson Correlation Coefficient (ρ):** Normalizes covariance to the range $[-1, 1]$:

$$\rho = \frac{\mathrm{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

  (Note: Correlation does not imply causation).

4. Common Distributions in Data Science

Choosing the right distribution to model your data is a key step in Machine Learning and Statistical Modeling. Below are the most frequently encountered probability distributions.

4.1 Discrete Distributions

Discrete distributions model variables that take on distinct, countable values.

- **Bernoulli Distribution ($\mathrm{Bernoulli}(p)$):** Models a single trial with exactly two outcomes: Success (1) with probability p, and Failure (0) with probability $1 - p$.
  - *Parameters:* $p \in [0, 1]$ (success probability).
  - *PMF:* $P(X = x) = p^x (1 - p)^{1-x}$ for $x \in \{0, 1\}$.
  - *Mean / Variance:* $\mathbb{E}[X] = p$, $\mathrm{Var}(X) = p(1 - p)$.
  - *Example:* Flipping a coin, whether a user clicks an ad or not.
- **Binomial Distribution ($\mathrm{Bin}(n, p)$):** Models the number of successes in n independent Bernoulli trials.
  - *Parameters:* $n \in \mathbb{N}$ (number of trials), $p \in [0, 1]$.
  - *PMF:* $P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$.

- *Mean / Variance:* $\mathbb{E}[X] = np$, $\text{Var}(X) = np(1-p)$.
- *Example:* The number of clicks from 100 users visiting a website.

- **Poisson Distribution ($\text{Poisson}(\lambda)$):** Models the number of times an event occurs within a fixed interval of time or space.
  - *Parameters:* $\lambda > 0$ (average rate of occurrence).
  - *PMF:* $P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$.
  - *Mean / Variance:* $\mathbb{E}[X] = \lambda$, $\text{Var}(X) = \lambda$.
  - *Example:* Number of customer service calls received per hour.

4.2 Continuous Distributions

Continuous distributions model variables that take on any real value within a continuous range or interval.

- **Uniform Distribution ($\mathcal{U}(a, b)$):** All values within the continuous interval $[a, b]$ are equally likely to occur.
  - *Parameters:* a (lower bound), b (upper bound).
  - *PDF:* $f(x) = \frac{1}{b-a}$ for $x \in [a, b]$.
  - *Mean / Variance:* $\mathbb{E}[X] = \frac{a+b}{2}$, $\text{Var}(X) = \frac{(b-a)^2}{12}$.
  - *Example:* Random seed generation in model initialization.
- **Normal / Gaussian Distribution ($\mathcal{N}(\mu, \sigma^2)$):** The most important distribution in Data Science. It features a symmetric, bell-shaped curve centered around the mean.
  - *Parameters:* μ (mean), σ^2 (variance).
  - *PDF:* $f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$.
  - *Mean / Variance:* $\mathbb{E}[X] = \mu$, $\text{Var}(X) = \sigma^2$.
  - *Example:* Distribution of measurement errors, model residuals, or human heights.
- **Exponential Distribution ($\text{Exp}(\beta)$):** Models the time or distance between consecutive events in a Poisson process.
  - *Parameters:* $\beta > 0$ (rate parameter).
  - *PDF:* $f(x) = \beta e^{-\beta x}$ for $x \geq 0$.
  - *Mean / Variance:* $\mathbb{E}[X] = \frac{1}{\beta}$, $\text{Var}(X) = \frac{1}{\beta^2}$.
  - *Example:* The waiting time between phone calls at a call center.
- **Beta Distribution ($\text{Beta}(\alpha, \beta)$):** A continuous distribution defined on the interval $[0, 1]$, widely used in Bayesian inference to model the probability distribution of an *unknown* probability parameter.
  - *Parameters:* $\alpha > 0$ (success shape parameter), $\beta > 0$ (failure shape parameter).
  - *PDF:* $f(x) = \frac{x^{\alpha-1}(1-x)^{\beta-1}}{\text{B}(\alpha,\beta)}$, where B is the Beta function.
  - *Example:* Modeling the uncertainty of the click-through rate (CTR) of a new banner ad.

Visualization with Python (Scipy & Matplotlib)

Here is how to use the `scipy.stats` library to work with the Normal distribution.

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# Create X-axis data
x = np.linspace(-5, 5, 1000)

# PDF of the Normal distribution with mu=0, sigma=1 (Standard Normal)
pdf = norm.pdf(x, loc=0, scale=1)

# CDF of the Normal distribution
cdf = norm.cdf(x, loc=0, scale=1)

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(x, pdf, color='blue')
plt.title("Probability Density Function (PDF)")
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(x, cdf, color='red')
plt.title("Cumulative Distribution Function (CDF)")
plt.grid(True)
plt.show()

# Calculate probability P(X < 1.96)
prob = norm.cdf(1.96, loc=0, scale=1)
print(f"P(X < 1.96) = {prob:.4f}") # Output: 0.9750
```

Convergence & Limit Theorems

In Data Science and Machine Learning, we rarely have access to an entire "population". Instead, we collect a finite "sample" of size n.

Limit Theorems are the fundamental theorems of probability theory that describe how sample statistics (such as the sample mean $\bar{X}_n$) behave as the sample size approaches infinity ($n \to \infty$).

---

1. What Are Limit Theorems?

A Limit Theorem describes the asymptotic behavior of a sequence of random variables $(X_n)_{n=1}^{\infty}$.

In probability theory, there are two primary pillars of Limit Theorems:

- **Law of Large Numbers (LLN):** Governs the **value convergence** — establishing that as $n \to \infty$, the sample mean $\bar{X}_n$ converges to the true population expectation μ.
- **Central Limit Theorem (CLT):** Governs the **distribution shape convergence** — establishing that as $n \to \infty$, the normalized sum or sample mean converges in distribution to a standard Normal Distribution $\mathcal{N}(0, 1)$, regardless of the underlying distribution shape.

---

2. Modes of Convergence

To understand how a sequence of random variables X_n behaves as $n \to \infty$, mathematicians define three primary modes of convergence:

2.1 Convergence in Probability ($X_n \xrightarrow{P} X$)

A sequence X_n converges in probability to a random variable X if, as the sequence progresses, the probability that the difference between them is larger than any arbitrarily small positive number ϵ vanishes to zero.

Mathematical Definition:

$$\lim_{n \to \infty} P(|X_n - X| > \epsilon) = 0 \quad \forall \epsilon > 0$$

> **Machine Learning Intuition:** This forms the basis of the **Weak Law of Large Numbers (WLLN)**. It guarantees that as you collect more data points, the sample statistic (like the sample mean) will cluster closer and closer to the true population value, with the probability of a large error dropping to zero.

2.2 Almost Sure Convergence ($X_n \xrightarrow{a.s.} X$)

A sequence X_n converges almost surely (with probability 1) to X if the sequence of actual values realized by X_n converges to the realized value X for almost all outcomes.

Mathematical Definition:

$$P\left(\lim_{n \to \infty} X_n = X\right) = 1$$

> **Machine Learning Intuition:** This is a stronger form of convergence that underpins the **Strong Law of Large Numbers (SLLN)**. While convergence in probability states that large deviations become rare at any single step n, almost sure convergence guarantees that the entire path of sample statistics will eventually enter and never leave an arbitrarily small interval around the true mean.

2.3 Convergence in Distribution ($X_n \xrightarrow{d} X$)

A sequence X_n converges in distribution to X if their Cumulative Distribution Functions (CDFs) converge to the CDF of X at all points where the limiting CDF is continuous.

Mathematical Definition:

$$\lim_{n \to \infty} F_{X_n}(x) = F_X(x) \quad \forall x \text{ where } F_X \text{ is continuous}$$

> **Machine Learning Intuition:** This is the weakest form of convergence and serves as the mathematical foundation for the **Central Limit Theorem (CLT)**. It doesn't mean the individual sample values become equal, but rather that the overall shape of the histogram of these values settles into a specific probability distribution (like the Gaussian bell curve).

2.4 Hierarchy of Convergence

The relationship and strength between these three modes of convergence can be ordered hierarchically:

$$X_n \xrightarrow{a.s.} X \implies X_n \xrightarrow{P} X \implies X_n \xrightarrow{d} X$$

(Almost Sure Convergence is the strongest mode, implying Convergence in Probability, which in turn implies Convergence in Distribution. The reverse implications do not hold in general without additional conditions).

---

3. Law of Large Numbers (LLN)

The Law of Large Numbers (LLN) is a fundamental theorem in probability. It states that as the number of independent trials increases, the empirical average (observed average) of the results converges to the theoretical expected value.

For example, when rolling a fair 6-sided die, the theoretical expected value (mean) of a roll is:

$$\mathbb{E}[X] = \frac{1+2+3+4+5+6}{6} = 3.5$$

If you roll the die 10 times, your average roll might be 2.1 or 4.8 (high variance). However, if you roll it 10,000 times, the average will be extremely close to 3.5 as $n \to \infty$.

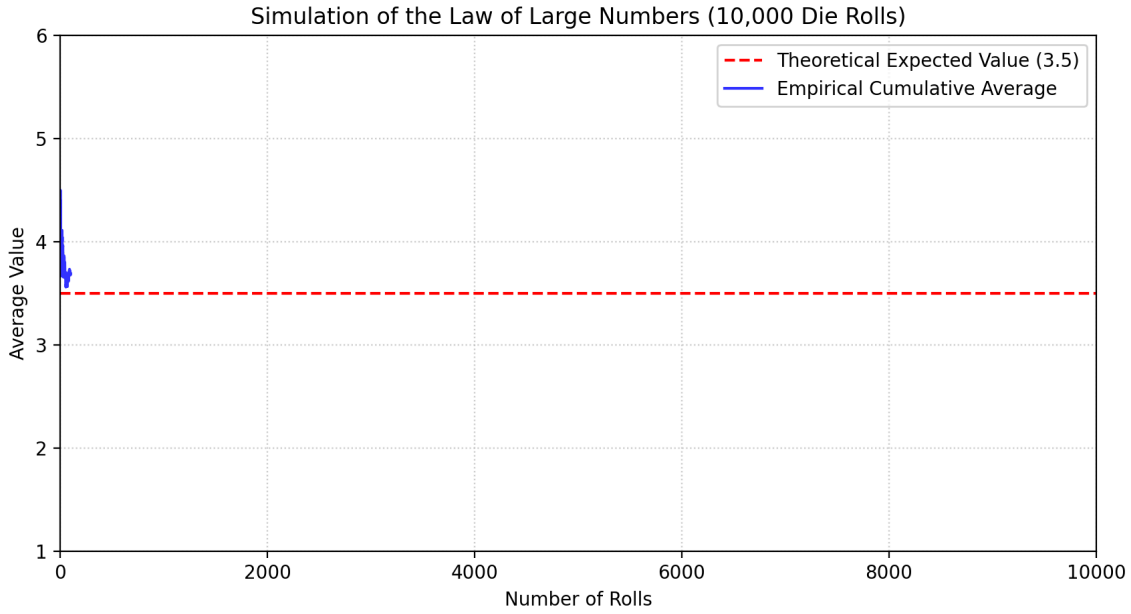


Fig. 15

Simulation demonstrating the Law of Large Numbers.

Why LLN Matters in Data Science

- **Guarantees Sample Reliability:** Collecting more data increases estimation precision.
- **Foundation of Machine Learning:** Ensures that training model parameters on finite sample datasets generalizes to true population distributions.

Python Simulation: Roll Average Comparison

```python
import numpy as np

# Theoretical expected value for a fair die roll
expected_value = 3.5

for rolls in [10, 100, 1000, 10000, 100000]:
    # Simulate die rolls (1 to 6)
    simulated_rolls = np.random.randint(1, 7, size=rolls)
    empirical_average = np.mean(simulated_rolls)
    difference = abs(empirical_average - expected_value)
    print(f"Rolls: {rolls:<6} | Average: {empirical_average:.4f} | Difference: {difference:.4f}")
```

Python Simulation: Cumulative Convergence Curve

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
N = 1000
rolls = np.random.randint(1, 7, size=N)
sample_means = np.cumsum(rolls) / np.arange(1, N + 1)

plt.figure(figsize=(9, 4.5))
plt.plot(sample_means, label=r'Sample Mean $\bar{X}_n$')
plt.axhline(y=3.5, color='r', linestyle='--', label=r'True Mean $\mu = 3.5$')
plt.xlabel('Number of rolls (n)')
plt.ylabel('Sample Mean')
plt.title('Law of Large Numbers Convergence')
plt.legend()
plt.grid(True)
plt.show()
```

---

4. Central Limit Theorem (CLT)

While LLN explains *what value* the sample mean converges to, the Central Limit Theorem (CLT) explains the *shape of the distribution* of sample means.

Theorem Statement

Given independent and identically distributed (i.i.d) random variables $X_1, X_2, \ldots, X_n$ with finite mean μ and variance σ^2, as $n \to \infty$ (typically $n \geq 30$):

$$\bar{X}_n \sim \mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right)$$

Or in standardized form:

$$Z = \frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} \mathcal{N}(0,1)$$

Why CLT is the “Mathematical Miracle” of Statistics

- **Distribution Agnostic:** The original population can be uniform, exponential, or skewed. The distribution of sample means $\bar{X}_n$ will always form a Gaussian bell curve.
- **Enables Hypothesis Testing:** Provides the foundation for confidence intervals, z-tests, t-tests, and A/B testing in Data Science.

Python Simulation: CLT

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

n_samples = 50
n_trials = 10000

# Generate Uniform random matrix (10,000 trials x 50 samples)
data = np.random.uniform(0, 1, size=(n_trials, n_samples))
sample_means = np.mean(data, axis=1)

plt.figure(figsize=(8, 4.5))
sns.histplot(sample_means, kde=True, color='blue', bins=50)
plt.title(f'CLT: Distribution of Sample Means (n={n_samples})')
plt.xlabel('Sample Mean')
plt.ylabel('Density')
plt.grid(True)
plt.show()
```

Information Theory Basics

Although originating from telecommunications engineering (by Claude Shannon), Information Theory has incredibly deep mathematical connections to Probability. In modern Deep Learning and Machine Learning, concepts from information theory are used as the default **Loss functions** for almost all Classification problems.

This chapter will visually explain Entropy, Cross-Entropy, and KL Divergence.

1. Information Content

Imagine you are waiting for the outcome of an event.

- If that event is **100% certain to happen** (e.g., "The sun rises in the East"), when it actually happens, you gain no new information.
- Conversely, if that event is **very rare** (e.g., "Winning the lottery jackpot"), when it happens, you receive a massive amount of information.

Mathematically, the Information Content (I) of an event x (with probability $P(x)$) is defined as inversely proportional to its probability:

$$I(x) = -\log_2(P(x))$$

*Note: We use base 2 to measure in units of **bits** (as in computers).*

2. Equivalence of Probability and Information

There is a direct mathematical equivalence between probability, uncertainty, and information. Whenever we reduce uncertainty about an outcome by eliminating alternative possibilities, we are gaining information.

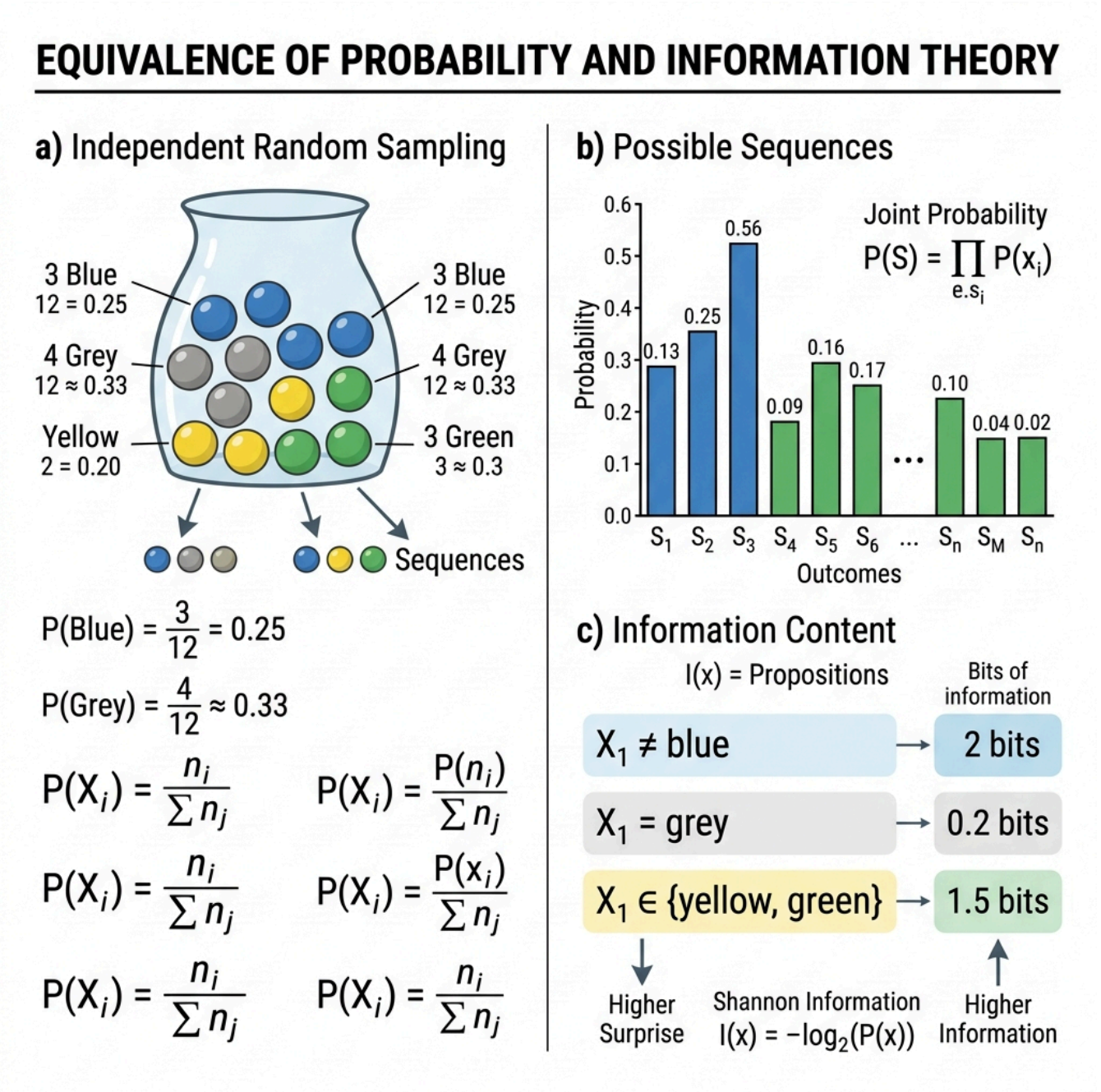


Fig. 16

Visualizing the equivalence of probability reduction and information gain.

As illustrated in the diagram above:

- **a) Independent Random Sampling:** Consider drawing colored marbles from an urn with replacement. The probabilities are:

$$p(\text{blue}) = \frac{1}{2}, \quad p(\text{grey}) = \frac{1}{4}, \quad p(\text{yellow}) = \frac{1}{8}, \quad p(\text{green}) = \frac{1}{8}$$

- **b) 2-Sequences Distribution:** Shows the combined joint probability distribution of drawing two marbles consecutively (X_1, X_2), yielding 16 possible outcomes.
- **c) Information Content of Propositions:** By learning a proposition about the two draws is true, we eliminate certain possible outcomes, reducing the probability mass and gaining information:
  - **Proposition $X_1 \neq \text{blue}, X_2 \neq \text{blue}$:** Eliminates all sequences containing blue marbles. The remaining probability mass is $\frac{8}{32}$ (a 75% reduction), yielding exactly **2 bits** of information.
  - **Proposition $X_1 \neq \text{green}$:** Eliminates green outcomes. The remaining probability mass is $\frac{28}{32}$, yielding only **0.2 bits** of information (since green was already rare).
  - **Proposition $X_1 = X_2$:** Learning that both marbles have the same color has a probability of $\frac{11}{32}$, yielding **1.5 bits** of information.

Core Rule: Eliminating probability mass, reducing uncertainty, and gaining information are mathematically equivalent. A **50% reduction** in the remaining probability mass corresponds to exactly **1 bit** of information gained.

---

3. Entropy (Uncertainty / Chaos)

While $I(x)$ only measures the information content of a *single* specific event, **Entropy (H)** measures the **expected** information content of the *entire probability distribution*. In other words, Entropy measures the level of "uncertainty" or "chaos" of a random variable X.

$$H(X) = E[I(X)] = -\sum_{x} P(x) \log_2(P(x))$$

> **Note**
>
> **Mathematical Edge Cases & Practical Notes:**
>
> - **Convention for $0 \log_2 0$:** By convention, $0 \log_2(0) = 0$ because $\lim_{p \to 0^+} p \log_2(p) = 0$.
> - **Base 2 vs. Natural Log:**
>   - Base 2 ($\log_2$) measures information in **bits** (standard in pure Information Theory).
>   - Natural log ($\ln$) measures information in **nats** (standard in Machine Learning frameworks like PyTorch and TensorFlow for loss computations).

- **Low Entropy:** The system is very predictable (high certainty). Example: A rigged coin that always lands on Heads ($P(H) = 1, P(T) = 0$). In this case, $H(X) = 0$.
- **High Entropy:** The system is completely chaotic and most unpredictable. Example: A fair coin ($P(H) = 0.5, P(T) = 0.5$). In this case, $H(X)$ reaches its maximum.

---

4. Cross-Entropy

Cross-Entropy answers the question: *If the true data follows distribution P, but we assume it follows an approximate distribution Q, how many bits on average do we need to encode the information?*

$$H(P, Q) = -\sum_{x} P(x) \log_2(Q(x))$$

In Machine Learning:

- P **(True distribution):** The actual distribution of the data (usually ground-truth labels, e.g., `[1, 0, 0]` for a picture of a dog).
- Q **(Predicted distribution):** The distribution predicted by the AI model (e.g., `[0.7, 0.2, 0.1]`).

The goal of an ML model is to make Q as similar to P as possible. When $Q = P$, Cross-Entropy reaches its minimum value (which is exactly the Entropy of P). Therefore, **Cross-Entropy Loss** is used as the loss function to optimize Neural Networks in Classification problems.

---

5. Kullback-Leibler (KL) Divergence (Distribution Distance)

KL Divergence (KLD) measures the **difference** between two probability distributions P and Q. It answers the question: *By using Q instead of P, how much extra information content are we wasting?*

$$D_{KL}(P||Q) = H(P, Q) - H(P)$$

$$D_{KL}(P||Q) = \sum_{x} P(x) \log_2 \left(\frac{P(x)}{Q(x)} \right)$$

Important Properties:

- $D_{KL}(P||Q) \geq 0$. It is only equal to 0 when P and Q are identical.
- It is not symmetric: $D_{KL}(P||Q) \neq D_{KL}(Q||P)$. Therefore, it is not a true "distance metric" in a geometric sense.

In Machine Learning (especially Generative AI like VAEs - Variational Autoencoders or t-SNE), KLD is used as a Loss function to force the predicted distribution Q (latent space) to closely match a standard distribution P.

Illustration with Python

How to calculate Entropy and KL Divergence using Numpy and Scipy.

```python
import numpy as np
from scipy.stats import entropy

# Distribution P (actual - ground truth)
P = np.array([0.8, 0.15, 0.05])

# Distribution Q (prediction from model 1 - quite good)
Q1 = np.array([0.7, 0.2, 0.1])

# Distribution Q (prediction from model 2 - very bad)
Q2 = np.array([0.2, 0.3, 0.5])

# 1. Calculate Entropy of P (base 2)
# entropy() in scipy defaults to natural log (ln), we change the base with base=2
H_P = entropy(P, base=2)
print(f"Entropy H(P): {H_P:.4f} bits")

# 2. Calculate KL Divergence: D_KL(P || Q1) and D_KL(P || Q2)
kl_1 = entropy(P, qk=Q1, base=2)
kl_2 = entropy(P, qk=Q2, base=2)
print(f"KL Divergence (P || Q1): {kl_1:.4f} bits (Good model, small KLD)")
print(f"KL Divergence (P || Q2): {kl_2:.4f} bits (Bad model, large KLD)")

# 3. Calculate Cross-Entropy H(P, Q) = H(P) + D_KL(P || Q)
ce_1 = H_P + kl_1
print(f"Cross-Entropy H(P, Q1): {ce_1:.4f} bits")
```

Statistics

Statistics is the discipline that concerns the collection, organization, analysis, interpretation, and presentation of data. While probability helps us predict future outcomes based on a known model, statistics works in reverse: we analyze observed data to infer the properties of the underlying system.

---

1. Descriptive vs. Inferential Statistics

- **Descriptive Statistics**: Summarizes and describes the characteristics of a dataset (e.g., mean, median, standard deviation).
- **Inferential Statistics**: Uses a sample of data to make predictions or draw conclusions about a larger population.

---

2. Descriptive Statistics in Python

Mean, Median, and Mode

- **Mean**: The arithmetic average.
- **Median**: The middle value when data is sorted. Highly resistant to outliers.
- **Mode**: The most frequent value.

Variance and Standard Deviation

Variance (σ^2) and Standard Deviation (σ) measure the spread of data points around the mean.

- **Population Variance**:

$$\sigma^2 = \frac{\sum(x_i - \mu)^2}{N}$$

- **Sample Variance** (Bessel's correction uses $n - 1$ to account for sample bias):

$$s^2 = \frac{\sum(x_i - \bar{x})^2}{n - 1}$$

```
import numpy as np
from scipy import stats

data = [12, 15, 12, 18, 22, 15, 14, 15, 90]  # Note the outlier: 90

mean = np.mean(data)
median = np.median(data)
mode = stats.mode(data, keepdims=True).mode[0]

# Sample Standard Deviation (ddof=1)
std_dev = np.std(data, ddof=1)

print(f"Mean              : {mean:.2f}")
print(f"Median            : {median:.2f}")
print(f"Mode              : {mode}")
print(f"Sample Std Dev    : {std_dev:.2f}")
```

Output:

```
Mean              : 22.56
Median            : 15.00
Mode              : 15
Sample Std Dev    : 25.56
```

---

3. The Normal Distribution

The normal (or Gaussian) distribution is a continuous symmetric bell-shaped curve defined by its mean (μ) and standard deviation (σ).

Z-score

A Z-score indicates how many standard deviations a value is from the mean:

$$Z = \frac{x - \mu}{\sigma}$$

Python Implementation of Normal CDF and Percentiles

```
from scipy.stats import norm

# Standard Normal Distribution (mean=0, std=1)
# What percentage of data falls below Z = 1.96?
pct_below_z = norm.cdf(1.96)
print(f"Percent below Z=1.96: {pct_below_z*100:.2f}%")

# Inverse CDF: What Z-score marks the 95th percentile?
z_95th = norm.ppf(0.95)
print(f"Z-score at 95th percentile: {z_95th:.4f}")
```

Output:

```
Percent below Z=1.96: 97.50%
Z-score at 95th percentile: 1.6449
```

---

4. Central Limit Theorem (CLT)

The **Central Limit Theorem** states that if you take sufficiently large random samples from any population (even a non-normal one), the distribution of the sample means will approach a normal distribution as the sample size increases.

This allows us to perform statistical inference (like hypothesis testing) on non-normal data!

---

5. Confidence Intervals

A confidence interval provides a range of values that is likely to contain the true population parameter.

The formula for the confidence interval of a mean is:

$$\text{CI} = \bar{x} \pm Z \cdot \frac{s}{\sqrt{n}}$$

```
import numpy as np
from scipy import stats

sample_data = [12, 15, 12, 18, 22, 15, 14, 15, 19, 20]
n = len(sample_data)
mean = np.mean(sample_data)
sem = stats.sem(sample_data)  # Standard Error of the Mean = s / sqrt(n)

# 95% Confidence Interval
ci_95 = stats.t.interval(0.95, df=n-1, loc=mean, scale=sem)
print(f"95% Confidence Interval for the mean: {ci_95[0]:.2f} to {ci_95[1]:.2f}")
```

Output:

```
95% Confidence Interval for the mean: 13.91 to 18.49
```

---

6. Hypothesis Testing & P-Values

Hypothesis testing is a systematic way to evaluate claims about a population.

- **Null Hypothesis (H_0)**: The status quo; there is no effect or no difference.
- **Alternative Hypothesis (H_1)**: The claim we want to test.
- **P-value**: The probability of obtaining test results at least as extreme as the observed results, assuming H_0 is true. If $p \leq 0.05$ (the significance level α), we reject H_0.

Example: One-Sample T-test

A manufacturer claims their lightbulbs last 1000 hours on average. We test 10 bulbs and get the following lifespans:

```
from scipy import stats

lifespans = [980, 1010, 990, 970, 1005, 985, 995, 960, 1020, 980]

# Perform one-sample t-test against H0: mean = 1000
t_stat, p_val = stats.ttest_1samp(lifespans, 1000)

print(f"T-statistic: {t_stat:.4f}")
print(f"P-value    : {p_val:.4f}")

if p_val < 0.05:
    print("Reject H0: The mean lifespan is significantly different from 1000 hours.")
else:
    print("Fail to reject H0: No significant difference from 1000 hours.")
```

Output:

```
T-statistic: -1.7454
P-value    : 0.1149
Fail to reject H0: No significant difference from 1000 hours.
```

---

7. Z-Test vs. T-Test

When performing hypothesis tests on a sample mean, you must choose between a Z-test and a T-test:

Feature	Z-Test	T-Test
Sample Size	Large ($n \geq 30$)	Small ($n < 30$)
Population Variance (σ^2)	Known	Unknown (must use sample variance s^2)
Underlying Distribution	Normal Distribution	Student's T-Distribution (wider tails to account for uncertainty)

---

8. Type I and Type II Errors

When testing hypothesis, decisions are made under uncertainty. This can lead to two types of errors:

- **Type I Error (α / False Positive)**: Rejecting the null hypothesis (H_0) when it is actually true. *Example*: A spam filter classifying a legitimate email as spam.
- **Type II Error (β / False Negative)**: Failing to reject the null hypothesis (H_0) when it is actually false. *Example*: A spam filter letting a malicious email pass through to the inbox.

---

9. Chi-Square Test (χ^2) for Categorical Data

A **Chi-Square Test** is used to determine whether there is a statistically significant association between two categorical variables.

For instance, suppose we want to test if a customer's subscription plan preference (Basic, Premium) is dependent on their region (US, EU):

```python
import numpy as np
from scipy.stats import chi2_contingency

# Contingency table (observed counts)
# Rows: US, EU
# Columns: Basic, Premium
observed = np.array([
    [150, 100],  # US
    [120, 130]   # EU
])

# Perform Chi-Square Test
chi2_stat, p_val, dof, expected = chi2_contingency(observed)

print(f"Chi2 statistic : {chi2_stat:.4f}")
print(f"P-value        : {p_val:.4f}")

if p_val < 0.05:
    print("Subscription preferences are significantly associated with region.")
else:
    print("Subscription preferences are independent of region.")
```

Exercises

Exercise 1

You have a dataset of house prices: `[250, 270, 310, 290, 850, 300, 260]` (in thousands of dollars).

1. Calculate the mean and median.
2. Which measure is more appropriate for summarizing this dataset, and why?

Exercise 2

A sample of 50 students scored an average of 78 on an exam, with a sample standard deviation of 8. Compute the 95% confidence interval for the population mean.

Exercise 3

An online store wants to see if changing the color of the “Buy” button increases conversions.

- Control group (Red): 100 purchases out of 1000 visits.
- Treatment group (Green): 130 purchases out of 1000 visits. Perform a two-sample z-test for proportions to see if the green button performs significantly better ($\alpha = 0.05$).

Solution — Exercise 3

```python
from statsmodels.stats.proportion import proportions_ztest
import numpy as np

count = np.array([130, 100]) # successes
nobs = np.array([1000, 1000]) # observations

# Perform z-test
stat, pval = proportions_ztest(count, nobs, alternative='larger')
print(f"Z-statistic: {stat:.4f}")
print(f"P-value    : {pval:.5f}")
```

Statistical Inference

While Probability goes from a general mathematical model to predict data (Model $\rightarrow$ Data), **Statistical Inference** does the reverse: it goes from collected data (Sample) to predict and draw conclusions about the general model of the real world (Data $\rightarrow$ Model).

This is the foundation of the training process in Machine Learning: relying on a finite training dataset to find the best parameters for a model.

1. Point Estimation

The goal of point estimation is to find a single value (called $\hat{\theta}$) that best approximates the true population parameter θ based on sample data.

Maximum Likelihood Estimation (MLE)

MLE is the most common method in Machine Learning. The core idea: **"Choose the parameter θ such that the probability of observing the current dataset is maximized."**

The Likelihood function $L(\theta)$ is the probability of the data X given the parameter θ. If the data points x_i are independent and identically distributed (i.i.d): $$L(\theta) = P(X|\theta) = \prod_{i=1}^{n} P(x_i|\theta)$$

For ease of computation (to avoid underflow and turn products into sums), we typically use **Log-Likelihood**: $$\log L(\theta) = \sum_{i=1}^{n} \log P(x_i|\theta)$$

The goal of MLE is to find θ to maximize this Log-Likelihood function. *(Note: In Deep Learning, minimizing the Cross-Entropy Loss is exactly performing MLE!)*.

Maximum A Posteriori (MAP)

MAP is an extension of MLE based on Bayes' theorem, allowing us to incorporate our **initial belief (Prior - $P(\theta)$)** into the estimation process. $\hat{\theta}_{MAP} = \arg\max_\theta P(X|\theta)P(\theta)$ *(Note: Adding Regularization (L1/L2) in Machine Learning is exactly performing MAP estimation where the Prior is a Laplace/Gaussian distribution, respectively)*.

2. Confidence Intervals (CI)

A point estimate (like MLE) only gives us a single number, but it doesn't tell us the certainty of that number. A **Confidence Interval** provides a range of values within which we believe the true parameter θ lies with a certain level of confidence (e.g., 95%).

Suppose we calculate the sample mean $\bar{X}$ and the sample standard deviation S. Based on the Central Limit Theorem (CLT), the 95% confidence interval for the population mean μ is: $CI = \bar{X} \pm 1.96\frac{S}{\sqrt{n}}$ (The number 1.96 is the z-score corresponding to 95% of the area under the Normal distribution curve).

3. Hypothesis Testing and p-value

Hypothesis testing is the backbone of scientific research and **A/B Testing** in the industry (e.g., Does Interface A or B bring more clicks?).

The basic steps:

1. **Establish H_0 (Null Hypothesis):** The default assumption (usually "No difference", "No effect").
2. **Establish H_A (Alternative Hypothesis):** What we want to prove ("There is a difference", "The drug has an effect").
3. **Calculate the Test Statistic:** A statistical metric derived from the sample data (e.g., t-score, z-score).
4. **Calculate the p-value:** The probability of obtaining a result equal to (or more extreme than) the current sample data, *assuming that H_0 is true*.
5. **Conclusion:** If the p-value is smaller than a threshold α (usually 0.05), we reject H_0 and accept H_A (The result is Statistically significant).

Careful: The p-value is **not** the probability that H_0 is true. It is only the probability of observing this data if H_0 were true.

Common types of tests:

- **T-test:** Used to compare the means of 1 or 2 groups (when the population variance is unknown or the sample size is small). Frequently used in A/B testing.
- **ANOVA:** Used to compare the means of 3 or more groups.
- **Chi-square Test:** Used for categorical data to check for independence between categorical variables.

Python Example (T-test for A/B Testing)

Suppose we run an A/B Test for two buttons on a website, recording the time users stay on the page (in seconds).

```
import numpy as np
from scipy import stats

np.random.seed(42)
# Interface A (Control)
time_A = np.random.normal(loc=120, scale=30, size=100)
# Interface B (Treatment) - Seems users stay slightly longer
time_B = np.random.normal(loc=130, scale=30, size=100)

# Perform an Independent 2-sample t-test
t_stat, p_value = stats.ttest_ind(time_B, time_A, alternative='greater')

print(f"T-statistic: {t_stat:.4f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05
if p_value < alpha:
    print("Reject H0: Interface B genuinely retains users longer than interface A (Statistically
significant).")
else:
    print("Not enough evidence to reject H0: The difference might just be due to random chance.")

# Output:
# T-statistic: 2.2223
# P-value: 0.0137
# Reject H0: Interface B genuinely retains users longer than interface A (Statistically
significant).
```

Stochastic Processes

Up until now, we have looked at static random variables (independent of time). However, in the real world, many systems change continuously over time in a random manner: stock prices, the weather, the position of a pollen grain in water, or the sequence of words generated by ChatGPT.

A **Stochastic Process** is a collection of random variables indexed by time: $\{X_t, t \in T\}$.

This chapter introduces the two most important stochastic process models in Machine Learning: Markov Chains and Random Walks.

1. Markov Chains

A Markov Chain is a discrete-time stochastic process that possesses a highly specific property called the **Markov Property**:

> "The future depends only on the present, not on the past."

Mathematically, the probability that the system transitions to state x_{t+1} tomorrow depends only on the state x_t of today, regardless of whatever states the system went through previously: $
$P(X_{t+1} = x_{t+1}|X_t = x_t, X_{t-1} = x_{t-1}, \dots, X_0 = x_0) = P(X_{t+1} = x_{t+1}|X_t = x_t)$

Transition Matrix

If a system has N states, we can describe a Markov Chain using an $N \times N$ square matrix P, where the element P_{ij} is the probability of transitioning from state i to state j.

Property: The sum of probabilities in each row is always equal to 1. $\sum_j P_{ij} = 1$

Applications in AI

- **Natural Language Processing (NLP):** Classic language models (n-grams) that predict the next word based on the current word(s) are essentially an application of Markov Chains.
- **Reinforcement Learning (RL):** The Markov Decision Process (MDP) is the mathematical foundation of RL, where an Agent learns to interact with an environment to maximize rewards.
- **PageRank (Google):** Google's original web page ranking algorithm simulated a user randomly clicking on web links (a Markov Chain) to figure out which page had the highest probability of being visited.

2. Random Walk

A Random Walk is a special case of a Markov Chain, describing a path consisting of a succession of random steps.

The simplest example is the 1-Dimensional Random Walk: Start at $X_0 = 0$. At each time step t, you flip a coin:

- If Heads: Take one step to the right ($+1$).
- If Tails: Take one step to the left (-1).

Your position at step t is: $$X_t = X_{t-1} + Z_t = \sum_{i=1}^{t} Z_i (Where Z_i \in \{-1, 1\}$$).

Interesting Properties

- The expected value of your position is always 0: $E[X_t] = 0$ (you don't have a tendency to drift to either side).
- However, the actual distance (measured by variance) increases over time: $Var(X_t) = t$. This means the longer you walk, the more likely you are to drift further away from the starting point (The standard deviation is proportional to $\sqrt{t}$).

Applications

- **Finance:** The Random Walk Theory suggests that stock prices move randomly and cannot be predicted based on past history (Efficient Market Hypothesis).
- **Physics:** Brownian motion describes the random movement of particles suspended in a fluid.

Visualizing a Random Walk with Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# Simulate 3 Random Walks, each consisting of 1000 steps
n_steps = 1000
n_walks = 3

# Generate random steps +1 or -1
steps = np.random.choice([-1, 1], size=(n_steps, n_walks))

# Calculate the cumulative position
positions = np.cumsum(steps, axis=0)

# Add the starting position 0 at the beginning
positions = np.vstack([np.zeros((1, n_walks)), positions])

# Plot the graph
plt.figure(figsize=(10, 5))
plt.plot(positions)
plt.title("Simulating a 1D Random Walk")
plt.xlabel("Time (t)")
plt.ylabel("Position (X_t)")
plt.grid(True)
plt.show()
```

Markov Chains

A **Markov Chain** is a stochastic process that models a sequence of events where the probability of each event depends only on the state attained in the previous event. In Machine Learning, Markov Chains are the foundation for models like Markov Decision Processes (MDPs) in Reinforcement Learning, Hidden Markov Models (HMMs) in speech processing, and PageRank in web search.

---

1. The Markov Property

A discrete-time stochastic process $\{X_0, X_1, X_2, \ldots\}$ has the **Markov Property** (or memoryless property) if the conditional probability distribution of future states depends only upon the present state, not on the path of past states:

$$P(X_{t+1} = x_{t+1} \mid X_t = x_t, X_{t-1} = x_{t-1}, \ldots, X_0 = x_0) = P(X_{t+1} = x_{t+1} \mid X_t = x_t)$$

In plain terms: *"The future depends only on the present, not on the past."*

---

2. Transition Matrix (P)

If a system has n distinct states, we define the probability of transitioning from state i to state j as P_{ij}. We can group these probabilities into an $n \times n$ matrix called the **Transition Matrix (or Stochastic Matrix)**:

$$P = \begin{bmatrix} P_{11} & P_{12} & \dots & P_{1n} \\ P_{21} & P_{22} & \dots & P_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ P_{n1} & P_{n2} & \dots & P_{nn} \end{bmatrix}$$

Important Properties of P

- **Non-negativity:** Every element is a probability, so $P_{ij} \geq 0$.
- **Sum of Outgoing Probabilities:** The sum of probabilities in each row (representing transitions from a given state to all possible next states, including itself) must equal 1:

$$\sum_{j} P_{ij} = 1 \quad \forall i$$

---

3. State Transition Diagrams

A Markov Chain can be represented as a directed graph where nodes are states and edges represent transition probabilities.

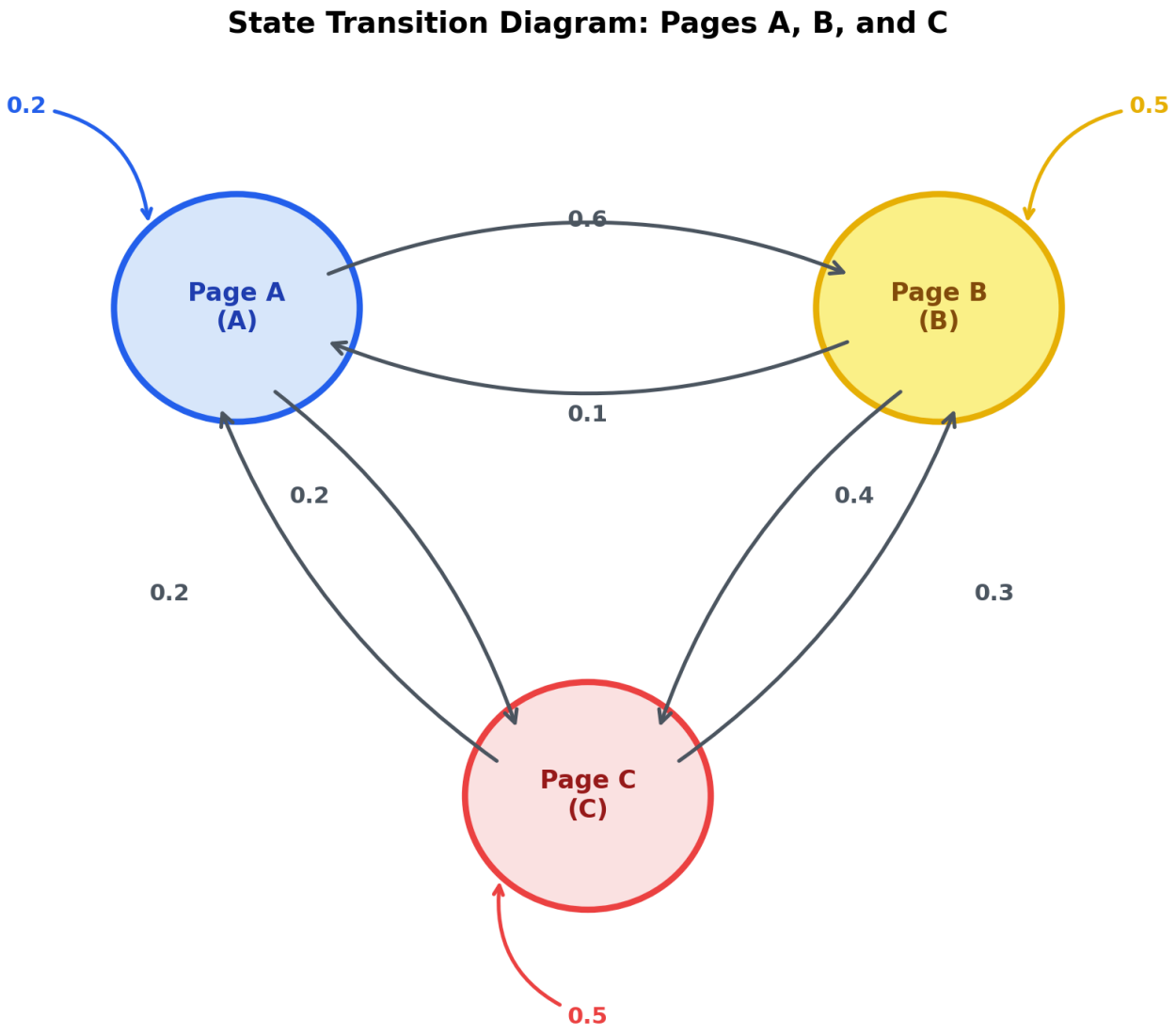


Fig. 17

State transition diagram for a three-state web browsing model (Pages A, B, and C).

In the web browsing model above, the transition matrix is defined as:

$$P = \begin{bmatrix} P_{AA} & P_{AB} & P_{AC} \\ P_{BA} & P_{BB} & P_{BC} \\ P_{CA} & P_{CB} & P_{CC} \end{bmatrix} = \begin{bmatrix} 0.2 & 0.6 & 0.2 \\ 0.1 & 0.5 & 0.4 \\ 0.2 & 0.3 & 0.5 \end{bmatrix}$$

- **Rows** represent the starting page.
- **Columns** represent the destination page.
- **Sum Rule check (Row A):** $P_{AA} + P_{AB} + P_{AC} = 0.2 + 0.6 + 0.2 = 1.0$.

4. State Vector Update over Time

Let $\pi^{(t)}$ be a row vector representing the probability distribution of the system's states at time t:

$$\pi^{(t)} = [P(A) \quad P(B) \quad P(C)]$$

To compute the probability distribution at the next time step, we perform a vector-matrix multiplication:

$$\pi^{(t+1)} = \pi^{(t)} \cdot P$$

By induction, the state probability vector after k steps is:

$$\pi^{(k)} = \pi^{(0)} \cdot P^k$$

Multi-Step Calculation Example

Suppose a user starts browsing and is definitely on **Page B** at $t = 0$:

$$\pi^{(0)} = [0 \quad 1 \quad 0]$$

- **At step 1 ($t = 1$):**

$$\pi^{(1)} = \pi^{(0)} P = [0 \quad 1 \quad 0] \begin{bmatrix} 0.2 & 0.6 & 0.2 \\ 0.1 & 0.5 & 0.4 \\ 0.2 & 0.3 & 0.5 \end{bmatrix} = [0.1 \quad 0.5 \quad 0.4]$$

---

5. Stationary Distribution (π)

As $t \to \infty$, many Markov Chains reach a steady state where the probability distribution across states no longer changes. This limiting distribution is called the **Stationary Distribution (π)**:

$$\pi P = \pi$$

Or represented as a linear system:

$$[\pi_A \quad \pi_B \quad \pi_C] \begin{bmatrix} 0.2 & 0.6 & 0.2 \\ 0.1 & 0.5 & 0.4 \\ 0.2 & 0.3 & 0.5 \end{bmatrix} = [\pi_A \quad \pi_B \quad \pi_C]$$

Analytical Solution

To find a unique probability vector, we must combine the system of equations with the **Normalization Constraint** (since the sum of all state probabilities must equal 1):

$$\pi_A + \pi_B + \pi_C = 1 \quad \text{and} \quad \pi_i \geq 0$$

Expanding the matrix multiplication $\pi P = \pi$, we get:

1. $0.2\pi_A + 0.1\pi_B + 0.2\pi_C = \pi_A \implies 0.1\pi_B + 0.2\pi_C = 0.8\pi_A \implies \pi_B + 2\pi_C = 8\pi_A$
2. $0.6\pi_A + 0.5\pi_B + 0.3\pi_C = \pi_B \implies 0.6\pi_A + 0.3\pi_C = 0.5\pi_B$

From (1), we express $\pi_B = 8\pi_A - 2\pi_C$. Substituting this into (2):

$$6\pi_A + 3\pi_C = 5(8\pi_A - 2\pi_C) \implies 13\pi_C = 34\pi_A \implies \pi_C = \frac{34}{13}\pi_A$$

Substituting π_C back to find π_B:

$$\pi_B = 8\pi_A - 2\left(\frac{34}{13}\pi_A\right) = \frac{36}{13}\pi_A$$

Using the normalization constraint $\pi_A + \pi_B + \pi_C = 1$:

$$\pi_A + \frac{36}{13}\pi_A + \frac{34}{13}\pi_A = 1 \implies \pi_A\left(\frac{83}{13}\right) = 1 \implies \pi_A = \frac{13}{83} \approx 0.1566$$

Solving for the rest of the states:

$$\pi_B = \frac{36}{83} \approx 0.4337, \quad \pi_C = \frac{34}{83} \approx 0.4096$$

Thus, the stationary distribution is:

$$\pi = [0.1566 \quad 0.4337 \quad 0.4096]$$

> **Note**
>
> **Note on Convergence (Ergodicity):** A Markov Chain is guaranteed to converge to a *unique* stationary distribution regardless of the initial starting state $\pi^{(0)}$ if the chain is **Ergodic**. An ergodic Markov Chain must be:
>
> - **Irreducible:** It is possible to get from any state to any other state (all states communicate).
> - **Aperiodic:** The return times to any state do not follow a fixed periodic cycle.

6. Eigenvalues and Eigenvectors Connection

In Linear Algebra, the eigenvector equation for a square matrix M is:

$$vM = \lambda v$$

Comparing this to our stationary distribution equation $\pi P = \pi$, we find that:

- The stationary distribution vector π is the **Left Eigenvector** of the transition matrix P.
- The corresponding **Eigenvalue** is $\lambda = 1$.

In any stochastic transition matrix, the largest eigenvalue is always 1, guaranteeing the existence of a stationary distribution.

---

7. Python Implementation (NumPy)

We can simulate state transitions and solve for the stationary distribution using Python:

```
import numpy as np

# Define Transition Matrix P
P = np.array([
    [0.2, 0.6, 0.2],
    [0.1, 0.5, 0.4],
    [0.2, 0.3, 0.5]
])

# 1. Simulation over 15 steps starting from Page B: [0.0, 1.0, 0.0]
v = np.array([0.0, 1.0, 0.0])
for step in range(1, 16):
    v = v.dot(P)
    if step in [1, 2, 5, 10, 15]:
        print(f"Step {step:02d} Probabilities: A = {v[0]:.4f}, B = {v[1]:.4f}, C = {v[2]:.4f}")

# 2. Solving for Stationary Distribution analytically using Eigenvectors
# Since \pi P = \pi is equivalent to P^T \pi^T = \pi^T,
# \pi^T is the right eigenvector of P^T associated with eigenvalue 1.
eigenvalues, eigenvectors = np.linalg.eig(P.T)

# Find the eigenvector corresponding to eigenvalue 1.0
idx = np.argmin(np.abs(eigenvalues - 1.0))
stationary = eigenvectors[:, idx].real

# Normalize the eigenvector to sum to 1
stationary = stationary / np.sum(stationary)
print(f"\nStationary Distribution: A = {stationary[0]:.4f}, B = {stationary[1]:.4f}, C =
{stationary[2]:.4f}")
```

---

8. Application: Google PageRank Algorithm

PageRank models a “Random Surfer” who clicks on links on the web.

To prevent the surfer from getting trapped in pages with no outgoing links (dead ends) or circular loops, PageRank introduces a **Damping Factor ($d \approx 0.85$)**:

- With probability d, the surfer clicks a random link on the current page.
- With probability $1 - d$, the surfer gets bored and jumps to a completely random page on the entire web.

Mathematical Formulation

Given an adjacency matrix A where $A_{ij} = 1$ if page i links to page j, we construct a transition probability matrix M where $M_{ij} = \frac{A_{ij}}{\text{out-degree}(i)}$.

The PageRank Google Matrix G is defined as:

$$G = d \cdot M + \frac{1 - d}{N} \mathbf{E}$$

where N is the total number of pages and $\mathbf{E}$ is an $N \times N$ matrix of all ones.

Python Simulation: PageRank Calculation

Here is how to calculate PageRank scores for a 4-page web graph using **NumPy** and **NetworkX**:

```python
import numpy as np
import networkx as nx

# 1. Define Web Graph Structure (4 Pages: A, B, C, D)
# Directed links: A -> B, A -> C, B -> D, C -> A, C -> B, C -> D, D -> C
edges = [('A', 'B'), ('A', 'C'), ('B', 'D'), ('C', 'A'), ('C', 'B'), ('C', 'D'), ('D', 'C')]
nodes = ['A', 'B', 'C', 'D']
N = len(nodes)

# 2. Build Transition Matrix M
# M[i, j] = probability of moving from node i to node j
M = np.zeros((N, N))
node_to_idx = {node: i for i, node in enumerate(nodes)}

for u, v in edges:
    M[node_to_idx[u], node_to_idx[v]] = 1

# Normalize rows so each row sums to 1
row_sums = M.sum(axis=1, keepdims=True)
M = M / row_sums

# 3. Apply Damping Factor (d = 0.85) to create Google Matrix G
d = 0.85
E = np.ones((N, N))
G = d * M + ((1 - d) / N) * E

# 4. Power Iteration Method to find Stationary Distribution
pagerank = np.ones(N) / N  # Initial uniform distribution

for step in range(100):
    pagerank = pagerank.dot(G)

print("PageRank Scores (Power Iteration):")
for node, score in zip(nodes, pagerank):
    print(f"Page {node}: {score:.4f}")

# 5. Verification using NetworkX library
G_nx = nx.DiGraph(edges)
pr_nx = nx.pagerank(G_nx, alpha=0.85)

print("\nPageRank Scores (NetworkX Verification):")
for node, score in sorted(pr_nx.items()):
    print(f"Page {node}: {score:.4f}")
```

Introduction to Optimization

Optimization is the process of finding the best solution from a set of feasible alternatives. A mathematical optimization problem has the general form:

$$\begin{array}{ll} \text{minimize} & f_0(x) \\ \text{subject to} & f_i(x) \leq b_i, \quad i = 1, \ldots, m \end{array}$$

Where:

- $x = (x_1, \ldots, x_n)$ is the **optimization variable**.
- $f_0 : \mathbb{R}^n \to \mathbb{R}$ is the **objective function**.
- $f_i(x) \leq b_i$ are the **constraint functions**.

1. Least-Squares & Linear Programming

Depending on the mathematical properties of f_0 and f_i, optimization problems are classified into several major families.

Least-Squares

A least-squares problem has no constraints and an objective function that is a sum of squares:

$$\text{minimize} \quad \|Ax - b\|_2^2 = \sum_{i=1}^{k} (a_i^T x - b_i)^2$$

This problem can be solved analytically ($x = (A^T A)^{-1} A^T b$). It is the foundation of **Linear Regression** in machine learning.

Linear Programming (LP)

In Linear Programming, both the objective and constraint functions are linear:

$$\begin{aligned} \text{minimize} \quad & c^T x \\ \text{subject to} \quad & a_i^T x \leq b_i, \quad i = 1, \ldots, m \end{aligned}$$

LPs do not have analytical solutions but can be solved extremely fast using algorithms like the Simplex method or Interior-point methods.

---

2. Convex vs. Non-linear Optimization

The boundary in optimization is not between linearity and non-linearity, but between **convexity** and **non-convexity**.

Property	Convex Optimization	Non-convex / Nonlinear Optimization
Geometry	objective function is convex, constraint set is convex.	objective or constraints are non-convex (curves bend down).
Minima	Any local minimum is a **global minimum**.	Many local minima and saddle points exist.
Solvability	Can be solved efficiently, reliably, and globally.	Hard to solve; algorithms often get stuck in local minima.

Exercises

Exercise 1

Classify the following optimization problem (find if it is LP, Least-Squares, or Nonlinear):

$$\text{minimize} \quad (3x_1 + x_2 - 5)^2 + (x_1 - 2x_2 + 1)^2$$

Solution — Exercise 1

This objective is a sum of squared linear terms:

$$f(x) = \|Ax - b\|_2^2$$

Where $A = \begin{bmatrix} 3 & 1 \\ 1 & -2 \end{bmatrix}$ and $b = \begin{bmatrix} 5 \\ -1 \end{bmatrix}$. Therefore, this is a **Least-Squares** problem.

Convex Sets & Functions

This page covers the mathematical definition and geometric properties of convex sets and convex functions, based on Chapters 2 and 3 of Stephen Boyd's *Convex Optimization*.

Part I: Convex Sets

A set $C \subseteq \mathbb{R}^n$ is **convex** if the line segment between any two points in C lies entirely in C. That is, for any $x_1, x_2 \in C$ and $0 \leq \theta \leq 1$:

$$\theta x_1 + (1 - \theta) x_2 \in C$$

Part II: Convex Functions (Chapter 3)

A function $f : \mathbb{R}^n \to \mathbb{R}$ is **convex** if its domain $\operatorname{dom} f$ is a convex set and for all $x, y \in \operatorname{dom} f$, $0 \leq \theta \leq 1$:

$$f(\theta x + (1 - \theta) y) \leq \theta f(x) + (1 - \theta) f(y)$$

1. Basic Properties and Examples (3.1)

First-order Condition for Convexity

A differentiable function f is convex if and only if $\mathrm{dom}\ f$ is convex and:

$$f(y) \geq f(x) + \nabla f(x)^T (y - x) \quad \text{for all } x, y \in \mathrm{dom}\ f$$

Geometrically, the tangent plane at any point acts as a global lower bound.

Second-order Condition for Convexity

A twice-differentiable function f is convex if and only if its Hessian matrix is positive semi-definite:

$$\nabla^2 f(x) \succeq 0 \quad \text{for all } x \in \mathrm{dom}\ f$$

Important Examples:

- **Exponential**: e^{ax} is convex on $\mathbb{R}$.
- **Powers**: x^p is convex on $\mathbb{R}_{++}$ for $p \geq 1$ or $p \leq 0$.
- **Negative Logarithm**: $-\ln(x)$ is convex on $\mathbb{R}_{++}$.
- **Max function**: $\max(x_1, \ldots, x_n)$ is convex on $\mathbb{R}^n$.

---

2. Operations that Preserve Convexity (3.2)

- **Nonnegative weighted sum**: $f = w_1 f_1 + w_2 f_2$ is convex if f_1, f_2 are convex and $w_1, w_2 \geq 0$.
- **Composition with affine mapping**: $g(x) = f(Ax + b)$ is convex if f is convex.
- **Pointwise maximum**: If $f_s(x)$ is convex for each $s \in \mathcal{S}$, then $f(x) = \sup_{s \in \mathcal{S}} f_s(x)$ is convex.
- **Partial Minimization**: If $g(x, y)$ is convex in (x, y) and C is a convex set, then $f(x) = \inf_{y \in C} g(x, y)$ is convex.

---

3. The Conjugate Function (3.3)

Let $f : \mathbb{R}^n \to \mathbb{R}$. The **conjugate function** $f^* : \mathbb{R}^n \to \mathbb{R}$ (also known as the Fenchel conjugate) is defined as:

$$f^*(y) = \sup_{x \in \mathrm{dom}\ f} (y^T x - f(x))$$

The conjugate function f^* is **always convex**, even if the original function f is not convex, because it is the pointwise supremum of a family of affine functions of y.

Example:

For $f(x) = \frac{1}{2}x^T Q x$ where Q is symmetric positive definite:

$$f^*(y) = \frac{1}{2}y^T Q^{-1} y$$

---

4. Quasiconvex Functions (3.4)

A function $f : \mathbb{R}^n \to \mathbb{R}$ is **quasiconvex** (or unimodal) if its domain and all its **sublevel sets** are convex:

$$S_\alpha = \{x \in \operatorname{dom} f \mid f(x) \leq \alpha\}$$

Modified Jensen's Inequality

A function f is quasiconvex if and only if its domain is convex and for any $x, y \in \operatorname{dom} f$ and $0 \leq \theta \leq 1$:

$$f(\theta x + (1-\theta)y) \leq \max(f(x), f(y))$$

First-order Condition for Quasiconvexity

A differentiable function f with convex domain is quasiconvex if and only if:

$$f(y) \leq f(x) \implies \nabla f(x)^T (y - x) \leq 0 \quad \text{for all } x, y \in \operatorname{dom} f$$

This means that if we move in a direction that decreases the function value, the directional derivative must be negative or zero.

Key Property: Sums

Unlike convex functions, the sum of two quasiconvex functions is **not necessarily quasiconvex**. For example, the functions:

$$f(x) = x^3 \quad \text{and} \quad g(x) = -x$$

are both quasiconvex on $\mathbb{R}$, but their sum:

$$(f+g)(x) = x^3 - x$$

is not (it is not unimodal and has multiple local extrema on $\mathbb{R}$).

---

5. Log-Concave and Log-Convex Functions (3.5)

A function $f : \mathbb{R}^n \to \mathbb{R}$ is **log-concave** if $f(x) > 0$ for all $x \in \operatorname{dom} f$ and the logarithm $\ln f(x)$ is a concave function:

$$f(\theta x + (1-\theta)y) \geq f(x)^\theta f(y)^{1-\theta} \quad \text{for } 0 \leq \theta \leq 1$$

Similarly, f is **log-convex** if $\ln f(x)$ is convex.

Second-order Condition for Log-Concavity

A twice-differentiable function f with convex domain is log-concave if and only if:

$$f(x)\nabla^2 f(x) \preceq \nabla f(x) \nabla f(x)^T \quad \text{for all } x \in \operatorname{dom} f$$

Important Examples

- **Powers**: $f(x) = x^a$ on $\mathbb{R}_{++}$ is log-convex for $a \leq 0$, and log-concave for $a \geq 0$.
- **Normal Distribution**: The Gaussian PDF is log-concave.
- **Cumulative Gaussian Distribution**: The cumulative distribution function is log-concave:

$$\Phi(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x} e^{-u^2/2}\, du$$

Properties and Integration

- **Product & Sum Rules**:
  - The product of log-concave functions is log-concave.
  - The sum of log-concave functions is **not always** log-concave.
- **Integration Theorem**: If $f : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}$ is log-concave, then the marginal integral function:

$$g(x) = \int f(x, y)\, dy$$

is log-concave.

- **Consequences**:
  - **Convolution**: The convolution of two log-concave functions is log-concave:

$$(f * g)(x) = \int f(x - y)g(y)\, dy$$

  - **Probability over Convex Sets**: If y is a random variable with a log-concave PDF, the probability that $x + y$ lies in a convex set C is log-concave in x:

$$f(x) = \mathrm{prob}(x + y \in C)$$

  - **Yield Function Example**: In manufacturing, the yield function (where x represents nominal parameters, w is random variation, and S is the set of acceptable values) is log-concave:

$$Y(x) = \mathrm{prob}(x + w \in S)$$

    Therefore, the yield regions $\{x \mid Y(x) \geq \alpha\}$ are convex sets.

---

6. Convexity with Respect to Generalized Inequalities (3.6)

Let $K \subseteq \mathbb{R}^m$ be a proper cone.

- A function $f : \mathbb{R}^n \to \mathbb{R}^m$ is **K-convex** if for all $x, y \in \mathrm{dom}\ f$ and $0 \leq \theta \leq 1$:

$$f(\theta x + (1 - \theta)y) \preceq_K \theta f(x) + (1 - \theta)f(y)$$

This maps the classical definition of convexity to vector-valued functions using cone-based ordering relations.

Example: Matrix Square

The matrix square function $f : \mathbb{S}^m \to \mathbb{S}^m$, defined as $f(X) = X^2$, is $\mathbb{S}^m_+$-convex.

Proof: For any fixed vector $z \in \mathbb{R}^m$, we can evaluate the scalar function:

$$g(X) = z^T X^2 z = z^T XXz = ||Xz||_2^2$$

Since the Euclidean norm is convex, $g(X) = ||Xz||_2^2$ is a convex function of X. Therefore, for all $X, Y \in \mathbb{S}^m$ and $0 \leq \theta \leq 1$:

$$z^T(\theta X + (1-\theta)Y)^2 z \leq \theta z^T X^2 z + (1-\theta) z^T Y^2 z$$

Since this inequality holds for all vectors $z \in \mathbb{R}^m$, by definition of the positive semidefinite cone $\mathbb{S}^m_+$, it implies:

$$(\theta X + (1-\theta)Y)^2 \preceq_{\mathbb{S}^m_+} \theta X^2 + (1-\theta)Y^2$$

This proves that $f(X) = X^2$ is indeed $\mathbb{S}^m_+$-convex.

Exercises

Exercise 1

Find the conjugate function $f^*(y)$ of the negative entropy function:

$$f(x) = x \ln x \quad (\text{domain } x > 0)$$

Solution — Exercise 1

The conjugate is defined as $f^*(y) = \sup_{x>0}(yx - x\ln x)$. To find the supremum, take the derivative with respect to x and set to zero:

$$\frac{d}{dx}(yx - x\ln x) = y - \ln x - 1 = 0 \implies \ln x = y - 1 \implies x^* = e^{y-1}$$

Substitute x^* back into the equation:

$$f^*(y) = y(e^{y-1}) - e^{y-1}(y-1) = e^{y-1}(y - y + 1) = e^{y-1}$$

Thus, the conjugate function is $f^*(y) = e^{y-1}$.

Convex Optimization Problems

This page covers the mathematical formulation and classification of optimization problems based on Chapter 4 of Stephen Boyd's *Convex Optimization*.

---

1. General Optimization Problems (4.1)

A general mathematical optimization problem has the form:

$$\begin{array}{ll} \text{minimize} & f_0(x) \\ \text{subject to} & f_i(x) \leq 0, \quad i = 1, \ldots, m \\ & h_i(x) = 0, \quad i = 1, \ldots, p \end{array}$$

Where:

- $x \in \mathbb{R}^n$ is the **optimization variable**.
- $f_0(x)$ is the **objective function**.
- $f_i(x) \leq 0$ are the **inequality constraints**.
- $h_i(x) = 0$ are the **equality constraints**.

The set of points for which the objective and all constraint functions are defined is the **domain** $\mathcal{D}$. A point $x \in \mathcal{D}$ is **feasible** if it satisfies all constraints. The optimal value p^* is defined as:

$$p^* = \inf\{f_0(x) \mid f_i(x) \leq 0,\, i = 1, \ldots, m,\, h_i(x) = 0,\, i = 1, \ldots, p\}$$

---

2. Convex Optimization Problems (4.2)

A **convex optimization problem** is one of the form:

$$\begin{array}{ll} \text{minimize} & f_0(x) \\ \text{subject to} & f_i(x) \leq 0, \quad i = 1, \ldots, m \\ & a_i^T x = b_i, \quad i = 1, \ldots, p \end{array}$$

Where:

- The objective function f_0 must be **convex**.
- The inequality constraint functions $f_1, \ldots, f_m$ must be **convex**.
- The equality constraints must be **affine** ($Ax = b$).

Key Property: Local Minima are Global

For a convex optimization problem, any local optimal point is also globally optimal.

---

3. Linear Optimization Problems - LP (4.3)

When both the objective and constraint functions are affine, the problem is called a **Linear Program (LP)**:

$$\begin{array}{ll} \text{minimize} & c^T x + d \\ \text{subject to} & Gx \leq h \\ & Ax = b \end{array}$$

LPs are widely used in operations research, logistics, and resource allocation.

---

4. Quadratic Optimization Problems - QP & QCQP (4.4)

A **Quadratic Program (QP)** has a convex quadratic objective and affine constraints:

$$\begin{array}{ll} \text{minimize} & \frac{1}{2} x^T P x + q^T x + r \\ \text{subject to} & Gx \leq h \\ & Ax = b \end{array}$$

Where P is symmetric positive semi-definite ($P \succeq 0$).

Quadratically Constrained Quadratic Program (QCQP)

If the inequality constraints are also quadratic, the problem is a **QCQP**:

$$\begin{array}{ll} \text{minimize} & \frac{1}{2} x^T P_0 x + q_0^T x + r_0 \\ \text{subject to} & \frac{1}{2} x^T P_i x + q_i^T x + r_i \leq 0, \quad i = 1, \ldots, m \\ & Ax = b \end{array}$$

where $P_0, P_1, \ldots, P_m \succeq 0$. QCQPs are used in robust design and signal processing.

---

5. Second-Order Cone Programming - SOCP

A **Second-Order Cone Program (SOCP)** is a generalization of QPs and QCQPs. It allows constraints that require a vector to lie within a second-order cone (also known as the Lorentz or ice-cream cone):

$$\begin{array}{ll} \text{minimize} & f^T x \\ \text{subject to} & \|A_i x + b_i\|_2 \leq c_i^T x + d_i, \quad i = 1, \ldots, m \\ & Fx = g \end{array}$$

where $x \in \mathbb{R}^n$, $A_i \in \mathbb{R}^{n_i \times n}$, $b_i \in \mathbb{R}^{n_i}$, $c_i \in \mathbb{R}^n$, and $d_i \in \mathbb{R}$.

Second-Order Cone Constraint

The constraint $\|A_i x + b_i\|_2 \leq c_i^T x + d_i$ requires the affine combination $(A_i x + b_i, c_i^T x + d_i)$ to lie in the second-order cone:

$$\mathcal{K}_i = \{(y, t) \in \mathbb{R}^{n_i} \times \mathbb{R} \mid \|y\|_2 \leq t\}$$

SOCPs are widely applied in portfolio optimization with tracking error constraints, filter design, and robust linear programming under ellipsoidal uncertainty.

---

6. Semidefinite Programming - SDP

A **Semidefinite Program (SDP)** is an optimization problem where the variables are symmetric matrices constrained to be positive semidefinite:

$$\begin{array}{ll} \text{minimize} & c^T x \\ \text{subject to} & x_1 F_1 + x_2 F_2 + \cdots + x_n F_n \preceq F_0 \\ & Ax = b \end{array}$$

where $x \in \mathbb{R}^n$, and $F_0, F_1, \ldots, F_n$ are symmetric matrices in $\mathbb{S}^k$ (the space of symmetric $k \times k$ matrices).

Linear Matrix Inequality (LMI)

The constraint $x_1 F_1 + x_2 F_2 + \cdots + x_n F_n \preceq F_0$ is called a **Linear Matrix Inequality (LMI)**. The symbol $\preceq$ denotes matrix inequality, meaning:

$$F_0 - \sum_{i=1}^{n} x_i F_i \succeq 0 \quad (\text{is Positive Semidefinite})$$

SDPs generalize LPs and SOCPs. They are foundational in control theory (Lyapunov stability), structural design, and machine learning (kernel matrix learning and maximum variance unfolding).

---

7. Geometric Programming - GP (4.5)

A **Geometric Program (GP)** is an optimization problem involving objective and constraint functions that are posynomials.

Monomials and Posynomials

- A **monomial** $f : \mathbb{R}^n \to \mathbb{R}$ is a function of the form:

$$f(x) = cx_1^{a_1}x_2^{a_2}\dots x_n^{a_n}$$

  where $c > 0$ and $a_i \in \mathbb{R}$.

- A **posynomial** is a sum of monomials:

$$f(x) = \sum_{k=1}^{K} c_k x_1^{a_{1k}}x_2^{a_{2k}}\dots x_n^{a_{nk}}$$

Standard Geometric Program

A standard GP has the form:

$$\begin{array}{ll} \text{minimize} & f_0(x) \\ \text{subject to} & f_i(x) \leq 1, \dots, m \\ & h_i(x) = 1, \dots, p \end{array}$$

where $f_0, \dots, f_m$ are posynomials, and $h_1, \dots, h_p$ are monomials.

Convex Transformation

GPs are non-convex in their original form. However, we can transform them into convex problems by introducing a change of variables $y_i = \ln(x_i)$ and taking the natural logarithm of the objective and constraints. This results in the transformed problem:

$$\begin{array}{ll} \text{minimize} & \tilde{f}_0(y) = \ln f_0(e^y) \\ \text{subject to} & \tilde{f}_i(y) = \ln f_i(e^y) \leq 0, \quad i = 1, \dots, m \\ & \tilde{h}_i(y) = \ln h_i(e^y) = 0, \quad i = 1, \dots, p \end{array}$$

Since $\tilde{f}_i$ are log-sum-exp functions (which are convex), the transformed problem is a convex optimization problem.

8. Vector Optimization (4.7)

In many practical situations, we want to optimize multiple objectives simultaneously (multicriterion optimization):

$$\text{minimize (with respect to proper cone } K) \quad f_0(x) = (f_1(x), \ldots, f_q(x))$$

Since it is usually impossible to minimize all objectives at once, we look for **Pareto optimal** solutions. A feasible point x is Pareto optimal if there is no other feasible point y that performs better or equal in all objectives and strictly better in at least one.

Exercises

Exercise 1

Formulate the following minimization problem as a standard Quadratic Program (QP):

$$\begin{array}{ll} \text{minimize} & (x_1 - x_2)^2 + 3(x_2 - 2)^2 \\ \text{subject to} & x_1 + x_2 \geq 1 \end{array}$$

Solution — Exercise 1

To express the problem in the standard QP form:

$$\begin{aligned} \text{minimize} \quad & \frac{1}{2}x^T P x + q^T x + r \\ \text{subject to} \quad & Gx \leq h \end{aligned}$$

we follow these mathematical steps:

Step 1: Expand the Objective Function Expand the algebraic terms in the objective function $f(x)$:

$$\begin{aligned} f(x) &= (x_1^2 - 2x_1x_2 + x_2^2) + 3(x_2^2 - 4x_2 + 4) \\ &= x_1^2 - 2x_1x_2 + 4x_2^2 - 12x_2 + 12 \end{aligned}$$

Step 2: Extract Matrix Parameters We match the expanded objective to the quadratic matrix form $\frac{1}{2}x^T P x + q^T x + r$, where $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$:

- **Hessian Matrix (P)**:

$$P = \begin{bmatrix} 2 & -2 \\ -2 & 8 \end{bmatrix}$$

  (Note that $\frac{1}{2}x^T P x = x_1^2 - 2x_1x_2 + 4x_2^2$).

- **Linear Term Vector (q)**:

$$q = \begin{bmatrix} 0 \\ -12 \end{bmatrix}$$

- **Constant Scalar (r)**:

$$r = 12$$

Step 3: Verify Convexity For a valid QP, P must be positive semidefinite ($P \succeq 0$). The eigenvalues of P are found by solving $\det(P - \lambda I) = 0$:

$$\det \begin{bmatrix} 2-\lambda & -2 \\ -2 & 8-\lambda \end{bmatrix} = \lambda^2 - 10\lambda + 12 = 0 \implies \lambda \approx 8.61,\ 1.39$$

Since both eigenvalues are strictly positive ($\lambda > 0$), P is symmetric positive definite ($P \succ 0$), guaranteeing that the objective function is strictly convex.

Step 4: Convert Constraints to Standard Form The inequality constraint $x_1 + x_2 \geq 1$ must be converted to the standard form $Gx \leq h$:

$$-x_1 - x_2 \leq -1 \implies \begin{bmatrix} -1 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \leq -1$$

Thus, we identify:

$$G = \begin{bmatrix} -1 & -1 \end{bmatrix}, \quad h = -1$$

Conclusion The problem is successfully formulated as a standard convex Quadratic Program (QP).

Duality & KKT Conditions

Duality allows us to view an optimization problem from two perspectives: the **Primal** problem (original variables) and the **Dual** problem (constraints coefficients). This page covers Lagrange Duality and the famous Karush-Kuhn-Tucker (KKT) optimality conditions.

---

1. The Lagrange Dual Function

For the primal problem:

$$\begin{array}{ll} \text{minimize} & f_0(x) \\ \text{subject to} & f_i(x) \leq 0, \quad i = 1, \ldots, m \\ & h_i(x) = 0, \quad i = 1, \ldots, p \end{array}$$

We define the **Lagrangian** $L : \mathbb{R}^n \times \mathbb{R}^m \times \mathbb{R}^p \to \mathbb{R}$ by associating Lagrange multipliers $\lambda_i \geq 0$ and ν_i with the constraints:

$$L(x, \lambda, \nu) = f_0(x) + \sum_{i=1}^{m} \lambda_i f_i(x) + \sum_{i=1}^{p} \nu_i h_i(x)$$

The **Lagrange dual function** $g(\lambda, \nu)$ is the minimum value of the Lagrangian over x:

$$g(\lambda, \nu) = \inf_{x \in \mathcal{D}} L(x, \lambda, \nu)$$

The Lower Bound Property

For any $\lambda \geq 0$ and any ν, the dual function gives a lower bound on the optimal value p^* of the primal problem:

$$g(\lambda, \nu) \leq p^*$$

2. Strong Duality & Weak Duality

The **Lagrange dual problem** is to find the best lower bound:

$$\begin{array}{ll} \text{maximize} & g(\lambda, \nu) \\ \text{subject to} & \lambda \succeq 0 \end{array}$$

Let d^* be the optimal value of the dual problem.

- **Weak Duality**: $d^* \leq p^*$ always holds (even for non-convex problems).
- **Strong Duality**: $d^* = p^*$ holds (usually for convex problems satisfying Slater's constraint qualification).

---

3. Karush-Kuhn-Tucker (KKT) Conditions

For any optimization problem with differentiable objective and constraint functions for which strong duality holds, any primal-optimal x^* and dual-optimal (λ^*, ν^*) must satisfy the **KKT conditions**:

1. **Primal Feasibility**:

$$f_i(x^*) \leq 0, \quad i = 1, \ldots, m$$

$$h_i(x^*) = 0, \quad i = 1, \ldots, p$$

2. **Dual Feasibility**:

$$\lambda^* \succeq 0$$

3. **Complementary Slackness**:

$$\lambda_i^* f_i(x^*) = 0, \quad i = 1, \ldots, m$$

4. **Lagrangian Stationarity** (Gradient is zero):

$$\nabla f_0(x^*) + \sum_{i=1}^{m} \lambda_i^* \nabla f_i(x^*) + \sum_{i=1}^{p} \nu_i^* \nabla h_i(x^*) = 0$$

If the primal problem is convex, the KKT conditions are not only necessary but also **sufficient** for optimality.

Exercises

Exercise 1

Explain the meaning of **Complementary Slackness** ($\lambda_i^* f_i(x^*) = 0$).

Solution — Exercise 1

Complementary slackness states that for each inequality constraint $f_i(x) \leq 0$:

- If the constraint is inactive at the optimum ($f_i(x^*) < 0$), then the multiplier must be zero ($\lambda_i^* = 0$).
- If the multiplier is positive ($\lambda_i^* > 0$), then the constraint must be active ($f_i(x^*) = 0$).

This is the mathematical backing for **Support Vectors** in SVMs, where only data points on the boundary (active constraints) have non-zero Lagrange weights.

Gradient Descent & Hessian Curvature

In continuous optimization, we find minimum or maximum values of functions using derivatives. This page covers Gradient Descent (first-order optimization) and the Hessian matrix (second-order optimization).

---

1. Local Extrema & Second Derivative Test

To find the minimum or maximum of a multivariable function $f(x, y)$, we start by finding the **critical points** where the gradient is zero:

$$\nabla f(x, y) = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Once we find a critical point, we use the **Hessian matrix** $\mathbf{H}$ (which captures the curvature of the function) to determine its nature:

$$\mathbf{H} = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{bmatrix}$$

Using the eigenvalues of the Hessian matrix at the critical point:

- **Local Minimum**: If $\mathbf{H}$ is positive-definite (all eigenvalues > 0). The curve bends upwards.
- **Local Maximum**: If $\mathbf{H}$ is negative-definite (all eigenvalues < 0). The curve bends downwards.

- **Saddle Point**: If $\mathbf{H}$ has both positive and negative eigenvalues. The curve bends up in one direction and down in another.

---

2. General Descent & Gradient Descent

General Descent Method

To minimize a convex function f, we use a general iterative descent method. Given a starting point $x \in \operatorname{dom} f$, the algorithm repeats the following steps:

1. **Determine descent direction**: Find a search direction Δx (for gradient descent, $\Delta x = -\nabla f(x)$).
2. **Line Search**: Choose a step size $t > 0$ using exact or backtracking line search.
3. **Update**: Set $x := x + t\Delta x$.

This process repeats until a **stopping criterion** is satisfied, usually of the form:

$$\|\nabla f(x)\|_2 \leq \epsilon$$

For strongly convex functions, the convergence satisfies $f(x^{(k)}) - p^* \leq c^k(f(x^{(0)}) - p^*)$, where $c \in (0, 1)$ depends on the minimum/maximum eigenvalues of the Hessian, the starting point, and the line search method.

Gradient Descent Step

Since the gradient ∇f points in the direction of the steepest increase, updating our variables in the opposite direction ($-\nabla f$) moves us down toward the minimum:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla J(\mathbf{w})$$

Where α is the **learning rate** (step size).

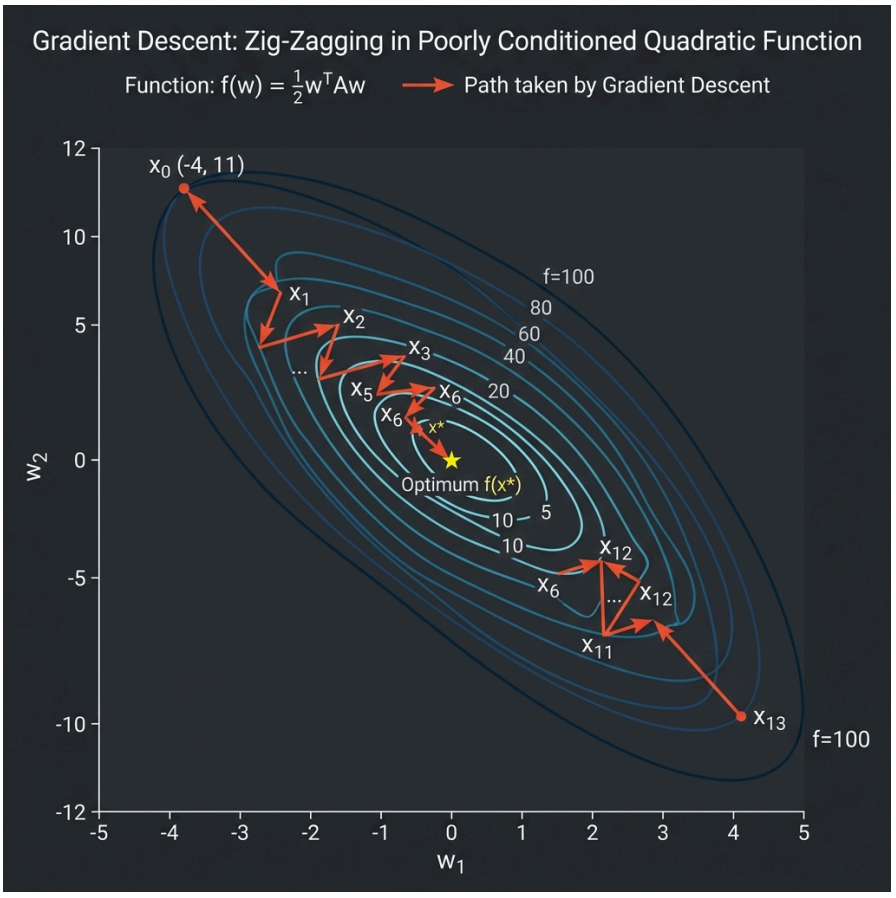
Convergence Behavior on Ill-Conditioned Functions

When the contour lines of a function are highly elongated (i.e., the Hessian has a very large condition number $\gamma \gg 1$), the gradient does not point directly towards the optimum. As a result, Gradient Descent exhibits a **zig-zagging behavior** and converges very slowly.

For example, minimizing the quadratic function $f(x) = \frac{1}{2}(x_1^2 + \gamma x_2^2)$ starting from $x^{(0)} = (\gamma, 1)$ using exact line search yields the coordinates at step k:

$$x_1^{(k)} = \gamma\left(\frac{\gamma-1}{\gamma+1}\right)^k, \quad x_2^{(k)} = \left(-\frac{\gamma-1}{\gamma+1}\right)^k$$

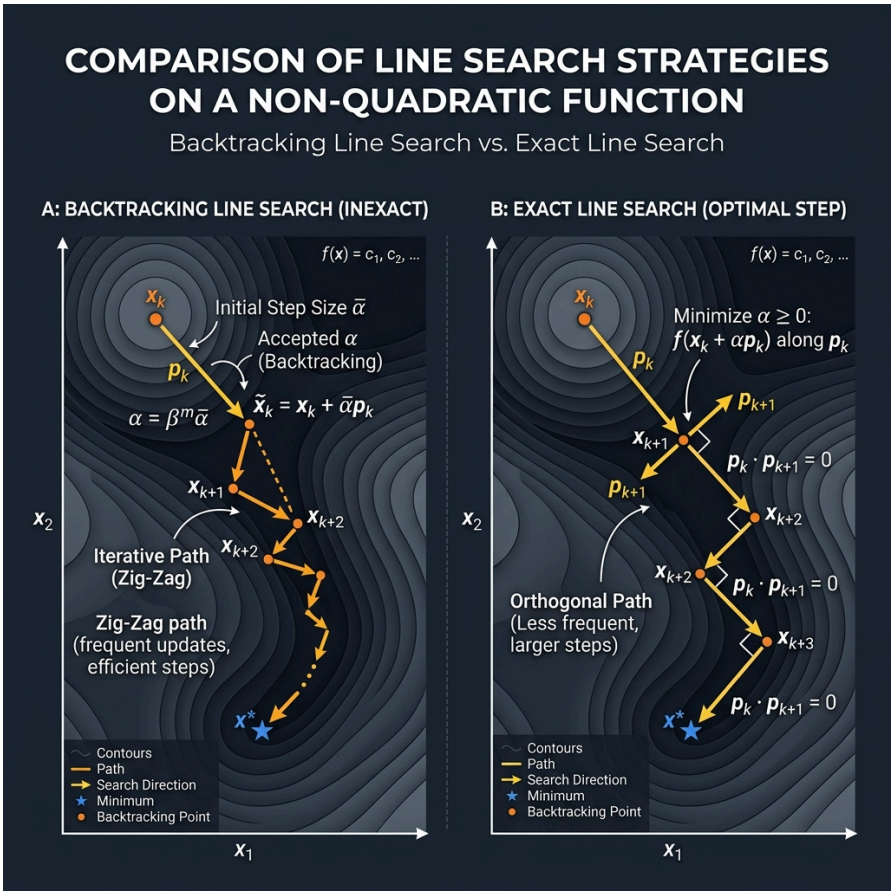
If $\gamma = 10$, convergence requires many small, zig-zagging steps to reach the optimum:



Line Search Strategies (Backtracking vs Exact)

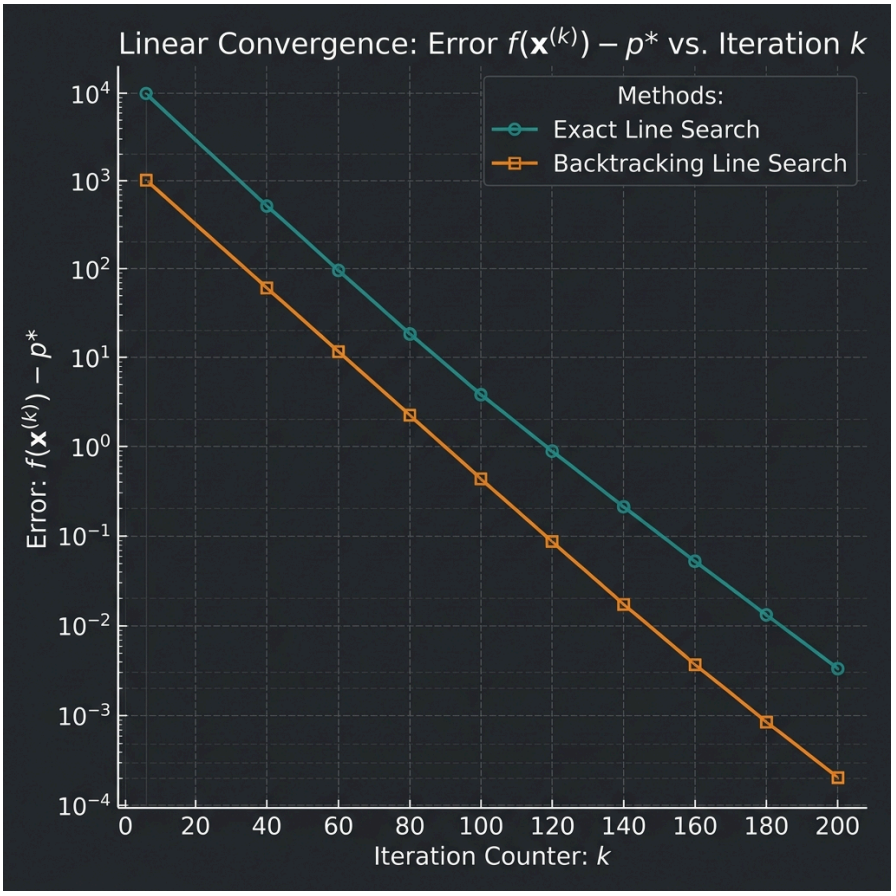
Choosing the correct step size α at each iteration is crucial:

- **Exact Line Search**: Minimizes the function along the search direction p_k. At each step, the next search direction is orthogonal to the previous one ($\nabla f(x_{k+1})^T \nabla f(x_k) = 0$).
- **Backtracking Line Search**: An inexact line search method that starts with a large step size and decreases it exponentially (scaled by β) until it satisfies the Armijo condition. It is computationally cheaper and highly effective in practice:



Linear Convergence on Semilog Plots

For strongly convex functions, the error $f(x^{(k)}) - p^*$ decreases exponentially with k. This behavior is called **linear convergence**. When plotted on a semilogarithmic scale (where the y-axis is logarithmic and the x-axis is linear), this exponential decay appears as a straight line:



3. Steepest Descent Method (General Norms)

The **Steepest Descent Method** generalizes gradient descent by defining the descent step with respect to a general norm $\|\cdot\|$.

Normalized Steepest Descent Direction

The normalized steepest descent direction Δx_{nsd} is defined as the step of unit norm that minimizes the directional derivative of f:

$$\Delta x_{\text{nsd}} = \text{argmin}\{\nabla f(x)^T v \mid \|v\| = 1\}$$

The unnormalized steepest descent step is scaled by the dual norm: $\Delta x_{\text{sd}} = \|\nabla f(x)\|_* \Delta x_{\text{nsd}}$, which satisfies $\nabla f(x)^T \Delta x_{\text{sd}} = -\|\nabla f(x)\|_*^2$.

Steepest Descent Directions for Different Norms

- **Euclidean Norm ($\|\cdot\|_2$)**: The steepest descent direction is the negative gradient:

$$\Delta x_{\text{sd}} = -\nabla f(x)$$

- **Quadratic Norm ($\|x\|_P = (x^T P x)^{1/2}$)**: The steepest descent direction is:

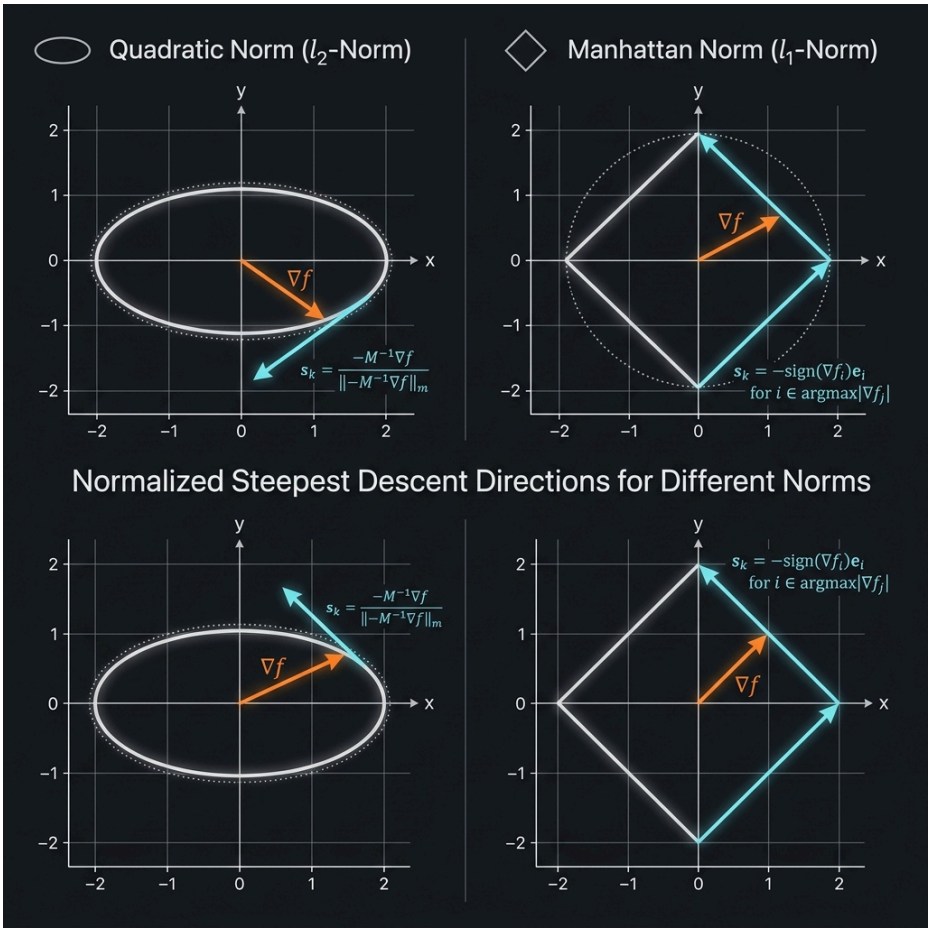
$$\Delta x_{\text{sd}} = -P^{-1}\nabla f(x)$$

This is equivalent to performing standard gradient descent after a coordinate transformation (change of variables) $\bar{x} = P^{1/2}x$. By choosing $P \approx \nabla^2 f(x^*)$, we can align the coordinate system to eliminate ill-conditioning and dramatically speed up convergence.

- **ℓ_1-Norm ($\|\cdot\|_1$)**: The steepest descent step updates only a single coordinate direction:

$$\Delta x_{\text{sd}} = -\frac{\partial f(x)}{\partial x_i} e_i$$

where i is the index corresponding to the largest component of the gradient: $|\partial f(x)/\partial x_i| = \|\nabla f(x)\|_\infty$. This is the basis of coordinate descent methods.



4. Quadratic Optimization with SymPy

Let's minimize the function $f(x) = x^2 - 4x + 4$ (which has a minimum at $x = 2$, where the gradient is $2x - 4$).

```
x = 10.0          # Starting point
learning_rate = 0.1
epochs = 20

for epoch in range(epochs):
    gradient = 2 * x - 4
    x = x - learning_rate * gradient
    loss = x**2 - 4*x + 4
    print(f"Epoch {epoch+1:02d}: x = {x:.4f}, Loss = {loss:.4f}")
```

3. Quadratic Optimization with SymPy

We can use SymPy to find the analytical Hessian and determine the nature of critical points:

```
import sympy as sp

x, y = sp.symbols('x y')
f = x**2 + y**2 - 4*x - 6*y

# 1. Find critical points by solving Gradient = 0
grad_x = sp.diff(f, x)
grad_y = sp.diff(f, y)
critical_points = sp.solve([grad_x, grad_y], (x, y))
print("Critical Point:", critical_points)

# 2. Compute the Hessian matrix
H = sp.hessian(f, (x, y))
print("\nHessian Matrix:")
sp.pprint(H)
```

Exercises

Exercise 1

Perform 3 iterations of Gradient Descent manually for the function $f(x) = x^2$. Start at $x_0 = 4$ with a learning rate $\alpha = 0.1$.

Solution — Exercise 1

The derivative is $f'(x) = 2x$.

- **Iteration 1**:

$$x_1 = x_0 - \alpha \cdot f'(x_0) = 4 - 0.1(2 \cdot 4) = 4 - 0.8 = 3.2$$

- **Iteration 2**:

$$x_2 = x_1 - \alpha \cdot f'(x_1) = 3.2 - 0.1(2 \cdot 3.2) = 3.2 - 0.64 = 2.56$$

- **Iteration 3**:

$$x_3 = x_2 - \alpha \cdot f'(x_2) = 2.56 - 0.1(2 \cdot 2.56) = 2.56 - 0.512 = 2.048$$

Optimization Algorithms

To solve unconstrained and constrained minimization problems, we rely on iterative numerical methods. This page covers Gradient Descent, Newton's Method, and Interior-point barrier methods.

---

1. Unconstrained Minimization

We want to solve:

$$\text{minimize} \quad f(x)$$

Where f is twice differentiable and convex.

Descent Methods

A descent method produces a sequence of steps $x^{(k+1)} = x^{(k)} + t^{(k)}\Delta x^{(k)}$ where:

- $\Delta x^{(k)}$ is the **step direction** (ensuring $\nabla f(x)^T \Delta x < 0$).
- $t^{(k)} > 0$ is the **step size** (learning rate), often calculated using backtracking line search.

Gradient Descent Method

The search direction is the negative gradient:

$$\Delta x = -\nabla f(x)$$

Newton's Method

Newton's method incorporates second-order curvature:

$$\Delta x_{\text{nt}} = -(\nabla^2 f(x))^{-1} \nabla f(x)$$

Newton's method converges much faster than gradient descent (quadratic convergence) because it scales the gradient step using the inverse Hessian.

---

2. Equality & Inequality Constraints (Interior-Point Methods)

Logarithmic Barrier

To solve problems with inequality constraints $f_i(x) \leq 0$, we approximate them using an indicator function, which we smooth using the **logarithmic barrier**:

$$\Phi(x) = -\sum_{i=1}^{m} \ln(-f_i(x))$$

As $f_i(x) \to 0$, the log barrier $\Phi(x) \to \infty$, preventing the optimization from stepping outside the feasible region.

The Barrier Method

The Barrier method solves a sequence of unconstrained problems:

$$\text{minimize} \quad t f_0(x) + \Phi(x)$$

for increasing values of $t > 0$, tracing the **central path** directly to the global optimum.

---

3. Python Comparison: Gradient Descent vs. Newton's

Method

```python
import numpy as np

# Function: f(x) = x^4, f'(x) = 4x^3, f''(x) = 12x^2
# Minimum is at x = 0

# 1. Gradient Descent
x_gd = 2.0
lr = 0.05
for i in range(5):
    grad = 4 * x_gd**3
    x_gd -= lr * grad
    print(f"GD Step {i+1}: x = {x_gd:.4f}")

# 2. Newton's Method
x_newton = 2.0
for i in range(5):
    grad = 4 * x_newton**3
    hessian = 12 * x_newton**2
    x_newton -= grad / hessian  # Newton step: - f'/f''
    print(f"Newton Step {i+1}: x = {x_newton:.4f}")
```

Exercises

Exercise 1

Calculate 1 step of Newton's method starting at $x_0 = 2$ for the function $f(x) = x^3 - 2x^2$.

Solution — Exercise 1

Derivatives:

- $f'(x) = 3x^2 - 4x \implies f'(2) = 3(4) - 4(2) = 4$
- $f''(x) = 6x - 4 \implies f''(2) = 6(2) - 4 = 8$

Newton step:

$$x_1 = x_0 - \frac{f'(x_0)}{f''(x_0)} = 2 - \frac{4}{8} = 1.5$$

Numerical Algorithms & Interior-Point Methods

This page covers numerical algorithms for solving constrained optimization problems based on Chapters 9, 10, and 11 of Stephen Boyd's *Convex Optimization*.

---

1. Newton's Method with Equality Constraints (Chapter 10)

For unconstrained optimization, Newton's method uses the second-order Taylor expansion to find the search direction. For problems with equality constraints:

$$\begin{array}{ll} \text{minimize} & f(x) \\ \text{subject to} & Ax = b \end{array}$$

where f is convex and twice continuously differentiable. We must ensure that the search step keeps the variable feasible ($A(x + \Delta x) = b$).

Newton Step under Equality Constraints

The Newton step Δx_{nt} is obtained by solving the following KKT system of linear equations:

$$\begin{bmatrix} \nabla^2 f(x) & A^T \\ A & 0 \end{bmatrix} \begin{bmatrix} \Delta x_{nt} \\ w \end{bmatrix} = \begin{bmatrix} -\nabla f(x) \\ 0 \end{bmatrix}$$

where:

- $\nabla^2 f(x)$ is the Hessian of f.
- $\nabla f(x)$ is the gradient of f.
- w is the vector of optimal Lagrange multipliers for the equality constraints.

This KKT matrix is invertible if $\nabla^2 f(x)$ is positive definite on the nullspace of A.

---

2. The Barrier Method (Chapter 11)

The **Barrier Method** (or path-following method) is an interior-point method used to solve convex optimization problems with inequality constraints:

$$\begin{array}{ll} \text{minimize} & f_0(x) \\ \text{subject to} & f_i(x) \leq 0, \quad i = 1, \ldots, m \\ & Ax = b \end{array}$$

Logarithmic Barrier Function

We can approximate the inequality constraints using an indicator function, which we smooth using the **logarithmic barrier function**:

$$\Phi(x) = -\sum_{i=1}^{m} \ln(-f_i(x))$$

defined on the feasible interior domain $\operatorname{dom} \Phi = \{x \mid f_i(x) < 0,\ i = 1, \ldots, m\}$. As $f_i(x) \to 0$, the function $\Phi(x) \to \infty$, preventing the variable from leaving the feasible interior region.

The Central Path

For a parameter $t > 0$, we define the central path $x^*(t)$ as the solution to:

$$\begin{aligned} \text{minimize} \quad & t f_0(x) + \Phi(x) \\ \text{subject to} \quad & Ax = b \end{aligned}$$

As $t \to \infty$, the central path $x^*(t)$ converges to the optimal solution x^* of the original constrained problem.

The Barrier Algorithm

1. **Start** with a strictly feasible starting point x, $t = t^{(0)} > 0$, tolerance $\epsilon > 0$, and parameter $\mu > 1$.
2. **Centering Step**: Compute $x^*(t)$ by minimizing $t f_0(x) + \Phi(x)$ subject to $Ax = b$ (typically solved using Newton's method).
3. **Update**: $x := x^*(t)$.
4. **Stopping Criterion**: If $m/t < \epsilon$, stop and return x.
5. **Increase** t: $t := \mu t$, and repeat from step 2.

---

3. Primal-Dual Interior-Point Methods

Unlike the Barrier Method which executes a full centering step (Newton minimization) for each parameter update, **Primal-Dual Interior-Point Methods** update the primal variables x, dual variables λ (for inequalities), and nu (for equalities) simultaneously at each iteration.

Perturbed KKT Conditions

We solve the KKT conditions perturbed by a parameter t:

$$\begin{aligned} \nabla f_0(x) + \sum_{i=1}^{m} \lambda_i \nabla f_i(x) + A^T \nu &= 0 \\ -\lambda_i f_i(x) &= 1/t, \quad i = 1, \ldots, m \\ Ax - b &= 0 \end{aligned}$$

Applying Newton's method to this non-linear system of equations gives the primal-dual search direction $(\Delta x_{pd}, \Delta \lambda_{pd}, \Delta \nu_{pd})$. These methods often converge much faster than barrier methods because they do not require high accuracy centering steps.

---

4. Self-Concordance Analysis

The rate of convergence of Newton's method typically depends on unknown constants (like the Lipschitz constant of the Hessian). However, Stephen Boyd introduces **Self-concordance**, a properties-based analysis that allows bounding the number of Newton iterations without relying on coordinate-dependent constants.

Definition of Self-Concordant Functions

A convex function $f : \mathbb{R} \to \mathbb{R}$ is self-concordant if:

$$|f'''(x)| \leq 2f''(x)^{3/2}$$

For multi-dimensional functions $f : \mathbb{R}^n \to \mathbb{R}$, it must be self-concordant along every line.

- **Example**: The logarithmic barrier $-\sum \ln(x_i)$ is self-concordant.
- **Significance**: For self-concordant functions, the number of Newton iterations required to reach a specific accuracy is bounded by a constant that depends only on the initial suboptimality, providing rigorous convergence guarantees.

Python Example: Implementing the Barrier Method

Here is a simple Python script implementing the Barrier Method to solve a quadratically constrained problem:

$$\begin{aligned} &\text{minimize} && x^2 \\ &\text{subject to} && x \geq 1 \quad (\text{written as } 1 - x \leq 0) \end{aligned}$$

```python
import numpy as np

def barrier_method(x0, t=1.0, mu=10.0, epsilon=1e-6):
    x = x0
    m = 1 # Number of inequality constraints: f1(x) = 1 - x <= 0

    print(f"Starting Barrier Method from feasible x = {x:.4f}\n")

    iteration = 0
    while True:
        # Centering step: minimize t * f0(x) - ln(-f1(x))
        # objective: t * x^2 - ln(x - 1)
        # We solve this mini-optimization using Newton's method:
        for _ in range(50):
            # Gradient of centering objective
            grad = 2 * t * x - 1 / (x - 1)
            # Hessian of centering objective
            hess = 2 * t + 1 / ((x - 1) ** 2)

            step = -grad / hess
            x += step

            # Convergence check for Newton step
            if abs(step) < 1e-8:
                break

        duality_gap = m / t
        print(f"Iter {iteration:02d} | t = {t:8.1f} | x = {x:.6f} | Duality Gap =
{duality_gap:.8f}")

        if duality_gap < epsilon:
            break

        t *= mu
        iteration += 1

    return x

# Strictly feasible starting point (x > 1)
optimal_x = barrier_method(x0=5.0)
print(f"\nOptimal Solution found: x = {optimal_x:.6f} (Analytical limit = 1.000000)")
```

Geometric and Statistical Optimization

This page covers applications of convex optimization in geometric problems and statistical estimation based on Chapters 7 and 8 of Stephen Boyd's *Convex Optimization*.

---

1. Statistical Estimation (Chapter 7)

Many foundational statistical estimation problems can be formulated directly as convex optimization problems, allowing them to be solved efficiently and with guaranteed global optimality.

Maximum Likelihood Estimation (MLE)

In MLE, we estimate the parameter vector θ of a probability distribution $p(y;\theta)$ from observed data y. The goal is to maximize the likelihood function, or equivalently, minimize the **negative log-likelihood function**:

$$\begin{aligned} \text{minimize} \quad & -\sum_{i=1}^{N} \ln p(y_i;\theta) \\ \text{subject to} \quad & \theta \in \Theta \end{aligned}$$

If $p(y;\theta)$ is log-concave in θ, then the negative log-likelihood is convex, and finding the MLE is a convex optimization problem.

Example: Linear Regression with Gaussian Noise

Consider $y_i = x_i^T\theta + v_i$ where $v_i \sim \mathcal{N}(0,\sigma^2)$ are independent Gaussian noise terms. The MLE objective is:

$$\text{minimize} \quad \frac{N}{2}\ln(2\pi\sigma^2) + \frac{1}{2\sigma^2}\sum_{i=1}^{N}(y_i - x_i^T\theta)^2$$

This is a classic Least-Squares optimization problem, which is a convex QP and can be solved analytically or numerically.

---

2. Geometrical Problems (Chapter 8)

Convex optimization provides elegant tools to solve geometric problems involving distances, projections, and bounding volumes.

2.1 Projection on a Convex Set

The projection of a point x_0 onto a closed convex set C is the point $P_C(x_0) \in C$ that is closest to x_0:

$$\begin{aligned} \text{minimize} \quad & \|x - x_0\|_2^2 \\ \text{subject to} \quad & x \in C \end{aligned}$$

Since the objective is strictly convex, the projection is unique.

- **Hyperplane Projection**: Projecting x_0 onto $a^Tx = b$ has the analytical solution:

$$P_C(x_0) = x_0 - \frac{a^Tx_0 - b}{\|a\|_2^2}a$$

- **Halfspace Projection**: Projecting x_0 onto $a^T x \leq b$ has the solution:

$$P_C(x_0) = x_0 - \frac{\max(0, a^T x_0 - b)}{\|a\|_2^2} a$$

2.2 Distance Between Convex Sets

The distance between two closed convex sets C and D is the length of the shortest vector connecting them. We can find the closest points by solving:

$$\begin{array}{ll} \text{minimize} & \|x - y\|_2^2 \\ \text{subject to} & x \in C, \quad y \in D \end{array}$$

This is a convex optimization problem in variables x and y. If C and D do not intersect, their separating hyperplane can be derived directly from the optimal Lagrange multipliers.

2.3 Euclidean Distance and Angle Problems

Many geometric problems involve constraints on the relative distances or angles between points. For instance, we may wish to find the positions of k points $x_1, \ldots, x_k \in \mathbb{R}^n$ given partial bounds on their Euclidean distances:

$$d_{ij}^{\min} \leq \|x_i - x_j\|_2 \leq d_{ij}^{\max}$$

or bounds on the angles between vectors:

$$\theta_{ijk}^{\min} \leq \angle(x_i - x_j, x_k - x_j) \leq \theta_{ijk}^{\max}$$

These bounds can be represented using the **Gram Matrix** $G = X^T X$, where $X = [x_1, \ldots, x_k]$. Since $G \succeq 0$, we can express distance and angle bounds as linear constraints on the entries of G, transforming the coordinate estimation into a Semidefinite Program (SDP) with a rank constraint.

2.4 Extremal Volume Ellipsoids

An ellipsoid $\mathcal{E}$ in $\mathbb{R}^n$ can be represented as:

$$\mathcal{E} = \{x \mid \|Mx + d\|_2 \leq 1\}$$

where $M \in \mathbb{S}_{++}^n$ (symmetric positive definite matrix). The volume of $\mathcal{E}$ is proportional to $\det(M^{-1})$.

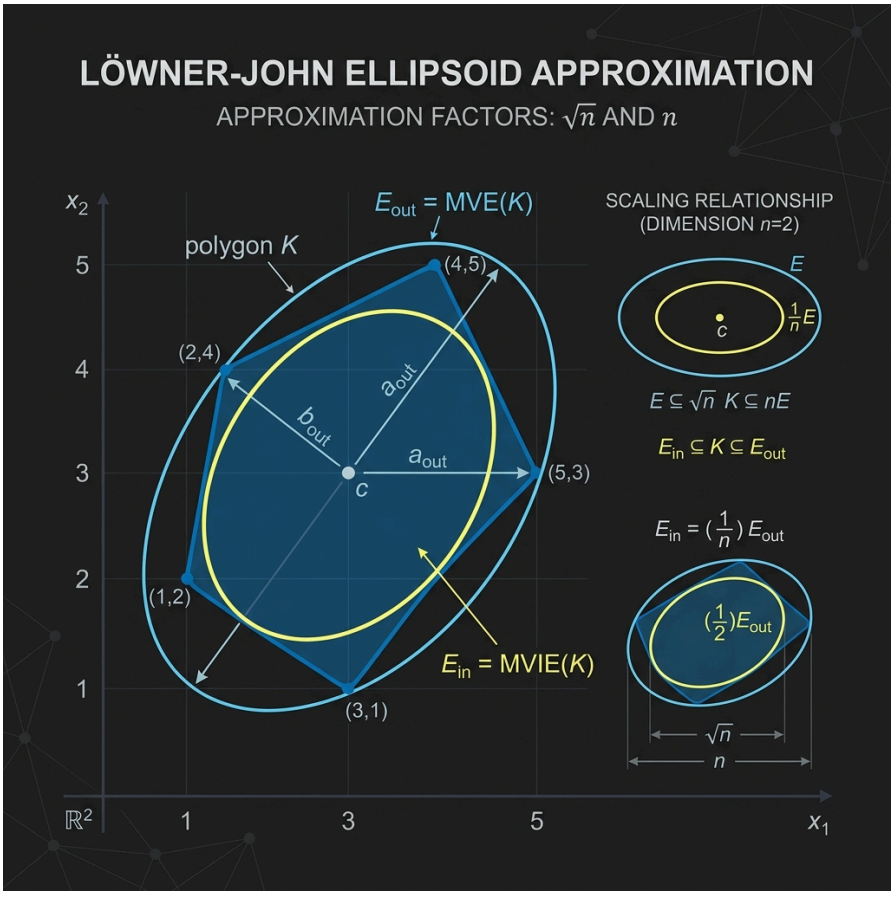
- **Löwner-John Ellipsoid (Minimum Volume enclosing ellipsoid)**: To find the minimum volume ellipsoid containing a set of points $x_1, \ldots, x_N$, we solve:

$$\begin{array}{ll} \text{minimize} & -\ln\det M \\ \text{subject to} & \|Mx_i + d\|_2 \leq 1, \quad i = 1, \ldots, N \\ & M \succ 0 \end{array}$$

  Since $-\ln\det M$ is a convex function on $\mathbb{S}^n_{++}$, this is a convex optimization problem (specifically an SDP).

- **Maximum Volume Inscribed Ellipsoid**: To find the largest ellipsoid that fits inside a polyhedron defined by linear inequalities $a_i^T x \leq b_i$, $i = 1, \ldots, m$, we solve:

$$\begin{array}{ll} \text{minimize} & -\ln\det M \\ \text{subject to} & \|Ma_i\|_2 + a_i^T d \leq b_i, \quad i = 1, \ldots, m \\ & M \succ 0 \end{array}$$



2.5 Centering Problems

Centering problems seek to find a representative "middle" point x inside a convex set C.

- **Chebyshev Center**: The center of the largest Euclidean ball $\mathcal{B}(x_c, r) = \{x \mid \|x - x_c\|_2 \leq r\}$ that can be inscribed in the set C. For a polyhedron $C = \{x \mid a_i^T x \leq b_i,\ i = 1, \ldots, m\}$, the Chebyshev center is found by solving the Linear Program (LP):

$$\begin{array}{ll} \text{minimize} & -r \\ \text{subject to} & a_i^T x_c + r\|a_i\|_2 \leq b_i, \quad i = 1, \ldots, m \\ & r \geq 0 \end{array}$$

- **Analytic Center**: The point that minimizes the logarithmic barrier function for the set of inequalities:

$$\text{minimize} \quad -\sum_{i=1}^{m} \ln(b_i - a_i^T x)$$

  This point is unique and lies strictly in the interior of the polyhedron.

2.6 Classification and Discrimination

We want to separate two sets of points $\{x_1, \ldots, x_N\}$ and $\{y_1, \ldots, y_M\}$ using a hyperplane $a^T z + b = 0$.

- **Linear Discrimination**: The sets are linearly separable if there exists a vector $a \in \mathbb{R}^n$ and a scalar $b \in \mathbb{R}$ such that:

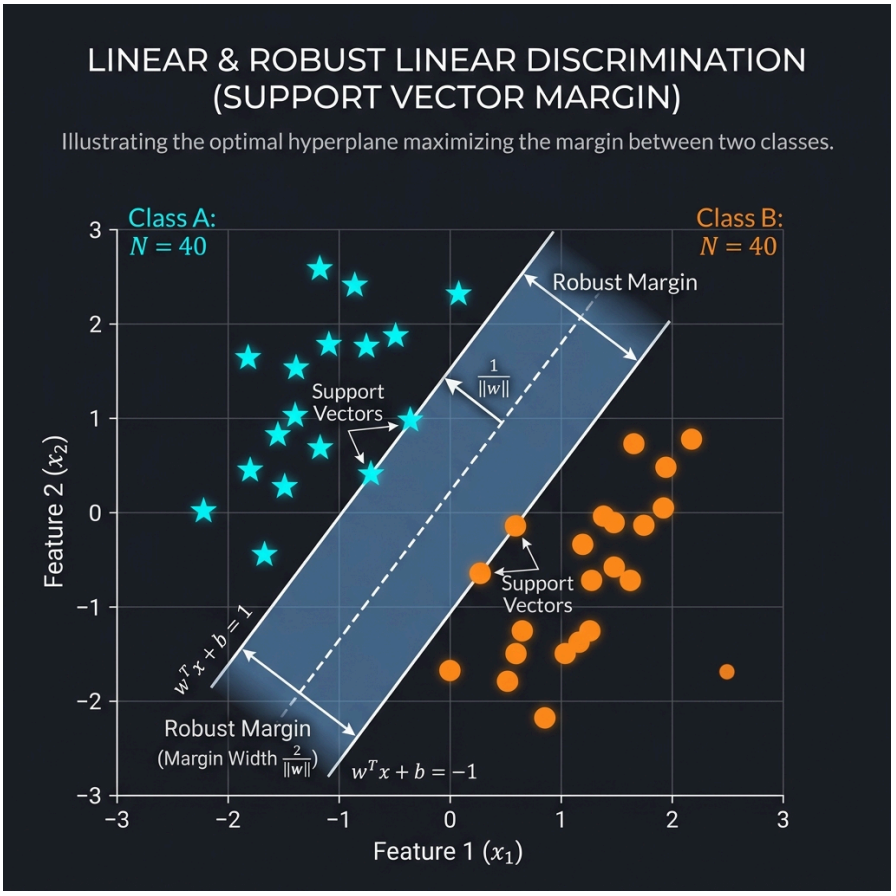
$$a^T x_i + b > 0, \quad i = 1, \ldots, N \quad \text{and} \quad a^T y_j + b < 0, \quad j = 1, \ldots, M$$

  This is solved using linear feasibility methods.

- **Support Vector Classifier (Max-Margin Separation)**: To find the separating hyperplane that maximizes the margin (distance) between the two classes, we solve:

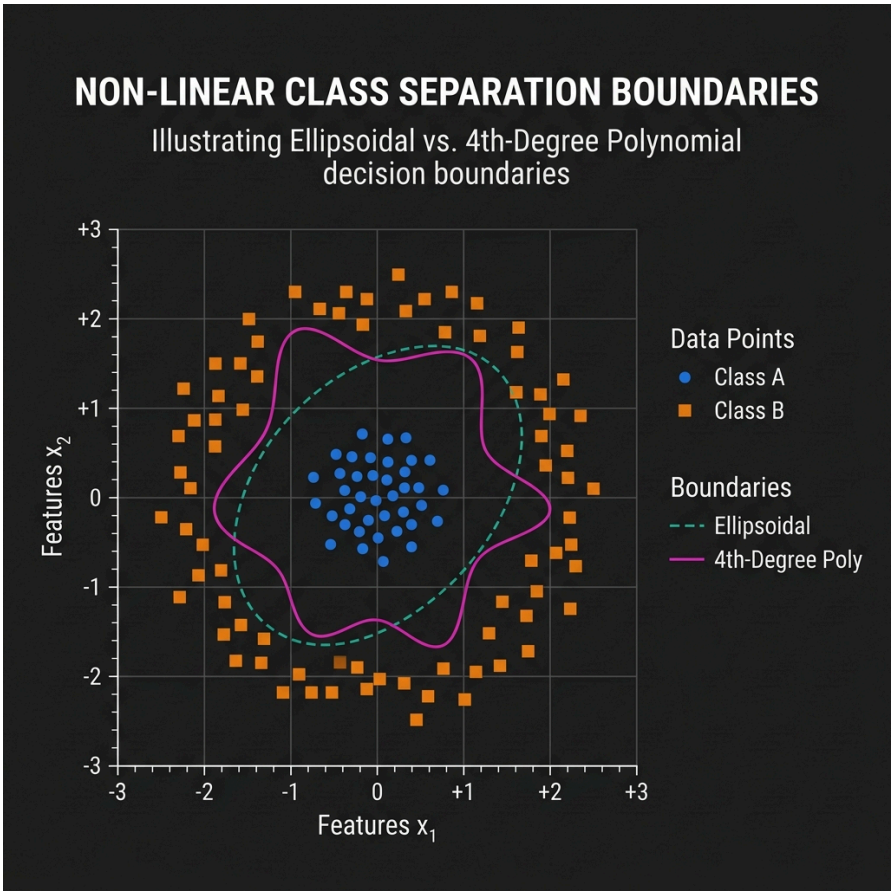
$$\begin{array}{ll} \text{minimize} & \frac{1}{2}\|a\|_2^2 \\ \text{subject to} & a^T x_i + b \geq 1, \quad i = 1, \ldots, N \\ & a^T y_j + b \leq -1, \quad j = 1, \ldots, M \end{array}$$

  This QP formulation is the foundation of Support Vector Machines (SVM).



Non-linear Class Separation

When two classes are not linearly separable, we can project the data into a higher-dimensional space or use non-linear boundary functions (such as ellipsoidal or high-degree polynomial kernels):



2.7 Placement and Location Problems

Placement problems involve positioning a set of nodes $x_1, \ldots, x_k \in \mathbb{R}^n$ in space to minimize a cost function based on the distances between connected nodes. For a graph with edge weights w_{ij}, we minimize the total connection cost:

$$\text{minimize} \quad \sum_{(i,j)\in\mathcal{E}} w_{ij} f(\|x_i - x_j\|_2)$$

where f is a monotonic, convex cost function.

- If $f(d) = d^2$ (squared Euclidean distance), the problem reduces to a set of linear equations (quadratic optimization).
- If $f(d) = d$ (Euclidean distance), the problem is called the multi-dimensional Weber problem, which is formulated as an SOCP.

Python Example: Projecting a Point onto a Polyhedron

Here is a Python script using `scipy.optimize` to project a 2D point onto a polyhedral convex set defined by linear inequalities ($Gx \leq h$):

```python
import numpy as np
from scipy.optimize import minimize

# Point to project
x0 = np.array([3.0, 4.0])

# Define the convex set (polyhedron): Gx <= h
# Inequalities:
# x1 >= 0      =>  -x1 <= 0
# x2 >= 0      =>  -x2 <= 0
# x1 + x2 <= 2 =>   x1 + x2 <= 2
G = np.array([[-1.0, 0.0],
              [0.0, -1.0],
              [1.0, 1.0]])
h = np.array([0.0, 0.0, 2.0])

# Objective function: squared Euclidean distance
def objective(x):
    return np.sum((x - x0) ** 2)

# Constraint definition for Scipy (must be >= 0)
# h - Gx >= 0
def constraint_func(x):
    return h - G @ x

constraints = {'type': 'ineq', 'fun': constraint_func}

# Run minimization
result = minimize(objective, x0, constraints=constraints)

projected_point = result.x
print(f"Point to project: {x0}")
print(f"Projected point:  {projected_point}")
print(f"Distance:         {np.sqrt(result.fun):.4f}")
```

Combinatorial Optimization & Scheduling

Unlike continuous optimization where variables can take any decimal value, **Combinatorial Optimization** deals with problems where the variables are discrete (e.g., integers, binary choices, or ordering permutations).

Many of these problems are **NP-hard**, meaning they cannot be solved to absolute perfection in polynomial time for large datasets. Instead, we use solvers and heuristics.

---

1. The Traveling Salesperson Problem (TSP)

In the **Traveling Salesperson Problem (TSP)**, a salesperson must visit a set of cities exactly once and return to the starting city. The goal is to find the **shortest possible route**.

Formulation

Given a distance matrix D where d_{ij} is the distance from city i to city j, we want to find a permutation of cities π that minimizes:

$$\text{minimize} \quad \sum_{i=1}^{n-1} d_{\pi(i)\pi(i+1)} + d_{\pi(n)\pi(1)}$$

TSP is a classic routing problem used in logistics, package delivery, and microchip manufacturing.

---

2. The Job Shop Scheduling Problem (JSSP)

The **Job Shop Scheduling Problem (JSSP)** is one of the most famous scheduling problems in industrial engineering and operations research.

Problem Definition:

- We have a set of **Jobs** (e.g., manufacturing specific products).
- Each Job consists of a sequence of **Tasks** that must be executed in a strict order.
- Each Task must be processed on a specific **Machine** for a given **duration**.
- **Constraints**:
    1. A machine can only process one task at a time (no overlap).
    2. Tasks of a single job must be processed in sequence.
- **Objective**: Minimize the **makespan** (the total time to complete all jobs).

---

3. Python Implementation: Solving JSSP with Google OR-Tools

We can use Google's **OR-Tools** (Constraint Programming CP-SAT solver) to solve a small JSSP instance.

Instance Definition:

- **Job 0**:
    - Task 1: Machine 0, duration 3
    - Task 2: Machine 1, duration 2
    - Task 3: Machine 2, duration 2
- **Job 1**:
    - Task 1: Machine 0, duration 2
    - Task 2: Machine 2, duration 1
    - Task 3: Machine 1, duration 4
- **Job 2**:

- Task 1: Machine 1, duration 4
- Task 2: Machine 2, duration 3

```
from ortools.sat.python import cp_model

# 1. Initialize CP-SAT model
model = cp_model.CpModel()

# 2. Define data
jobs_data = [
    [(0, 3), (1, 2), (2, 2)],  # Job 0
    [(0, 2), (2, 1), (1, 4)],  # Job 1
    [(1, 4), (2, 3)]           # Job 2
]

num_machines = 3
all_machines = range(num_machines)
all_jobs = range(len(jobs_data))

# Find horizon (upper bound of makespan)
horizon = sum(task[1] for job in jobs_data for task in job)

# 3. Create variables
all_tasks = {}
machine_to_intervals = {m: [] for m in all_machines}

for job_id, job in enumerate(jobs_data):
    for task_id, task in enumerate(job):
        machine = task[0]
        duration = task[1]
        suffix = f"_{job_id}_{task_id}"

        start_var = model.NewIntVar(0, horizon, f"start{suffix}")
        end_var = model.NewIntVar(0, horizon, f"end{suffix}")
        interval_var = model.NewIntervalVar(start_var, duration, end_var, f"interval{suffix}")

        all_tasks[job_id, task_id] = (start_var, end_var, interval_var)
        machine_to_intervals[machine].append(interval_var)

# 4. Add overlap constraints (no two tasks on the same machine can overlap)
for machine in all_machines:
    model.AddNoOverlap(machine_to_intervals[machine])

# 5. Add precedence constraints (tasks of a job must run sequentially)
for job_id, job in enumerate(jobs_data):
    for task_id in range(len(job) - 1):
        model.Add(all_tasks[job_id, task_id + 1][0] >= all_tasks[job_id, task_id][1])

# 6. Define makespan objective (minimize the maximum end time)
makespan = model.NewIntVar(0, horizon, "makespan")
model.AddMaxEquality(makespan, [all_tasks[job_id, len(job) - 1][1] for job_id, job in
enumerate(jobs_data)])
model.Minimize(makespan)

# 7. Solve
solver = cp_model.CpSolver()
status = solver.Solve(model)

if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:
    print(f"Optimal Makespan: {solver.ObjectiveValue()}")
    for job_id, job in enumerate(jobs_data):
        for task_id, task in enumerate(job):
            start = solver.Value(all_tasks[job_id, task_id][0])
            print(f"Job {job_id}, Task {task_id} starts at {start}")
```

Exercises

Exercise 1

Consider a JSSP with 2 machines (M_0, M_1) and 1 job. The job has 2 tasks:

- Task 0: requires M_0, duration = 4.
- Task 1: requires M_1, duration = 3. What is the minimum makespan if Task 1 must follow Task 0?

Solution — Exercise 1

Since there is only one job, and Task 1 must wait for Task 0 to finish:

- Task 0 starts at $t = 0$, ends at $t = 4$.
- Task 1 starts at $t = 4$ (earliest possible), ends at $t = 4 + 3 = 7$. The minimum makespan is **7**.

Functional Analysis & Operator Theory

Functional Analysis is the branch of mathematics that generalizes linear algebra from finite-dimensional vector spaces to infinite-dimensional spaces. It provides the rigorous mathematical framework for quantum mechanics, differential equations, and advanced machine learning models (such as kernel methods, Support Vector Machines, and Reproducing Kernel Hilbert Spaces).

Lecture 1: Basic Banach Space Theory

Normed Vector Spaces

Let X be a vector space over a field K (where $K = \mathbb{R}$ or $\mathbb{C}$). A **norm** on X is a function $\|\cdot\| : X \to \mathbb{R}$ satisfying three axioms for all $x, y \in X$ and $\alpha \in K$:

1. **Positive Definiteness**: $\|x\| \geq 0$, and $\|x\| = 0$ if and only if $x = 0$.
2. **Absolute Homogeneity**: $\|\alpha x\| = |\alpha| \|x\|$.
3. **Triangle Inequality**: $\|x + y\| \leq \|x\| + \|y\|$.

A vector space equipped with a norm is called a **normed vector space** $(X, \|\cdot\|)$.

Completeness and Banach Spaces

A sequence $\{x_n\}$ in a normed space X is a **Cauchy sequence** if for every $\epsilon > 0$, there exists an integer N such that:

$$\|x_n - x_m\| < \epsilon \quad \text{for all } n, m \geq N$$

A normed space X is **complete** if every Cauchy sequence in X converges to a limit that also lies within X.

- **Banach Space**: A complete normed vector space is called a **Banach space**.
- **Examples**:
  - **Finite-Dimensional**: $\mathbb{R}^n$ and $\mathbb{C}^n$ are complete under any norm.
  - **Sequence Spaces**: l^p spaces (the set of sequences whose p-th power sums are finite) are Banach spaces under the norm $\|x\|_p = (\sum |x_i|^p)^{1/p}$.
  - **Continuous Functions**: $C([a, b])$, the space of continuous functions on an interval, is a Banach space under the supremum norm $\|f\|_\infty = \sup_{t \in [a,b]} |f(t)|$.

Lecture 2: Bounded Linear Operators

A function $T : X \to Y$ between two normed vector spaces is a **linear operator** if $T(\alpha x + \beta y) = \alpha Tx + \beta Ty$ for all $x, y \in X$ and scalars α, β.

Boundedness and Continuity

A linear operator $T : X \to Y$ is **bounded** if there exists a constant $M \geq 0$ such that:

$$\|Tx\|_Y \leq M\|x\|_X \quad \text{for all } x \in X$$

- **Theorem**: For a linear operator T, the following statements are equivalent:
  1. T is continuous at $x = 0$.
  2. T is continuous everywhere on X.
  3. T is bounded.

The Operator Norm

For a bounded linear operator $T : X \to Y$, the **operator norm** $\|T\|$ is defined as the supremum of the ratio of the output norm to the input norm:

$$\|T\| = \sup_{x \neq 0} \frac{\|Tx\|_Y}{\|x\|_X} = \sup_{\|x\|_X \leq 1} \|Tx\|_Y$$

The set of all bounded linear operators from X to Y is denoted as $B(X, Y)$. If Y is a Banach space, then $B(X, Y)$ is also a Banach space under the operator norm.

Lecture 3: Quotient Spaces & The Baire Category Theorem

Quotient Spaces

Let X be a normed space and $M \subseteq X$ be a closed subspace. The **quotient space** X/M is the set of equivalence classes (cosets) $[x] = x + M$ under the norm:

$$\|[x]\| = \inf_{m \in M} \|x - m\|$$

- **Theorem**: If X is a Banach space, then the quotient space X/M is also a Banach space under this quotient norm.

Baire Category Theorem

The Baire Category Theorem is a foundational topological property of complete metric spaces.

- **Theorem**: Let X be a complete metric space. If X is written as a countable union of closed subsets $X = \bigcup_{n=1}^{\infty} F_n$, then at least one of the subsets F_n must have a non-empty interior.
- **Intuition**: A complete space cannot be constructed by gluing together a countable number of "thin" (nowhere dense) sets.

Uniform Boundedness Principle (Banach-Steinhaus Theorem)

Using Baire's theorem, we can prove that pointwise boundedness of a family of operators implies uniform boundedness.

- **Theorem**: Let X be a Banach space and Y be a normed space. Let $\mathcal{F} \subseteq B(X, Y)$ be a family of bounded linear operators. If for each $x \in X$, there exists $C_x \geq 0$ such that: $$ $\|Tx\|_Y \leq C_x \quad \text{for all } T \in \mathcal{F}$$ then the operators are uniformly bounded:

$$\sup_{T \in \mathcal{F}} \|T\| < \infty$$

Lecture 4: The Open Mapping & Closed Graph Theorems

These theorems describe the properties of linear operators between Banach spaces.

The Open Mapping Theorem

A mapping $T : X \to Y$ is an **open map** if the image of every open set in X is an open set in Y.

- **Theorem**: Let X and Y be Banach spaces. If $T : X \to Y$ is a bounded linear operator that is surjective (onto), then T is an open mapping.
- **Bounded Inverse Theorem**: If a bounded linear operator $T : X \to Y$ between Banach spaces is bijective (one-to-one and onto), then its inverse $T^{-1} : Y \to X$ is automatically bounded.

The Closed Graph Theorem

Let $T : X \to Y$ be an operator. The **graph** of T is the set $\Gamma(T) = \{(x, Tx) \mid x \in X\} \subseteq X \times Y$. We say T has a **closed graph** if $\Gamma(T)$ is a closed subspace of $X \times Y$ under the product norm.

- **Theorem**: Let X and Y be Banach spaces. A linear operator $T : X \to Y$ is bounded if and only if its graph is closed.
- **Practical Use**: To prove an operator is continuous, we only need to show that if $x_n \to 0$ and $Tx_n \to y$, then $y = 0$.

Lecture 5: Zorn's Lemma & The Hahn-Banach Theorem

The Hahn-Banach Theorem guarantees that there are "enough" continuous linear functionals on a normed space to separate points.

Zorn's Lemma

Zorn's Lemma is an axiom equivalent to the Axiom of Choice.

- **Poset**: A set equipped with a partial order $\leq$.
- **Chain**: A subset of a poset where every pair of elements is comparable.
- **Theorem**: If every chain in a poset P has an upper bound in P, then P contains at least one maximal element.

The Hahn-Banach Extension Theorem

Let X be a vector space and $p : X \to \mathbb{R}$ be a sublinear functional (satisfying $p(x + y) \leq p(x) + p(y)$ and $p(\alpha x) = \alpha p(x)$ for $\alpha \geq 0$).

- **Theorem (Real Vector Spaces)**: Let $M \subseteq X$ be a subspace, and $f : M \to \mathbb{R}$ be a linear functional bounded by p: $f(x) \leq p(x) \quad \text{for all } x \in MThenthereexistsalinearfunctional$F: X \to \mathbb{R}$extendingf$(F(x) = f(x)$forallx$ \in M$) such that:

$$F(x) \leq p(x) \quad \text{for all } x \in X$$

- **Normed Spaces Corollary**: Let M be a subspace of a normed space X, and $f \in M^*$. Then f can be extended to a functional $F \in X^*$ such that the norms are preserved: $\|F\|_{X^*} = \|f\|_{M^*}$

Lecture 6 (Part 1): The Double Dual & Reflexivity

The Dual Space (X^*)

For a normed space X, the space of all bounded linear functionals $f : X \to K$ is the **dual space** $X^* = B(X, K)$. Since the field K ($\mathbb{R}$ or $\mathbb{C}$) is complete, X^* is always a Banach space.

The Double Dual (X^{**})

The **double dual** (or bidual) $X^{**} = (X^*)^*$ is the dual of X^*.

There is a natural map called the **canonical injection** $J : X \to X^{**}$ defined by evaluating a functional at a point x:

$$J(x)(f) = f(x) \quad \text{for all } f \in X^*$$

- **Theorem**: The canonical injection J is an isometric isomorphism (it preserves norms: $\|J(x)\|_{X^{**}} = \|x\|_X$).
- **Reflexive Spaces**: If J is surjective (meaning $J(X) = X^{**}$), the Banach space X is said to be **reflexive**.
- **Examples**: All Hilbert spaces and L^p spaces for $1 < p < \infty$ are reflexive. The spaces L^1 and $C([a, b])$ are not reflexive.

Measure Theory & Lebesgue Integration

Classical Riemann integration is sufficient for basic functions, but fails for highly discontinuous functions (such as the Dirichlet function). **Measure Theory** provides the rigorous generalization of length, area, and volume, underpinning modern Probability Theory and integration.

Lecture 6 (Part 2): Outer Measure

To measure the “size” of arbitrary subsets of real numbers, we introduce the concept of **Outer Measure**.

Definition of Outer Measure

The **Lebesgue outer measure** $\mu^* : \mathcal{P}(\mathbb{R}) \to [0, \infty]$ of a subset $A \subseteq \mathbb{R}$ is the infimum of the sum of lengths of open intervals that cover A:

$$\mu^*(A) = \inf \left\{ \sum_{n=1}^{\infty} (b_n - a_n) \mid A \subseteq \bigcup_{n=1}^{\infty} (a_n, b_n) \right\}$$

Properties of Outer Measure

1. **Empty Set**: $\mu^*(\emptyset) = 0$.
2. **Monotonicity**: If $A \subseteq B$, then $\mu^*(A) \leq \mu^*(B)$.
3. **Countable Subadditivity**: For any countable collection of sets $\{A_n\}$, the outer measure of their union is bounded by the sum of their individual outer measures:

$$\mu^* \left(\bigcup_{n=1}^{\infty} A_n \right) \leq \sum_{n=1}^{\infty} \mu^*(A_n)$$

Lecture 7: Sigma Algebras (σ-algebras)

We cannot define a consistent measure on *all* subsets of $\mathbb{R}$ while preserving translation-invariance and countable additivity (due to Vitali sets). Therefore, we must restrict our measure to a collection of "well-behaved" sets called a σ-algebra.

Definition of a σ-algebra

Let X be a set. A collection Σ of subsets of X is a **σ-algebra** if it satisfies three axioms:

1. **Contains the Whole Set**: $X \in \Sigma$.
2. **Closed under Complement**: If $A \in \Sigma$, then $A^c = X \setminus A \in \Sigma$.
3. **Closed under Countable Unions**: If $\{A_n\}_{n=1}^{\infty} \subseteq \Sigma$, then:

$$\bigcup_{n=1}^{\infty} A_n \in \Sigma$$

The Borel σ-algebra ($\mathcal{B}(\mathbb{R})$)

The **Borel σ-algebra** on $\mathbb{R}$ is the smallest σ-algebra that contains all open intervals (a, b). It includes all open sets, closed sets (as complements of open sets), countable intersections of open sets (G_δ sets), and countable unions of closed sets (F_σ sets).

Lecture 8: Lebesgue Measurable Sets & Measure

Carathéodory's Condition

To select the well-behaved subsets of $\mathbb{R}$, we use Constantin Carathéodory's slicing condition.

A set $E \subseteq \mathbb{R}$ is **Lebesgue measurable** if it splits any arbitrary "test set" $A \subseteq \mathbb{R}$ additively:

$$\mu^*(A) = \mu^*(A \cap E) + \mu^*(A \cap E^c)$$

The collection of all Lebesgue measurable sets forms a σ-algebra, denoted by $\mathcal{M}$.

The Lebesgue Measure (μ)

When we restrict the outer measure μ^* to the σ-algebra of measurable sets $\mathcal{M}$, it is called the **Lebesgue measure** μ. Unlike outer measure, the Lebesgue measure is **countably additive**:

If $\{E_n\}_{n=1}^{\infty}$ is a countable collection of pairwise disjoint measurable sets, then:

$$\mu\left(\bigcup_{n=1}^{\infty} E_n\right) = \sum_{n=1}^{\infty} \mu(E_n)$$

- **Null Sets**: A set N is a null set if $\mu^*(N) = 0$. Every subset of a null set is Lebesgue measurable (with measure 0), making the Lebesgue measure space complete.

Lecture 9: Lebesgue Measurable Functions

In integration, we integrate functions. For a function to be integrable, it must map measurable sets to measurable sets.

Definition of Measurable Functions

Let (X, Σ) be a measurable space. A function $f : X \to \overline{\mathbb{R}}$ (where $\overline{\mathbb{R}} = \mathbb{R} \cup \{-\infty, \infty\}$) is **measurable** if the preimage of every open interval is a measurable set:

$$f^{-1}((a, \infty)) = \{x \in X \mid f(x) > a\} \in \Sigma \quad \text{for all } a \in \mathbb{R}$$

- **Theorem**: If $\{f_n\}$ is a sequence of measurable functions, then the functions $\inf f_n$, $\sup f_n$, $\liminf f_n$, and $\limsup f_n$ are also measurable. If the pointwise limit $f(x) = \lim_{n\to\infty} f_n(x)$ exists, then f is measurable.

Lecture 10: Simple Functions & The Lebesgue Integral

To construct the Lebesgue integral, we build it from the bottom up, starting with simple step-like functions.

Simple Functions

A **simple function** $\phi : X \to \mathbb{R}$ is a real-valued function that takes only a finite number of distinct values. It can be written in its **canonical representation**:

$$\phi(x) = \sum_{i=1}^{n} c_i \chi_{E_i}(x)$$

where:

- c_i are distinct real numbers.
- $E_i = \{x \in X \mid \phi(x) = c_i\}$ are pairwise disjoint measurable sets.
- χ_{E_i} is the **indicator function** of E_i ($\chi_{E_i}(x) = 1$ if $x \in E_i$, and 0 otherwise).

The Lebesgue Integral of Simple Functions

The **Lebesgue integral** of a non-negative simple function ϕ over a measurable set E is defined as:

$$\int_E \phi \, d\mu = \sum_{i=1}^{n} c_i \mu(E_i \cap E)$$

- **Constructing the General Lebesgue Integral**: For any non-negative measurable function f, its Lebesgue integral is defined as the supremum of the integrals of all simple functions bounded by f: $$\int_E f \, d\mu = \sup \left\{ \int_E \phi \, d\mu \mid 0 \le \phi \le f, \, \phi \text{ is simple} \right\}$$

Introduction

Topology is the branch of mathematics that studies geometric properties and spatial relations that remain unaffected by the continuous change of shape or size of figures. Often described as **“rubber-sheet geometry,”** topology focuses on qualitative properties like connectivity, holes, and boundaries rather than quantitative measurements like distance, coordinates, or angles.

In data science, machine learning, and deep learning, topology provides robust tools to understand the fundamental “shape” of high-dimensional data, optimize neural network structures, and perform noise-resistant feature engineering.

Core Intuitive Concepts in Topology

Before diving into formal definitions, it is helpful to understand the core intuitive concepts that topologists use to analyze spaces:

- **Topological Equivalence (Homeomorphism)**: In topology, two shapes are considered “equivalent” (or homeomorphic) if one can be continuously deformed, stretched, bent, or twisted into the other without tearing, punching holes, or gluing parts together.

  > **Note**
  >
  > A classic joke is that a topologist cannot tell a **coffee mug** apart from a **donut** (torus). Because both shapes contain exactly one hole, a clay coffee mug can be continuously morphed into a donut shape without tearing or pasting.

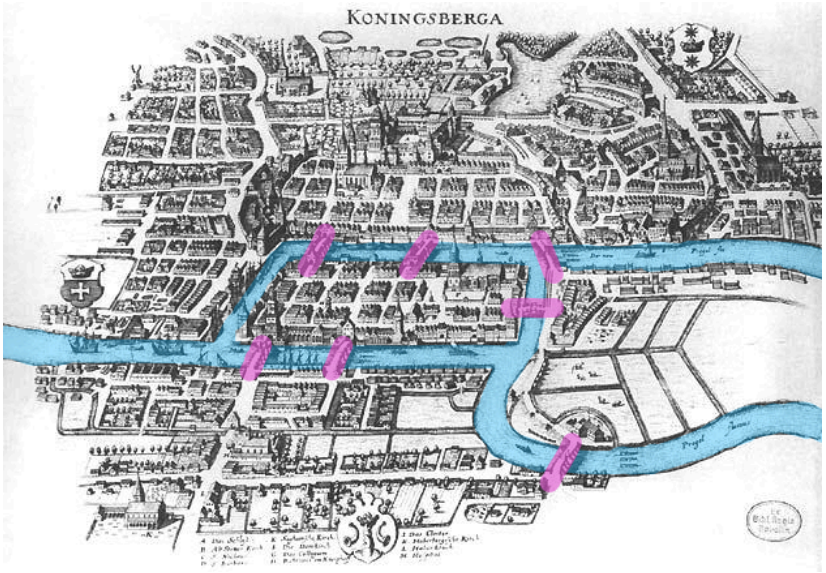
- **Topological Invariants**: These are properties of a geometric object that remain unchanged under any continuous deformation. Examples include:
  - **The number of holes** (Betti numbers)
  - **The number of connected components**
  - **Euler Characteristic** (χ)

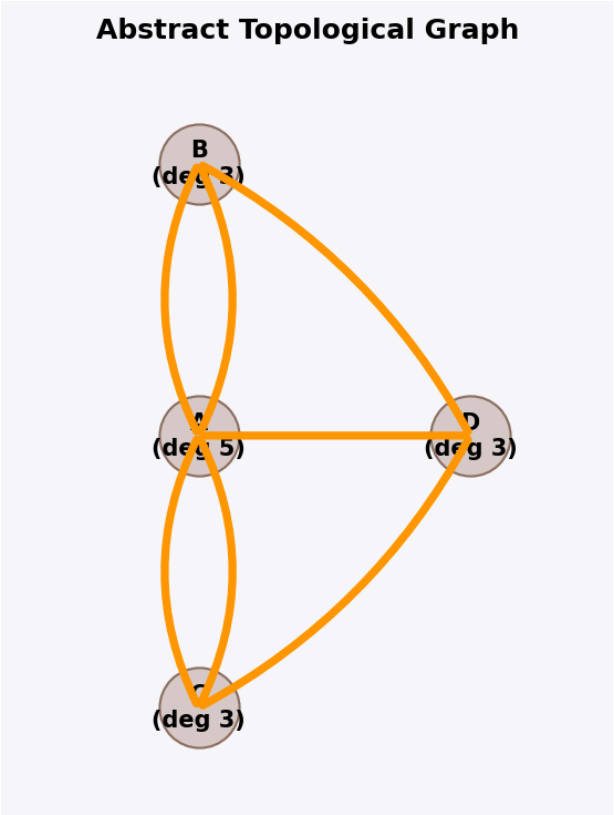
  If two spaces have different topological invariants, they cannot be continuously deformed into one another.

- **Orientability**: A surface is **orientable** if it has two distinct sides (e.g., an "inside" and an "outside" of a hollow sphere or balloon). A surface is **non-orientable** if it has only one side. As we will see later, structures like the **Möbius strip** and the **Klein bottle** are famous examples of non-orientable manifolds where the concepts of "inside" and "outside" collapse completely.
- **Coordinate-Free representation**: Traditional geometry describes objects using coordinate equations (like $x^2 + y^2 = r^2$ for a circle). Topology abstractly defines spaces based on how points are grouped together. This is highly useful in Data Science because real-world data is often noisy and shifting, but the global "shape" and connectivity of the data points contain the true underlying signal.

1. Historical Origin: The Seven Bridges of Königsberg

The mathematical foundation of both **Graph Theory** and **Topology** began in 1736 when Leonhard Euler solved the famous puzzle known as **The Seven Bridges of Königsberg**.





- **The Problem**: The city of Königsberg (now Kaliningrad, Russia) was divided by the Pregel River into four landmasses (islands A and banks B, C, D), connected by seven bridges. The challenge was to find a walk through the city that crossed each of the seven bridges **exactly once** and returned to the starting point.
- **Euler's Insight**: Euler realized that the shape of the landmasses and the length of the bridges were completely irrelevant to the solution. By representing landmasses as vertices (nodes) and bridges as edges, he simplified the map into a network (graph).

- **The Solution**: Euler proved that a path crossing every edge exactly once (now called an **Eulerian Path**) is only possible if the number of vertices with an odd number of edges (odd degree) is either 0 or 2:
  - In Königsberg, all four landmasses had an odd number of bridges connecting them (degrees 5, 3, 3, and 3).
  - Therefore, it was mathematically impossible to complete the walk.
- **The Birth of Topology**: By discarding coordinates, distances, and angles, and focusing purely on the connectivity and relationships between points, Euler's analysis was the very first paper in the history of topology—originally referred to as *geometry of position* (geometria situs).

---

2. Major Subfields of Topology

Modern topology is broadly divided into four major subfields, each focusing on different mathematical tools and structures:

1. **General Topology (Point-Set Topology)**:
   - **Focus**: Studies the foundational definitions of topological spaces, open/closed sets, neighborhoods, continuity, convergence, compactness, and connectedness. It generalizes the concepts of calculus without needing a distance coordinate system.
   - **Data Science Link**: Forms the mathematical basis for metric spaces and continuous transformations used in data representation.
2. **Algebraic Topology**:
   - **Focus**: Uses tools from abstract algebra (such as groups, rings, and homology) to analyze and classify topological spaces. It measures the "shapes" and counts the number of holes of different dimensions in a space.
   - **Data Science Link**: Directly powers **Topological Data Analysis (TDA)** and **Persistent Homology** (using Betti numbers to identify true data patterns vs. noise).
3. **Differential Topology**:
   - **Focus**: Studies differentiable (smooth) manifolds and differentiable functions. It deals with smooth structures, tangent vectors, and vector fields on manifolds without relying on metric distances.
   - **Data Science Link**: Essential for manifold optimization, gradient descent on Riemannian manifolds, and deep learning architectures like Spherical CNNs.
4. **Geometric Topology**:
   - **Focus**: Studies manifolds and their embeddings in other manifolds, with a strong focus on low-dimensional spaces (dimensions 2, 3, and 4), orientability, and knot theory.
   - **Data Science Link**: Explains complex, non-orientable surfaces (like the **Möbius strip** and **Klein bottle**) which serve as testing grounds for non-linear dimensionality reduction methods (like UMAP and Isomap).

Topological Spaces & Metric Spaces

This section covers the mathematical formulations of Point-Set Topology, defining how spaces are constructed, how limits and closeness are generalized, and how structure-preserving maps are defined.

---

1. What is a Topological Space?

Formally, a **topological space** is defined as a pair **(X, τ)**, where **X** is a set and **τ** is a collection of subsets of **X** (called the **topology** on **X**) that satisfies the following three axioms:

1. **Trivial Sets**: Both the empty set (∅) and the set **X** itself must belong to **τ**:

$$\emptyset \in \tau \quad \text{and} \quad X \in \tau$$

2. **Arbitrary Unions**: The union of any collection of sets in **τ** must also belong to **τ**:

$$\bigcup_{U \in S} U \in \tau \quad \text{for any } S \subseteq \tau$$

3. **Finite Intersections**: The intersection of any finite collection of sets in **τ** must also belong to **τ**:

$$\bigcap_{i=1}^{n} U_i \in \tau \quad \text{where } U_i \in \tau \text{ for all } i$$

The subsets inside **τ** are called the **open sets** of the space **X**. By abstractly defining which sets are “open”, a topology defines the local structure of the space (such as neighborhoods, limits, and convergence) without needing coordinates or distance measurements.

---

2. Metric Spaces: A Concrete Example

A **metric space** is a specific type of topological space where open sets are defined using a distance function (metric).

A metric space is a set X equipped with a metric $d : X \times X \to \mathbb{R}$ that satisfies four axioms for all points $x, y, z \in X$:

1. **Non-negativity**: $d(x, y) \geq 0$
2. **Identity of Indiscernibles**: $d(x, y) = 0 \iff x = y$
3. **Symmetry**: $d(x, y) = d(y, x)$
4. **Triangle Inequality**: $d(x, z) \leq d(x, y) + d(y, z)$

In a metric space, we define an **open ball** of radius $r > 0$ centered at x as:

$$B(x, r) = \{y \in X \mid d(x, y) < r\}$$

A set $U \subseteq X$ is then defined as **open** if every point $x \in U$ is the center of some open ball completely contained inside U. The collection of all such open sets forms a valid topology τ on X.

Common metrics in Data Science include:

- **Euclidean Distance (L_2)**: $d(x, y) = \sqrt{\sum (x_i - y_i)^2}$
- **Manhattan Distance (L_1)**: $d(x, y) = \sum |x_i - y_i|$
- **Cosine Distance**: $d(x, y) = 1 - \frac{x \cdot y}{\|x\| \|y\|}$

---

3. Continuous Functions and Homeomorphisms

In topology, the equivalent of a structure-preserving map between two spaces is a **continuous function**:

- **Continuous Function**: A function $f : X \to Y$ between two topological spaces is continuous if the preimage of every open set in Y is an open set in X:

$$f^{-1}(V) \in \tau_X \quad \text{for every open } V \subseteq Y$$

- **Homeomorphism**: A **homeomorphism** (or topological isomorphism) is a bijective function $f : X \to Y$ such that both f and its inverse f^{-1} are continuous.

If a homeomorphism exists between X and Y, the two spaces are said to be **homeomorphic** (topologically equivalent). They share all topological properties, meaning one can be continuously morphed into the other without tearing, cutting, or gluing (e.g., a donut morphing into a coffee cup).

---

4. Compactness and Connectedness

Using open sets, we can define two fundamental topological properties used in optimization and machine learning:

- **Compactness**: A space is compact if every open cover has a finite subcover (in $\mathbb{R}^n$, this means the set is closed and bounded). Compactness guarantees that continuous functions reach their minimum and maximum values—a crucial property for optimization algorithms.
- **Connectedness**: A space is connected if it cannot be partitioned into two disjoint open sets. Connectedness forms the mathematical backbone of clustering algorithms (e.g., Single Linkage Hierarchical Clustering).

---

5. Simplicial Homology

To formally count the number of holes (Betti numbers) in a geometric space, we use the algebraic tools of **Simplicial Homology**. Homology groups translate geometric connectivity into the language of vector spaces and linear algebra.

A. Chains and Chain Groups (C_k)

Let K be a simplicial complex. A **k-chain** is a formal linear combination of k-simplices in K. In computational topology, we typically use coefficients in the field of integers modulo 2 ($\mathbb{Z}_2 = \{0, 1\}$), where addition is equivalent to the exclusive-OR (XOR) operation.

A k-chain c is written as:

$$c = \sum_i a_i \sigma_i \quad \text{where } a_i \in \mathbb{Z}_2 \text{ and } \sigma_i \text{ is a } k\text{-simplex}$$

The set of all k-chains forms a vector space (or abelian group) called the **k-th Chain Group**, denoted as $C_k(K)$.

B. The Boundary Operator (∂_k)

The **boundary operator** is a linear transformation $\partial_k : C_k(K) \to C_{k-1}(K)$ that maps a k-simplex to its boundary faces.

For an oriented k-simplex spanned by vertices $(v_0, v_1, \ldots, v_k)$, the boundary operator is defined as:

$$\partial_k(v_0, v_1, \ldots, v_k) = \sum_{j=0}^{k} (-1)^j (v_0, \ldots, \hat{v}_j, \ldots, v_k)$$

where $\hat{v}_j$ indicates that the vertex v_j is omitted. With $\mathbb{Z}_2$ coefficients, signs do not matter, simplifying the equation to:

$$\partial_k(v_0, v_1, \ldots, v_k) = \sum_{j=0}^{k} (v_0, \ldots, \hat{v}_j, \ldots, v_k)$$

- **Example**: The boundary of a 1-simplex (edge) $e = (v_0, v_1)$ is $\partial_1(v_0, v_1) = v_0 + v_1$ (its two endpoint vertices). The boundary of a 2-simplex (triangle) (v_0, v_1, v_2) is $(v_1, v_2) + (v_0, v_2) + (v_0, v_1)$ (its three bounding edges).

C. The Fundamental Lemma: Boundaries Have No Boundary

A critical property of the boundary operator is that applying it twice in succession always yields zero:

$$\partial_{k-1} \circ \partial_k = 0$$

- **Intuition**: The boundary of a triangle consists of three edges forming a closed loop. The boundary of this loop (the endpoints of the edges) cancels out pairwise under $\mathbb{Z}_2$ addition, resulting in zero.

D. Cycles (Z_k), Boundaries (B_k), and Homology Groups (H_k)

Because the boundary of a boundary is zero, we can define two key subspaces of $C_k(K)$:

1. **k-Cycles ($Z_k(K)$)**: The set of all k-chains with empty boundaries (representing closed loops or shells). This is the kernel of the boundary operator:

$$Z_k(K) = \ker(\partial_k) = \{c \in C_k(K) \mid \partial_k(c) = 0\}$$

2. **k-Boundaries ($B_k(K)$)**: The set of all k-chains that are themselves the boundary of some higher $(k+1)$-chain. This is the image of the boundary operator:

$$B_k(K) = \text{im}(\partial_{k+1}) = \{\partial_{k+1}(d) \mid d \in C_{k+1}(K)\}$$

Since $\partial^2 = 0$, every boundary is automatically a cycle. Therefore, $B_k(K)$ is a subspace of $Z_k(K)$:

$$B_k(K) \subseteq Z_k(K)$$

The **k-th Homology Group** $H_k(K)$ is defined as the quotient space of cycles modulo boundaries:

$$H_k(K) = Z_k(K)/B_k(K) = \ker(\partial_k)/\text{im}(\partial_{k+1})$$

The dimension of this quotient vector space is the k-th Betti number, representing the count of independent k-dimensional holes in the space:

$$\beta_k = \dim H_k(K)$$

Manifolds & Dimensionality Reduction

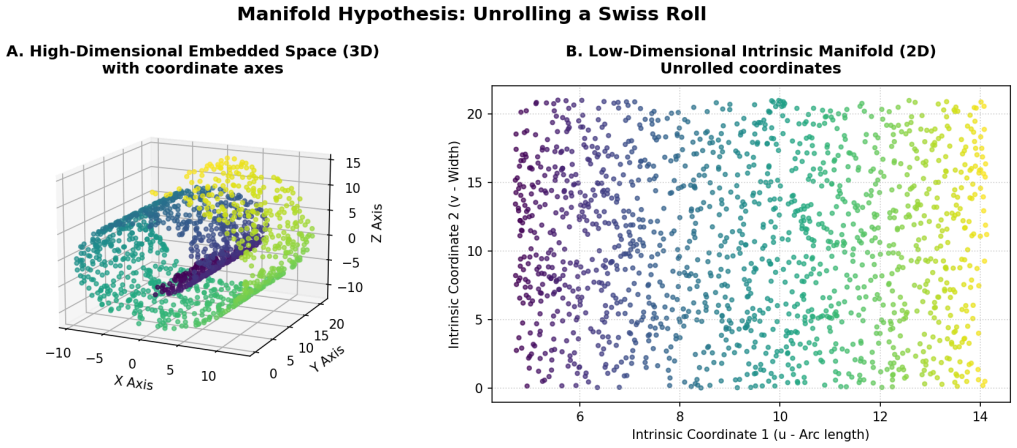
This section explores how manifold theory simplifies high-dimensional data representation, avoids numerical singularities, and enables smooth orientation changes in 3D applications.

---

1. What is a Manifold?

Formally, a **manifold** M of dimension n is a topological space that locally resembles Euclidean space $\mathbb{R}^n$ near each point. That is, every point $p \in M$ has a neighborhood $U \subseteq M$ that is **homeomorphic** to an open ball in $\mathbb{R}^n$.

- **Intuition**: The Earth is a 2D sphere (manifold) embedded in 3D space, but locally it looks like a flat 2D plane to us because any small patch of the Earth is homeomorphic to a flat disc.



2. The Manifold Hypothesis in Data Science

> **Note**
>
> **The Manifold Hypothesis** states that real-world high-dimensional data (such as images, text embeddings, or sensor signals) actually lies on or near a lower-dimensional **manifold** embedded within the high-dimensional space.

- **Deep Learning Application**: In Deep Learning, a Feedforward Neural Network can be viewed as learning a sequence of homeomorphisms. It continuously deforms the input space (stretching, bending, and folding) so that complex, non-linearly separable data becomes linearly separable at the final layer.

Topological Dimensionality Reduction: UMAP

While PCA assumes linear relations, algorithms like **t-SNE** and **UMAP (Uniform Manifold Approximation and Projection)** preserve local and global topological structures.

- **UMAP** assumes that the data lies on a local Riemannian manifold and uses fuzzy simplicial complexes to represent its topology. It then projects the data into a lower-dimensional space while minimizing the cross-entropy between the topological representations.

3. Real-World Application: 3D Game Engines & Gimbal Lock

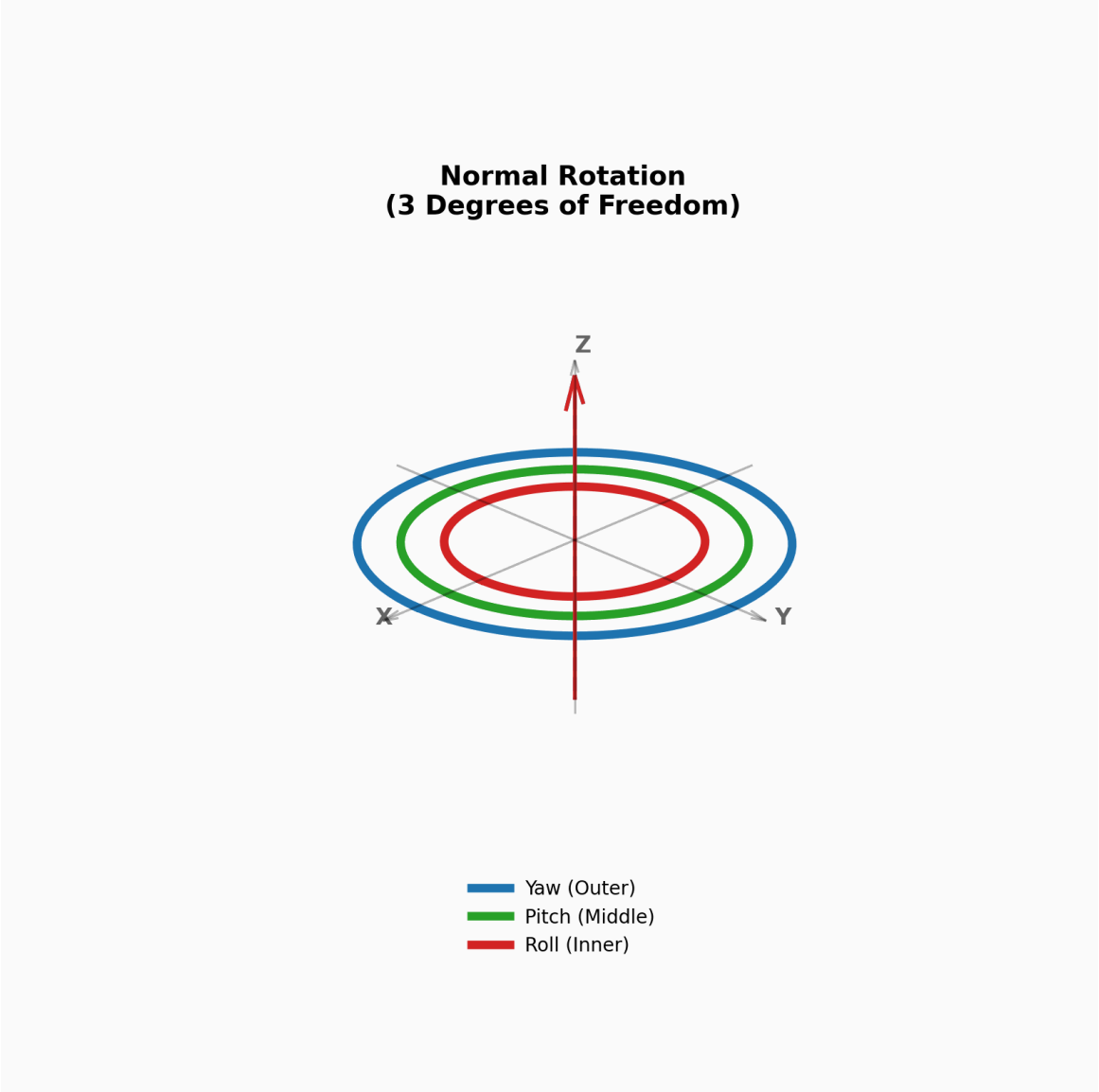
In game development, 3D computer graphics, and aerospace engineering, a famous manifestation of coordinate singularities is **Gimbal Lock**.

The Problem: Euler Angles and Gimbal Lock

To represent 3D rotation, the most intuitive method is using **Euler Angles** (Roll, Pitch, Yaw), which rotates an object sequentially around three orthogonal axes (e.g., first X, then Y, then Z).

Mathematically, Euler angles attempt to map the 3D rotation group (a 3D manifold called $SO(3)$) using only three coordinates. Just like mapping the 2D surface of the Earth collapses at the poles, any 3-parameter coordinate system for 3D rotations *must* contain coordinate singularities.

> **Warning**
>
> **Gimbal Lock** occurs when the Pitch (rotation around the Y-axis) reaches exactly 90° or -90°. The first and third axes of rotation (Roll and Yaw) align perfectly, collapsing the 3D rotation space locally into 2D. The system loses one degree of freedom and cannot rotate along the lost axis without experiencing unnatural jumps or distortions.



The Solution: Quaternions

To avoid this singularity, modern game engines (such as Unity, Unreal Engine, and Godot) represent rotations using **Quaternions** instead of Euler angles.

A quaternion is a 4-dimensional hypercomplex number:

$$q = w + xi + yj + zk \quad \text{where} \quad \|q\| = 1$$

- **Topological View**: The set of unit quaternions forms a **3-dimensional sphere (S^3)** embedded in 4-dimensional space $\mathbb{R}^4$. The 3-sphere S^3 acts as a double-cover of the 3D rotation group $SO(3)$.
- **Singularity-Free**: Because quaternions use 4 parameters to describe a 3D manifold, they avoid coordinate singularities entirely. There is no Gimbal Lock.
- **Smooth Interpolation**: Using quaternions, game engines can perform **SLERP (Spherical Linear Interpolation)**. This allows character models, camera rigs, and physics objects to rotate smoothly between any two orientations without visual glitches or acceleration spikes.

```python
import numpy as np
from scipy.spatial.transform import Rotation as R
from scipy.spatial.transform import Slerp

# 1. Define orientation using Euler angles (Roll, Pitch, Yaw)
# Pitch at 90 degrees triggers Gimbal Lock in traditional Euler systems
euler_angles = [0, 90, 0]  # Degrees
rotation = R.from_euler('xyz', euler_angles, degrees=True)

# 2. Convert to Quaternion [x, y, z, w] to resolve Gimbal Lock
quaternion = rotation.as_quat()
print(f"Gimbal Lock resolved. Quaternion representation: {quaternion}")

# 3. Perform Smooth Rotation Interpolation (SLERP)
# Define start and end orientations
rot_start = R.from_euler('xyz', [0, 0, 0], degrees=True)
rot_end = R.from_euler('xyz', [90, 45, 90], degrees=True)

# Set up SLERP interpolation
times = [0.0, 1.0]
key_rotations = R.from_quat([rot_start.as_quat(), rot_end.as_quat()])
slerp_operator = Slerp(times, key_rotations)

# Interpolate smoothly at t = 0.5 (halfway orientation)
interpolated_rot = slerp_operator(0.5)
print(f"Smoothly interpolated Quaternion: {interpolated_rot.as_quat()}")
```

Topological Data Analysis (TDA) & Persistent Homology

Topological Data Analysis (TDA) treats data points as a cloud from which we want to extract the underlying shape and connectivity.

---

1. Simplicial Complexes & Nerve Theorem

To analyze a discrete set of data points topologically, we connect them using **simplices** (points, lines, triangles, tetrahedrons) to build a continuous geometric structure called a **Simplicial Complex**:

- **0-simplex**: A point.
- **1-simplex**: A line segment (connecting two points).
- **2-simplex**: A filled triangle (connecting three points).
- **3-simplex**: A solid tetrahedron (connecting four points).

To construct a simplicial complex from a dataset **X** in a metric space, we grow closed balls of radius $\epsilon/2$ around each point $x \in X$, denoted as $B(x, \epsilon/2)$. There are three primary ways to build these complexes:

A. The Čech Complex ($\check{C}_\epsilon(X)$)

A simplex σ belongs to the **Čech complex** if the balls of radius $\epsilon/2$ centered at its vertices have a **non-empty global intersection**:

$$\bigcap_{x_i \in \sigma} B(x_i, \epsilon/2) \neq \emptyset$$

- **The Nerve Theorem**: The Nerve Theorem states that if the intersection of any subcollection of balls is contractible (which is true for convex balls in Euclidean space), then the Čech complex is **homotopy equivalent** (shares the same topological shape/holes) to the union of the balls.
- **Limitation**: Finding whether a large set of balls has a common intersection is computationally very expensive for high-dimensional simplices.

B. The Vietoris–Rips Complex ($VR_\epsilon(X)$)

A simplex σ belongs to the **Vietoris–Rips complex** if the balls of radius $\epsilon/2$ centered at its vertices intersect **pairwise** (which simply means the distance between any two vertices is $\leq \epsilon$):

$$d(x_i, x_j) \leq \epsilon \quad \text{for all } x_i, x_j \in \sigma$$

- **Pros & Cons**: It is much easier to compute because we only need to check pairwise distances. However, it does not satisfy the Nerve Theorem and can introduce "ghost" topological features that do not exist in the union of the balls.
- **Interleaving Relationship**: The Čech and Vietoris-Rips complexes approximate each other through the following squeeze/interleaving theorem:

$$\check{C}_{\epsilon/2}(X) \subseteq VR_\epsilon(X) \subseteq \check{C}_\epsilon(X)$$

C. The Alpha Complex (α-complex)

The **Alpha complex** is a subcomplex of the Delaunay triangulation. It restricts the balls $B(x, \epsilon/2)$ to their respective **Voronoi cells** (the region of space closer to x than to any other point).

- By intersecting each ball with its Voronoi cell, we get a much smaller simplicial complex that is still homotopy equivalent to the union of the balls (satisfying the Nerve Theorem) but is computationally much faster to construct in low dimensions.

---

2. Betti Numbers

Betti numbers (β_k) are topological invariants that count the number of k-dimensional holes in a simplicial complex:

- β_0: The number of **connected components**.
- β_1: The number of **1D circular holes** (like a loop or a circle).
- β_2: The number of **2D voids** (like the empty space inside a hollow sphere or balloon).

---

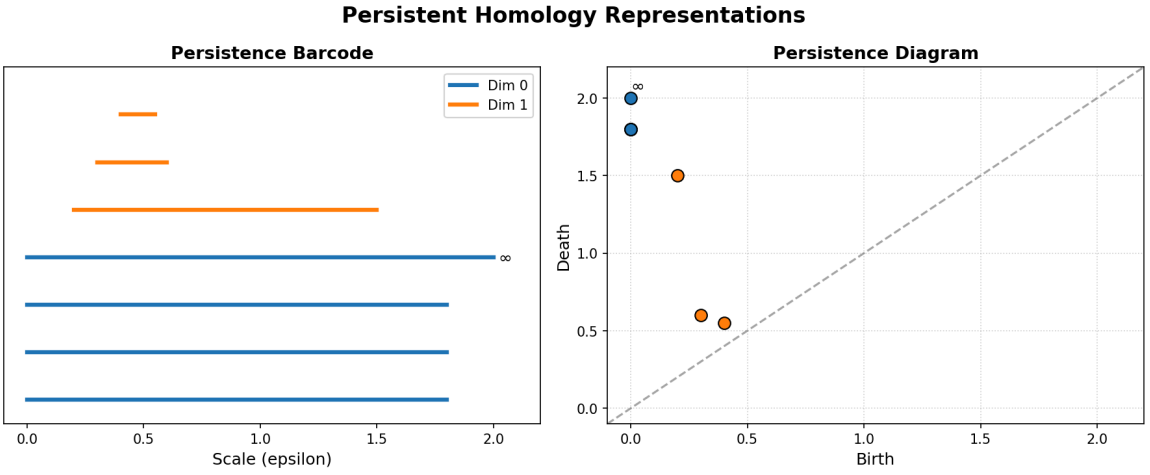
3. Persistent Homology

Instead of choosing a single radius ϵ, **Persistent Homology** computes the simplicial complexes for *all* values of ϵ from 0 to ∞. As ϵ grows:

- New simplices are born (connecting components).
- New holes are born.
- Existing holes are filled in and "die."

We track the **birth** and **death** of topological features across different scales of ϵ using:

- **Persistence Barcodes**: Horizontal lines showing the lifespan of each hole (x-axis is ϵ).
- **Persistence Diagrams**: 2D scatter plots where the x-axis is the birth scale and the y-axis is the death scale.



Long-lived features (bars that persist over a wide range of ϵ) represent true topological structures of the data, while short-lived features represent noise.

A. Algebraic Foundations: Persistence Modules

In mathematical TDA, a filtration of simplicial complexes yields a sequence of vector spaces connected by linear maps. This algebraic object is called a **Persistence Module**:

A persistence module V over $\mathbb{R}$ (or $\mathbb{Z}$) is a collection of vector spaces $\{V_t\}_{t\in\mathbb{R}}$ together with linear maps $v_{s,t} : V_s \to V_t$ for all $s \leq t$, satisfying:

1. **Identity**: $v_{t,t} = \mathrm{id}_{V_t}$ for all t.
2. **Composition**: $v_{t,r} \circ v_{s,t} = v_{s,r}$ for all $s \leq t \leq r$.

- **The Structure Theorem**: If a persistence module is *pointwise finite-dimensional* (each V_t is finite-dimensional), it decomposes uniquely into a direct sum of **interval modules** $I[b, d]$:

$$V \cong \bigoplus_{j\in J} I[b_j, d_j]$$

where an interval module $I[b, d]$ represents a topological feature that is born at scale b and dies at scale d. This algebraic decomposition is what justifies representing persistent homology as a **barcode** or **persistence diagram**.

B. The Persistence Algorithm & Matrix Reduction

To compute persistent homology, we represent the boundary operators across all simplices in a single **boundary matrix** D.

Let m be the total number of simplices in the filtration, sorted such that if simplex σ_i is a face of σ_j, then $i < j$. The boundary matrix D is an $m \times m$ matrix where:

- $D[i, j] = 1$ if simplex σ_i is a codimensional-1 face of simplex σ_j (using $\mathbb{Z}_2$ coefficients).
- $D[i, j] = 0$ otherwise.

Column Reduction Algorithm

We reduce D to a column-echelon form R using column operations. For any column j, let $\mathrm{low}(j)$ denote the index of the lowest row containing a 1. The algorithm is:

```
for j = 1 to m:
    while there exists column k < j such that low(k) == low(j):
        add column k to column j (modulo 2)
```

- **Pivots & Birth-Death Pairs**: Once reduced, if $\mathrm{low}(j) = i$, it means the simplex σ_i gave birth to a topological hole, and simplex σ_j killed it. This gives a birth-death interval $[t_i, t_j]$ in the barcode. If column j is reduced to zero and is never a pivot, it represents a feature that never dies (its interval is $[t_j, \infty)$).
- **The Clearing Optimization**: To speed up computation, if a column i is identified as a lowest row index ($\mathrm{low}(j) = i$), we immediately clear column i to zero ($R[:, i] = 0$). This is because any simplex that acts as a killer (boundary generator) cannot also give birth to a cycle in the same dimension, saving significant computation time.

C. Stability & The Isometry Theorem

To guarantee that TDA is a reliable tool, we must be able to compare persistence modules and prove that they are robust to noise.

- **Interleaving Distance (d_I)**: Two persistence diagrams are compared by finding an algebraic alignment between their underlying persistence modules. We say two modules V and W are δ-interleaved if there exist families of linear maps $f_t : V_t \to W_{t+\delta}$ and $g_t : W_t \to V_{t+\delta}$ that commute with the transition maps. The **interleaving distance** $d_I(V, W)$ is the infimum of all such δ for which an interleaving exists.
- **The Isometry Theorem**: The Isometry Theorem (proven by Bjerkevik and others) states that the algebraic interleaving distance between two persistence modules is exactly equal to the geometric bottleneck distance between their persistence diagrams:

$$d_I(V, W) = d_B(D(V), D(W))$$

- **The Stability Inequality**: This isometry underpins the stability of persistent homology. For any two continuous functions f, g on a space X, the bottleneck distance between their respective persistence diagrams is bounded by their supremum distance:

$$d_B(D(f), D(g)) \leq \|f - g\|_\infty$$

D. Persistent Cohomology

Cohomology is the algebraic dual of homology. Instead of study chains (sums of simplices), cohomology studies **cochains** (functions mapping simplices to coefficients, $C^k(K) = \mathrm{Hom}(C_k(K), \mathbb{Z}_2)$), using the coboundary operator $d_k : C^k(K) \to C^{k+1}(K)$.

- **Computational Advantage**: While persistent homology and persistent cohomology yield the exact same barcodes (due to dualities on field coefficients), the **persistent cohomology algorithm** is mathematically dual and runs significantly faster.
- **Why it is faster**: In cohomology, we build the filtration from the top down. The boundary matrix column reduction can be performed on the fly with much smaller active matrix sizes. This is why state-of-the-art TDA software (such as Ripser and GUDHI) defaults to computing persistent cohomology.

---

4. Python Example: Calculating Betti Numbers with `GUDHI`

You can perform Topological Data Analysis in Python using the `gudhi` library:

```
# To run this example, install gudhi: pip install gudhi
import numpy as np
import gudhi as gd

# Generate points on a circle with some noise
theta = np.linspace(0, 2 * np.pi, 100)
x = np.cos(theta) + np.random.normal(0, 0.05, 100)
y = np.sin(theta) + np.random.normal(0, 0.05, 100)
points = np.vstack((x, y)).T

# Compute Vietoris-Rips complex
rip_complex = gd.RipsComplex(points=points, max_edge_length=2.0)
simplex_tree = rip_complex.create_simplex_tree(max_dimension=2)

# Compute persistence
persistence = simplex_tree.persistence()

# Print topological features: dimension, (birth, death)
for feature in persistence:
    dim, (birth, death) = feature
    # We look for features that persist (have a large lifespan)
    if (death - birth) > 0.5:
        print(f"Dimension {dim} hole born at {birth:.3f}, dies at {death:.3f} (Lifespan: {death -
birth:.3f})")
```

Expected Output:

```
Dimension 0 hole born at 0.000, dies at inf (Lifespan: inf)
Dimension 1 hole born at 0.120, dies at 1.450 (Lifespan: 1.330)
```

- The infinite dimension 0 hole shows that all points eventually merge into a single connected component.
- The dimension 1 hole (large lifespan) indicates the loop structure of the circle we generated.

Advanced Topological Algorithms

This section introduces advanced computational topology algorithms used in visualization, dynamic systems, and multi-scale data analysis.

---

1. Reeb Graphs & The Mapper Algorithm

Reeb Graphs

A **Reeb graph** is a topological structure that captures the shape of a mathematical space by tracking how its connected components merge and split along a continuous function.

Formally, given a topological space X and a continuous function $f : X \to \mathbb{R}$, the Reeb graph is the quotient space of X under the equivalence relation:

$$x \sim y \iff f(x) = f(y) \text{ and } x, y \text{ belong to the same connected component of } f^{-1}(f(x))$$

The Mapper Algorithm

The **Mapper algorithm** is a discretization of the Reeb graph concept. It is one of the most popular tools in Topological Data Analysis for visualizing the shape of high-dimensional datasets.

The Mapper pipeline operates as follows:

```
[1. Input Data Point Cloud (X)]
                |
                ▼
[2. Apply Filter Function f(X)]
                |
                ▼
[3. Divide Range into Overlapping Bins]
                |
                ▼
[4. Cluster Points inside each Bin]
                |
                ▼
[5. Draw Edges between Overlapping Clusters]
                |
                ▼
[6. Output Topological Network Graph]
```

1. **Filter Function (Lens)**: We map the high-dimensional data points X to a lower-dimensional space (typically 1D or 2D) using a filter function $f : X \to \mathbb{R}$ (such as density estimates, PCA components, or geodesic distance).
2. **Overlapping Bins**: We divide the range of $f(X)$ into a set of overlapping intervals (bins).
3. **Clustering**: For each bin, we look at the original data points that fell into it and apply a clustering algorithm (such as DBSCAN or hierarchical clustering) to find connected components.
4. **Graph Construction**:
    - Each cluster acts as a **vertex** (node) in our output graph.
    - We draw an **edge** between two nodes if their respective clusters share one or more common data points (which occurs because the bins overlap).

- **Data Science Application**: Mapper represents complex, high-dimensional datasets as a simple, interactive network of nodes and edges, revealing branches, loops, and clusters that traditional dimensionality reduction methods might hide.

---

2. Zigzag Persistent Homology

Standard persistent homology assumes a nested, growing sequence of simplicial complexes (a filtration):

$$X_1 \subseteq X_2 \subseteq X_3 \subseteq \cdots \subseteq X_n$$

In this monotonic setup, all maps go in one direction. **Zigzag Persistent Homology** generalizes this by allowing the simplicial complexes to grow *and* shrink. The sequence of maps can point in either direction:

$$X_1 \longrightarrow X_2 \longleftarrow X_3 \longrightarrow X_4 \longleftarrow \dots$$

- **Why it is useful**: In dynamic systems, we often study time-varying datasets (such as moving particle clouds or video frames). Points are continuously added and deleted. Zigzag persistence allows us to track topological features (like loops or voids) as they appear, deform, and disappear over time, without requiring the spaces to be nested.

---

3. Multiparameter Persistent Homology

Standard TDA grows spheres of a single radius ϵ. However, real-world data is often noisy and contaminated with outliers. A single filtration parameter cannot separate true structure from noise.

Multiparameter (or multidimensional) persistence indexes simplicial complexes by two or more parameters simultaneously:

- **Parameter 1**: Sphere radius ϵ (representing scale).
- **Parameter 2**: Density threshold ρ (representing density).

By filtering by both scale and density, we can filter out low-density outliers while analyzing the shape of the dense core of the dataset.

The Algebraic Challenge

Unlike 1D persistent homology, the algebraic structure of multiparameter modules is extremely complex:

> ⚠ **Warning**
>
> **No Unique Interval Decomposition**: In 1D, the Structure Theorem guarantees that every persistence module decomposes uniquely into interval modules (barcodes). In 2D or higher, no such unique decomposition exists. This means we cannot represent multidimensional persistence as a single, simple barcode.

Computational Solutions

To analyze multiparameter persistence, researchers use:

- **Fibered Barcodes**: We project the 2D parameter space onto various 1D lines and compute the standard 1D barcodes along those lines.
- **Rank Invariants**: We compute the Betti numbers between pairs of coordinates in the multi-parameter grid.
- **RIVET (Rank Invariant Visualization and Exploration Tool)**: A software package specifically designed to calculate and visualize these rank invariants for 2D persistence modules.

Topological Surfaces & Singularities

This section covers special non-orientable topological surfaces, polar singularities, and vector field singularities on spheres.

---

1. Special Topological Surfaces

The Möbius Strip

The **Möbius strip** is a two-dimensional surface with only **one side** and **one boundary curve**.

You can easily construct a physical model of a Möbius strip by taking a rectangular strip of paper, giving one end a half-twist (180° turn), and then gluing the two ends together.

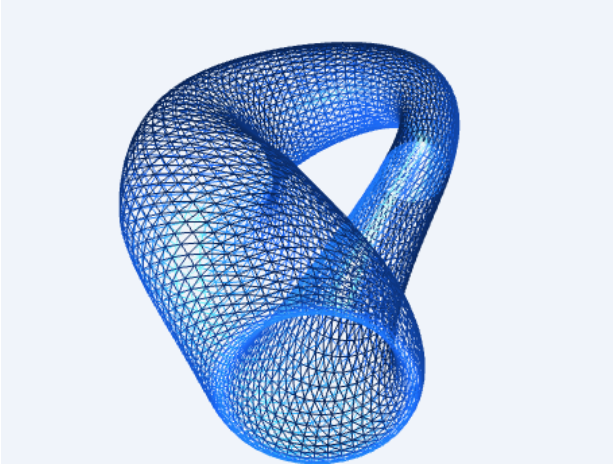
- **One-sided Property**: If you start drawing a line down the middle of a Möbius strip with a pen, you will eventually cover both “sides” of the paper and return to your starting point without ever lifting your pen

or crossing the boundary edge.

- **Topological Significance**: It is the simplest non-orientable surface. In data science, the Möbius strip is used to model cyclical datasets that have a twist or reflection in their state space.

The Klein Bottle

The **Klein bottle** is a closed 2D manifold (surface) that takes the concept of the Möbius strip one step further. It has **no boundaries** (like a sphere) but possesses only **one side** (like a Möbius strip).



- **Topological Connection to the Möbius Strip**: A Klein bottle can be mathematically constructed by **gluing two Möbius strips together along their circular boundaries**. Since a Möbius strip only has a single boundary edge (which is topologically a circle S^1), joining two Möbius strips along this single edge closes the surface, yielding a Klein bottle.
- **Dimensionality Embedding & Figure-8 Immersion**: A Klein bottle cannot be embedded in 3D space without passing through itself. However, it can be embedded perfectly in **4D space** ($d = 4$) without self-intersection.

  The illustration above shows the **Figure-8 immersion** (often called the Klein Bagel) in 3D:

  - It is represented as a clean blue wireframe mesh on a light background.
  - In 4D space, the self-intersection is resolved because the intersecting parts have different values in the fourth dimension (w-axis).
- **Data Science Application**: Non-orientable manifolds like the Klein bottle are used as challenging benchmark datasets in manifold learning. They test whether non-linear dimensionality reduction algorithms (like UMAP or Isomap) can map high-dimensional topological structures into lower dimensions without losing essential topological characteristics.

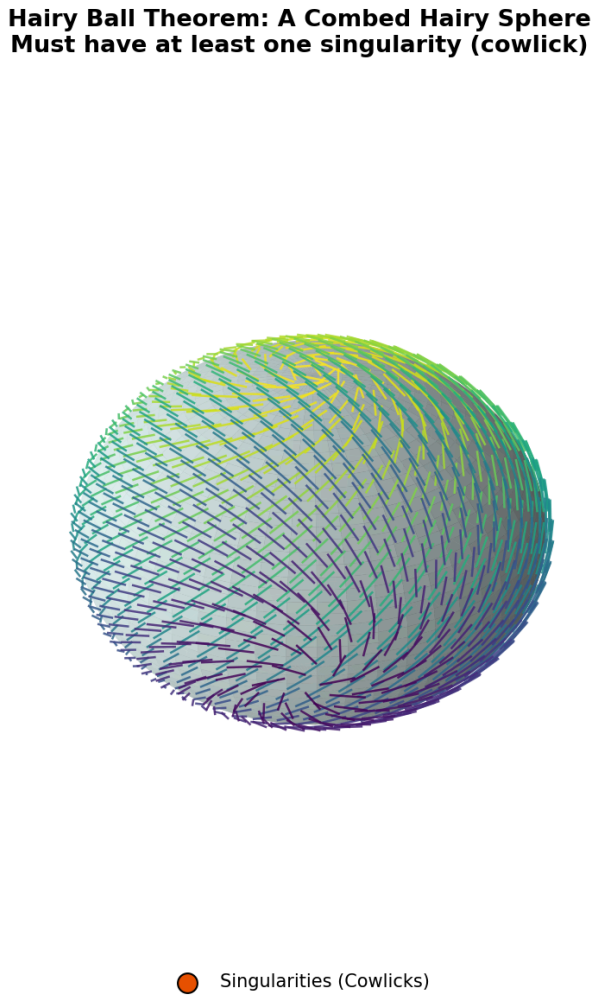
---

2. Singularities on a Sphere

A **singularity** is a point where a mathematical object loses its normal properties, collapses in dimension, or becomes mathematically undefined. Singularities on a sphere are studied in two major topological contexts:

A. Topological Vector Field Singularities: The Hairy Ball & Poincaré-Hopf Theorems

> **Important**
>
> **The Hairy Ball Theorem** from algebraic topology states that there is no continuous, non-vanishing tangent vector field on even-dimensional spheres (such as a 2D sphere S^2).
>
> *Intuition:* **"You cannot comb a hairy ball flat without creating at least one cowlick (singularity/whorl)."**



This theorem is a direct consequence of a much deeper topological principle: the **Poincaré-Hopf Theorem**.

Understanding the Poincaré-Hopf Theorem

The Poincaré-Hopf Theorem links the local behavior of a vector field (such as wind) to the global topology of the surface it lies on. It states that for any smooth vector field on a compact manifold M with isolated zeroes (singularities), the sum of the **indices** of these zeroes equals the **Euler characteristic** ($\chi(M)$) of the manifold:

$$\sum_i \text{index}_{z_i}(v) = \chi(M)$$

What is the “Index” of a Singularity?

The **index** measures how the vectors rotate as you walk counter-clockwise in a small loop around the singularity:

1. **Source / Outflow** (Wind blowing outward in all directions): **Index = +1**
2. **Sink / Inflow** (Wind blowing inward in all directions): **Index = +1**
3. **Vortex / Whorl** (Wind spiraling or rotating around a center): **Index = +1**
4. **Saddle Point / Col** (Wind coming in from two directions and leaving in two other directions): **Index = -1**

Meteorological Application: Wind Systems on Earth

The Earth’s surface is topologically a 2-sphere (S^2), which has an Euler characteristic of $\chi(S^2) = 2$. Applying the Poincaré-Hopf Theorem to the Earth’s wind vector field:

$$\sum \text{index} = 2$$

This leads to several fascinating meteorological rules:

- **At least one calm spot**: It is mathematically impossible to have a continuous wind blowing everywhere on Earth without any calm spots (points of zero wind speed). There must always be at least one cyclone, anticyclone, or saddle point.
- **The Balance of Weather Systems**:
  - If the Earth only has standard swirling systems (cyclones and anticyclones, which have an index of **+1**), there must be exactly **two** of them on the entire globe (e.g., 1 + 1 = 2, representing 1 Cyclone + 1 Anticyclone).
  - If a weather map features multiple storm centers (cyclones) and high-pressure centers (anticyclones), they must be balanced by **saddle points (cols)** (index of **-1**). For instance, if there are 3 cyclones/anticyclones (total index of **+3**), there must be exactly 1 saddle point (index of **-1**) to keep the global sum equal to 2: (+1) + (+1) + (+1) + (-1) = 2
  - Meteorologists use this topological constraint to validate global wind simulations and ensure that pressure systems and calm spots mapped by forecast models are topologically consistent.

B. Coordinate Singularities (Poles)

When mapping a 2D sphere using spherical coordinates (latitude ϕ and longitude θ), the North Pole and South Pole become **coordinate singularities**:

- Longitude θ is completely undefined at the poles (all longitude lines converge to a single point).

- The Jacobian matrix of coordinate transformations collapses (loses rank) at the poles.
- **Data Science & ML Relevance**: In global climate forecasting, geographical information systems (GIS), and deep learning on spherical grids (e.g., Spherical CNNs), polar singularities cause mathematical distortions and numerical instability during grid-based operations. To resolve this, researchers use grid-free spherical representations (such as Healpix grids) or project spherical coordinates into Cartesian 3D space (x, y, z) to eliminate the singularities.

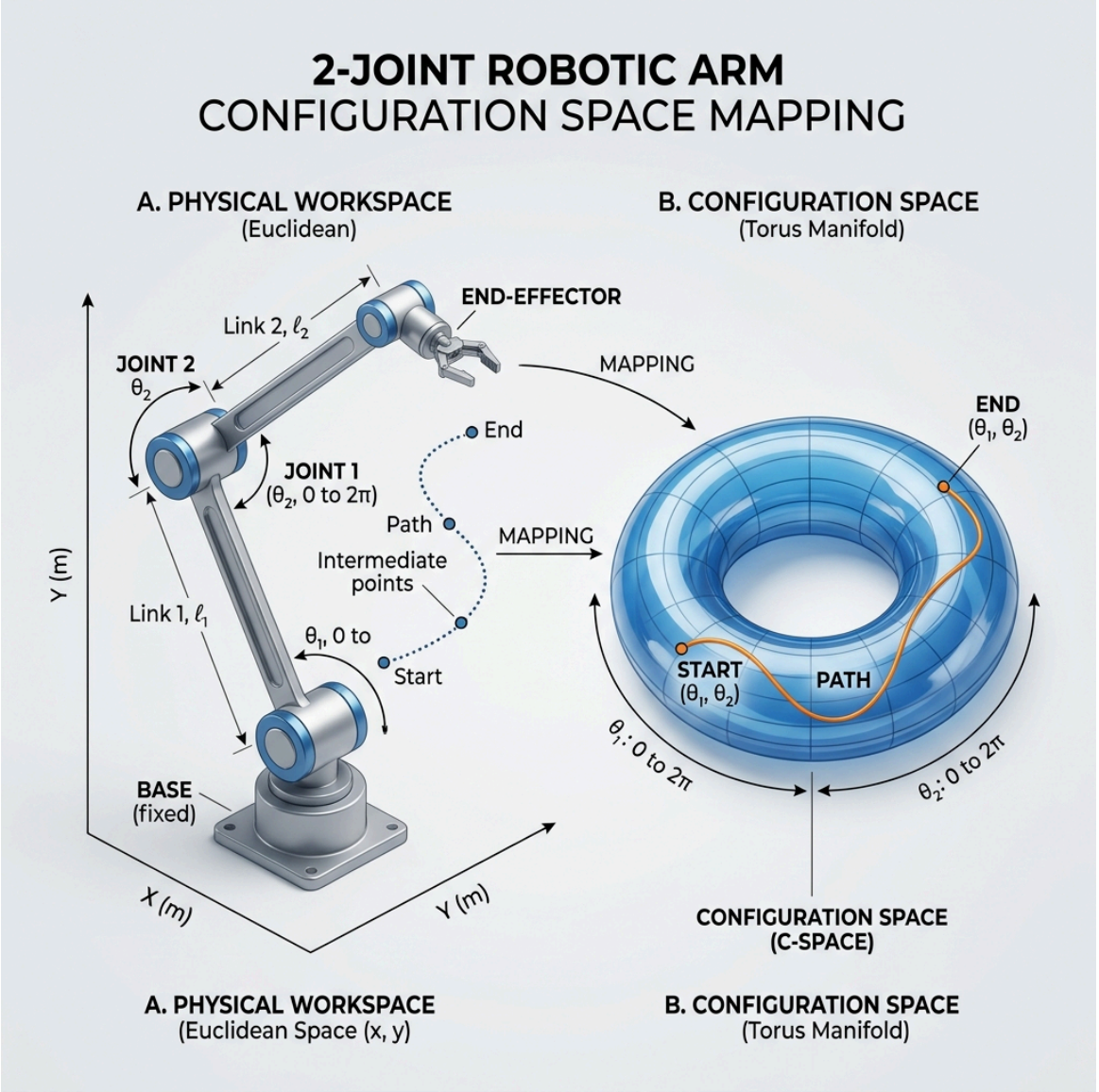
Interdisciplinary Applications of Topology

Topology is not just an abstract branch of pure mathematics; it has direct, high-impact applications across a wide variety of scientific and engineering fields.

---

1. Robotics & Motion Planning (Configuration Spaces)

In robotics, the set of all possible positions and orientations a robot can take is represented as a high-dimensional manifold called the **Configuration Space (C-space)**:



- **The Torus Mapping**: As shown above, for a simple 2-joint robotic arm, each joint angle (θ_1, θ_2) ranges from 0 to 2π (which topologically represents a circle S^1). The total configuration space is the product of these two circles ($S^1 \times S^1$), which is topologically a **Torus** (donut manifold).
- **Path Planning**: Finding a collision-free motion for the robot joint movements corresponds to finding a smooth, continuous path on this torus manifold from a start coordinate to an end coordinate while avoiding obstacles (which are represented as “holes” or cutouts on the manifold).

---

2. Wireless Sensor Networks & Relative Homology

In wireless network engineering and spatial monitoring, a critical problem is **Coverage Verification**: determining if a collection of sensors with overlapping coverage discs completely covers a given domain without leaving any “blind spots” (holes).

Using coordinates or geometric maps to verify coverage is highly susceptible to measurement noise and GPS errors. Topology provides a coordinate-free, coordinate-robust solution using **Relative Homology**.

- **Relative Homology ($H_k(X, A)$)**: Given a topological space X and a subspace $A \subseteq X$, relative homology group $H_k(X, A)$ counts the number of k-dimensional holes in X that are *not* contained within A. In essence, it treats the entire subspace A as a single, connected component, modding out any cycles that lie entirely within A.
- **The de Silva–Ghrist Theorem**: Let $D \subseteq \mathbb{R}^2$ be a bounded target domain monitored by sensors. We represent the sensors as a set of nodes X and construct a Vietoris-Rips complex $R(X)$ based on their communication range. Let R_∂ be the subcomplex representing sensors near the boundary of the domain. De Silva and Ghrist proved that if the relative homology mapping is non-trivial:

  $$H_2(R(X), R_\partial) \neq 0$$

  then it mathematically guarantees that the union of the sensors' coverage regions completely covers the domain D, leaving **no blind spots**. This verification requires no coordinate coordinates, only the network's connectivity data!

---

3. Physics & Materials Science

Topology has revolutionized modern physics, leading to multiple Nobel prizes:

- **Topological Insulators & Condensed Matter**: Topological materials possess electronic states that are protected by topology. For example, in 2D topological insulators, electrical current flows only along the edges of the material and is completely protected from backscattering (impurities in the crystal). David Thouless, Duncan Haldane, and Michael Kosterlitz were awarded the **2016 Nobel Prize in Physics** for discovering these topological phases of matter.
- **String Theory & Calabi-Yau Manifolds**: In string theory, the universe is modeled with 10 dimensions, where the 6 extra dimensions are curled up into tiny, complex 6D manifolds called **Calabi-Yau manifolds**. The topological classification of these manifolds dictates the physical properties and families of particles that can exist in the universe.
- **Cosmology**: Physical cosmologists use topology to describe the global, overall shape of the universe (spacetime topology), trying to determine if our universe is flat, spherical, or toroidal.

---

4. Molecular Biology & Nanotechnology

Biological systems utilize topological constraints to manage complex, folded structures:

- **DNA Knotting**: Inside a cell, DNA is a long, highly coiled molecule. Enzymes (like topoisomerases) manage DNA replication by cutting, twisting, and reconnecting DNA strands—essentially performing topological transformations to resolve knots and tangles. Biologists use **Knot Theory** to classify these knots and study how they affect DNA electrophoresis speeds.
- **Protein Folding**: The complex 3D structures of folded proteins are classified using **Circuit Topology** and **Knot Theory**, comparing pairwise arrangements of internal contacts and chain crossings to understand

protein stability and functions.

---

5. Computer Science & Program Semantics

Beyond data analysis (TDA), topology is used to formalize programming logic:

- **Domain Theory**: Programming language semantics are formalized using topological spaces. Computer scientists like Steve Vickers characterize topological open sets as **semidecidable (finitely observable) properties** of computer programs—representing properties that a program can verify in finite time.

---

6. Fiber Arts & Puzzles

- **Modular Fiber Arts**: In crochet or modular knitting, creating a continuous, seamless join of multiple pieces without breaking the yarn is an application of finding an **Eulerian Path** (traversing each edge of the modular mesh exactly once).
- **Disentanglement Puzzles**: Classic metal and string puzzles rely on topological features (like genus, loops, and boundary crossings) to be solved, where the solution is found by finding a topological deformation that allows two linked components to slide past each other.

Approximation Theory

In computational mathematics and data science, many functions (like e^x, $\ln(x)$, $\sin(x)$, or complex neural network mappings) cannot be calculated exactly. Instead, computers calculate highly accurate **approximations**.

Here we cover the core methods of function approximation used in numerical computing, calculators, and machine learning.

---

1. Taylor Series & Maclaurin Series

A **Taylor Series** represents a smooth function as an infinite sum of terms calculated from the values of the function's derivatives at a single point a.

For a function $f(x)$, the Taylor expansion is:

$$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \frac{f'''(a)}{3!}(x-a)^3 + \cdots = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!}(x-a)^n$$

When we center the approximation at $a = 0$, it is called a **Maclaurin Series**:

$$f(x) = f(0) + f'(0)x + \frac{f''(0)}{2!}x^2 + \frac{f'''(0)}{3!}x^3 + \dots$$

Common Maclaurin Series:

- **Exponential**: $e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$
- **Sine**: $\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots$
- **Cosine**: $\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \dots$

Python Implementation (Taylor Approximation)

Let's approximate e^x using SymPy:

```
import sympy as sp

x = sp.symbols('x')
f = sp.exp(x)

# Taylor expansion centered at 0 (Maclaurin) of degree 4
taylor_poly = f.series(x, 0, 5).removeO()
print("Taylor polynomial of e^x (degree 4):")
sp.pprint(taylor_poly)
```

---

2. Chebyshev Polynomials (Minimax Approximation)

Taylor series are extremely accurate *near the expansion point* a, but the approximation error grows rapidly as you move away from a.

To achieve a uniform error over an entire interval $[a, b]$, we use **Chebyshev Polynomials**. They minimize the maximum error (the "minimax" property) over the interval $[-1, 1]$.

The Chebyshev polynomials of the first kind are defined by the recurrence relation:

$$T_0(x) = 1$$

$$T_1(x) = x$$

$$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x)$$

For example, $T_2(x) = 2x^2 - 1$, $T_3(x) = 4x^3 - 3x$, etc.

These polynomials are the foundation of numerical libraries in calculators and coding environments (like NumPy's `numpy.polynomial.chebyshev`) to calculate trigonometric and exponential functions efficiently.

---

3. Exponential & Logarithmic Approximations

In machine learning and statistics, we often approximate exponentiation and logarithms to speed up calculations or prevent numerical overflow/underflow (e.g., in Softmax or Log-Loss).

For instance, when x is very small, we can approximate $\ln(1+x)$ using the first term of its Taylor series:

$$\ln(1+x) \approx x$$

Similarly:

$$e^x \approx 1 + x \quad (\text{for } x \approx 0)$$

Python Example (Log Approximation)

```
import numpy as np

x = 0.05
approx = x
actual = np.log(1 + x)

print(f"Approximation of ln(1 + {x}) : {approx}")
print(f"Actual value                 : {actual:.6f}")
print(f"Absolute Error               : {abs(approx - actual):.6f}")
```

---

4. Remez's Algorithm

Remez's Algorithm is an iterative minimax approximation algorithm. It finds the polynomial of a given degree n that minimizes the maximum absolute error for a continuous function $f(x)$ on an interval $[a, b]$.

Unlike Taylor series which focus on a single point, Remez's algorithm produces an approximation where the error oscillates between positive and negative bounds of equal magnitude (equioscillation theorem).

---

5. The Universal Approximation Theorem (UAT)

In deep learning, neural networks are fundamentally **universal function approximators**.

The **Universal Approximation Theorem** states that a standard feedforward neural network with:

1. A single hidden layer
2. A finite number of neurons
3. A non-linear activation function (such as ReLU, Sigmoid, or Tanh)

can approximate any continuous function on a compact subset of $\mathbb{R}^n$ to any desired degree of accuracy.

This theorem provides the mathematical justification for why deep learning models are capable of learning extremely complex mappings (images to labels, text to text) from data.

Exercises

Exercise 1

Find the Taylor polynomial of degree 3 for the function $f(x) = \ln(1+x)$ centered at $x = 0$.

Exercise 2

Using the Chebyshev recurrence relation $T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x)$, derive the formula for $T_4(x)$ given:

- $T_2(x) = 2x^2 - 1$
- $T_3(x) = 4x^3 - 3x$

Solution — Exercise 1

Compute the derivatives of $f(x) = \ln(1+x)$ at $x = 0$:

- $f(0) = \ln(1) = 0$
- $f'(x) = \frac{1}{1+x} \implies f'(0) = 1$
- $f''(x) = -\frac{1}{(1+x)^2} \implies f''(0) = -1$
- $f'''(x) = \frac{2}{(1+x)^3} \implies f'''(0) = 2$

Substitute into the Taylor formula:

$$f(x) \approx 0 + 1(x) + \frac{-1}{2!}x^2 + \frac{2}{3!}x^3 = x - \frac{x^2}{2} + \frac{x^3}{3}$$

Solution — Exercise 2

Using the recurrence relation for $n = 3$:

$$T_4(x) = 2xT_3(x) - T_2(x)$$

$$T_4(x) = 2x(4x^3 - 3x) - (2x^2 - 1)$$

$$T_4(x) = 8x^4 - 6x^2 - 2x^2 + 1$$

$$T_4(x) = 8x^4 - 8x^2 + 1$$

Approximation and Fitting

In data science, we often approximate complex data points or functions using mathematical models. This page covers norm approximation, regularization, robust approximation, and function fitting based on Chapter 6 of Stephen Boyd's *Convex Optimization*.

---

1. Norm Approximation

The simplest norm approximation problem has the form:

$$\text{minimize} \quad \|Ax - b\|$$

Where $A \in \mathbb{R}^{m \times n}$ ($m > n$) and $\|\cdot\|$ is any norm (such as L_1, L_2, or L_∞).

- L_2 **Norm (Least-squares)**: Minimizes the sum of squared residuals ($\sum r_i^2$). It is highly sensitive to outliers.
- L_1 **Norm (Robust approximation)**: Minimizes the sum of absolute residuals ($\sum |r_i|$). It is more robust to outliers and leads to sparse residuals.
- L_∞ **Norm (Chebyshev approximation)**: Minimizes the maximum absolute residual ($\max |r_i|$).

---

2. Least-Norm Problems

When a system of linear equations $Ax = b$ is underdetermined ($m < n$, meaning there are more variables than equations), we want to find the solution that has the smallest norm:

$$\begin{aligned} \text{minimize} \quad & \|x\| \\ \text{subject to} \quad & Ax = b \end{aligned}$$

For the L_2 norm, the analytical solution is $x = A^T(AA^T)^{-1}b$.

---

3. Regularized Approximation

To balance the trade-off between fitting the data well (minimizing $\|Ax - b\|$) and keeping the model weights small (minimizing $\|x\|$), we use **regularization**:

$$\text{minimize} \quad \|Ax - b\| + \gamma\|x\|$$

Where $\gamma > 0$ is the regularization parameter.

- **Tikhonov Regularization (L_2 / Ridge)**: Minimizes $\|Ax - b\|_2^2 + \gamma\|x\|_2^2$. It prevents overfitting by shrinking coefficients.
- **Lasso Regularization (L_1)**: Minimizes $\|Ax - b\|_2^2 + \gamma\|x\|_1$. It performs feature selection by driving some coefficients to exactly zero.

---

4. Robust Approximation

When there is uncertainty or noise in the matrix A, we use robust approximation to optimize for the worst-case scenario:

$$\text{minimize} \quad \sup_{u \in \mathcal{U}} \|(A + \Delta(u))x - b\|$$

Where $\mathcal{U}$ represents the set of possible perturbations. This ensures the model remains stable even under input perturbations.

---

5. Function Fitting & Interpolation

We want to fit a continuous function $f(x)$ using a linear combination of basis functions $\phi_i(x)$:

$$\hat{f}(x) = \sum_{i=1}^{k} c_i \phi_i(x)$$

Common Basis Functions:

- **Polynomial Fitting**: $\phi_i(x) = x^i$
- **Spline Interpolation**: Piecewise polynomials connected smoothly at knots.

Python Example: Polynomial Fitting (L2 vs. L1 Robust Fitting)

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

# Generate data with an outlier
np.random.seed(42)
x = np.linspace(-3, 3, 20)
y = 2 * x + 1 + np.random.normal(0, 0.5, 20)
y[15] += 10  # Introduce a massive outlier

# L2 Loss (Least-squares)
def l2_loss(coeffs):
    return np.sum((y - (coeffs[0] * x + coeffs[1]))**2)

# L1 Loss (Robust absolute loss)
def l1_loss(coeffs):
    return np.sum(np.abs(y - (coeffs[0] * x + coeffs[1])))

c_l2 = minimize(l2_loss, [0.0, 0.0]).x
c_l1 = minimize(l1_loss, [0.0, 0.0]).x

print("L2 Coefficients (Slope, Intercept):", c_l2)
print("L1 Coefficients (Slope, Intercept):", c_l1)
```

Exercises

Exercise 1

Explain why Lasso (L_1 regularization) produces sparse solutions (weights set to 0) while Ridge (L_2 regularization) does not.

Solution — Exercise 1

The constraint region for L_1 regularization is a diamond shape (with sharp corners on the axes), whereas the constraint region for L_2 regularization is a circle. When optimizing, the contours of the loss function are highly likely to hit the sharp corners of the L_1 diamond first (which lie exactly on the axes, meaning some variables are set to 0). For the L_2 circle, the contact point can be anywhere, resulting in small but non-zero weights.

Machine learning

Preface

Documentation on all topics that I learn on both Artificial intelligence and machine learning. Topics Covered:

- Artificial Intelligence Concepts
- Search
- Decision Theory
- Reinforcement Learning
- Artificial Neural Networks
- Back-propagation
- Feature Extraction
- Deep Learning
- Convolutional Neural Networks
- Deep Reinforcement Learning
- Distributed Learning
- Python/Matlab deep learning library

Methodology for usage

This chapter will give a recipe on how to tackle Machine learning problems

3 Step process

1. Define what you need to do and how to measure(metrics) how well/bad you are going.
2. Start with a very simple model (Few layers)
3. Add more complexity when needed.

Define goals

Normally by being slightly better than human performance(in terms of accuracy or prediction time), is already enough for a good product.

Metrics

Accuracy: % of correct examples Coverage: Number of processed examples per unit of time Precision: Number of detections that are correct Error: Amount of error

Start with simplest model

Look if available for the state-of-the-art method for solving that problem. If not available use the following recipe. 1. Lots of noise and now much structure(ex: House price from features like number of rooms,kitchen size, etc...): Don't use deep learning 2. Few noise but lot's of structure (ex: Images, video, text): Use deep learning

Examples for Non-deep methods:

Logistic Regression SVM Boosted decision trees (Previous favorite "default" algorithm), used a lot in robotics

What kind of deep

Few structure: Use only Fully-Connected layers 2-3 layers (Relu, Dropout, SGD+Momentum). Need at least few thousand examples per class. Spatial structure: Use CONV layers (Inception/Residual, Relu, Droput, Batch-Norm, SGD+Momentum). Sequencial structure (text, market movement): Use Recurrent networks (LSTM, SGD, Gradient clipping). When using Residual/Inception networks start with the shallowest example possible.

Solving High train errors

Do the following actions on this order: 1. Inspect for defects on the data. (Need human intervention) 2. Check for software bugs on your library. (Use gradient check, it's probably a backprop error) 3. Tune learning rate (Make it smaller) 4. Make network deeper. (You should start with a shallow network on the beginning)

Solving High test errors

Do the following actions on this order: 1. Do more data augmentation (Also try generative models to create more data) 2. Add dropout and batch-norm 3. Get more data (More data has more influence on Accuracy then anything else)

Some trends

Following the late 2016 Andrew Ng lecture these are the topics that we need to pay attention.

1. Scalability Have a computing system that scale well for more data and more model complexity.
2. Team Have on your team divided with with AI people and HPC (Cuda, OpenCl, etc...).
3. Data first Data is more important than your model, always try to get more quality data before trying to change your model.
4. Data Augmentation Use normal data augmentation techniques plus Generative models(Unsupervised).
5. Make sure that Validation Set and Test set come from same distribution This will avoid having a test or validation set that does not tell the reality. Also helps to check if your training is valid.
6. Have Human level performance metric Have a team of experts to compare with your current system performance. Also it drives decisions between getting more data, or making model more complex.
7. Data server Have a unified data-warehouse. All team must have access to data, with SSD quality access speed.
8. Using Games Using games are cool to help augment datasets, but attention because games does not have the same variants of the same class as real life. For example GTA does not have enough car models compared to real life.
9. Ensembles always help Training separately different networks and averaging their end results always gives some extra 2% accuracy. (Imagenet 2016 best results were simple ensambles)
10. What to do if you have more than 1000 classes Use hierarchical Softmax to increase performance.
11. How many samples per class do we need to have good result If training from scratch, use the same number of parameters. For example Model has 1000 parameters, so use 1000 samples per class. If doing transfer learning is much less (Much is not defined yet, more is better).

Supervised Learning

In this chapter we will learn about Supervised learning as well as talk a bit about Cost Functions and Gradient descent. Also we will learn about 2 simple algorithms: Linear Regression (for Regression) Logistic Regression (for Classification) The first thing to learn about supervised learning is that every sample data point x has an expected output or label y, in other words your training is composed of pairs.

Unsupervised Learning

As mentioned on previous chapters, unsupervised learning is about learning information without the label information. Here the term information means, “structure” for instance you would like to know how many groups exist in your dataset, even if you don’t know what those groups mean. Also we use unsupervised learning to visualize your dataset, in order to try to learn some insight from the data.

Unlabeled data example

Distributed Learning

Learn how the training of deep models can be distributed across multiple machines. Map Reduce Map Reduce can be described on the following steps:

1. Split your training set, in batches (ex: divide by the number of workers on your farm: 4)
2. Give each machine of your farm 1/4th of the data
3. Perform Forward/Backward propagation, on each computer node (All nodes share the same model)
4. Combine results of each machine and perform gradient descent
5. Update model version on all nodes.

Deep Learning Foundations

Deep Learning is a subset of Machine Learning that is based on Artificial Neural Networks (ANNs). It is designed to learn representations of data automatically by passing inputs through multiple layers of nodes (neurons), mimicking the human brain's neural connections.

---

1. The Perceptron: The Atom of Neural Networks

The basic building block of a neural network is the **neuron** (or Perceptron). It takes several inputs, multiplies them by their corresponding weights, adds a bias term, and passes the result through an **activation function** to produce an output.

Mathematically, a single neuron computes:

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b = \mathbf{w}^T \mathbf{x} + b$$

$$a = \sigma(z)$$

Where:

- $\mathbf{x}$ is the input vector.
- $\mathbf{w}$ is the weight vector.
- b is the bias.
- σ is the activation function.

---

2. Activation Functions

Without activation functions, a neural network is just a giant linear regression model (linear transformations stacked on linear transformations are still linear). Activation functions introduce **non-linearity**, allowing neural networks to learn complex, non-linear relationships.

Name	Formula	Use Case
Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$	Output layer of binary classification
ReLU (Rectified Linear Unit)	$f(z) = \max(0, z)$	Default for hidden layers
Tanh (Hyperbolic Tangent)	$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	Hidden layers (zero-centered outputs)

```python
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def relu(z):
    return np.maximum(0, z)

# Plotting the functions
z = np.linspace(-5, 5, 200)
plt.figure(figsize=(10, 4))
plt.plot(z, sigmoid(z), label='Sigmoid', color='blue')
plt.plot(z, relu(z), label='ReLU', color='red')
plt.axhline(0, color='black',linewidth=0.5, linestyle='--')
plt.axvline(0, color='black',linewidth=0.5, linestyle='--')
plt.title('Activation Functions')
plt.legend()
plt.grid()
plt.tight_layout()
plt.savefig('../../images/activations.png')
plt.show()
```

---

3. How a Neural Network Learns

Training a neural network is an iterative process consisting of three main phases:

Phase 1: Forward Propagation

Inputs are fed forward through the network layer by layer to compute a prediction ($\hat{y}$).

Phase 2: Loss (Cost) Computation

We evaluate the error of our prediction using a **Loss Function**. For regression, we typically use Mean Squared Error (MSE):

$$L = \frac{1}{2}(\hat{y} - y)^2$$

Phase 3: Backpropagation

Backpropagation calculates the gradient of the loss function with respect to each weight in the network. It uses the **Chain Rule** (from Calculus) to trace the gradient from the output layer back to the input layer.

For a weight w, we find:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Then we update the weight using **Gradient Descent**:

$$w_{new} = w_{old} - \alpha \frac{\partial L}{\partial w}$$

Where α is the **learning rate**.

---

4. Implementing a Basic Neural Network from Scratch in Python

Let's write a simple network with one input layer (2 features), one hidden layer (3 neurons), and one output layer (1 neuron) to solve a binary classification task.

```python
import numpy as np

# Activation functions
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

# Dataset (XOR problem)
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])

y = np.array([[0], [1], [1], [0]])

# Seed random numbers for reproducibility
np.random.seed(42)

# Weight initialization
input_size = 2
hidden_size = 3
output_size = 1

weights_input_hidden = np.random.uniform(-1, 1, (input_size, hidden_size))
weights_hidden_output = np.random.uniform(-1, 1, (hidden_size, output_size))

learning_rate = 0.5
epochs = 10000

# Training Loop
for epoch in range(epochs):
    # --- Forward Propagation ---
    hidden_input = np.dot(X, weights_input_hidden)
    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, weights_hidden_output)
    predictions = sigmoid(final_input)

    # --- Backward Propagation ---
    # 1. Compute loss derivative at output
    error = y - predictions
    d_predicted_output = error * sigmoid_derivative(predictions)

    # 2. Backpropagate error to hidden layer
    error_hidden_layer = d_predicted_output.dot(weights_hidden_output.T)
    d_hidden_layer = error_hidden_layer * sigmoid_derivative(hidden_output)

    # 3. Update weights
    weights_hidden_output += hidden_output.T.dot(d_predicted_output) * learning_rate
    weights_input_hidden += X.T.dot(d_hidden_layer) * learning_rate

# Final Predictions
print("Predictions after training:")
print(np.round(predictions, 2))
```

Output:

```
Predictions after training:
[[0.06]
 [0.93]
 [0.93]
 [0.08]]
```

The simple neural network successfully learned the non-linear XOR function!

Exercises

Exercise 1

Derive the derivative of the Sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$ by hand and show that:

$$\frac{d\sigma}{dz} = \sigma(z)(1 - \sigma(z))$$

Exercise 2

Modify the scratch Python neural network above to use a different learning rate (e.g., 0.05 and 2.0). Trace how the learning rate affects training convergence.

Exercise 3

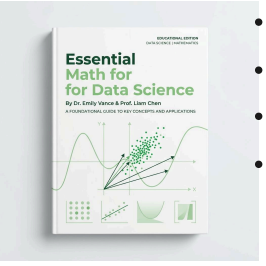
Install PyTorch and write a simple script using `torch.nn` to solve the same XOR classification problem.

Recommended Reading & Resources

To deepen your understanding of the mathematical foundations of Data Science, Machine Learning, and Topology, we highly recommend studying the following core textbooks and references. These resources form the pedagogical backbone of this blog.

Core Textbooks

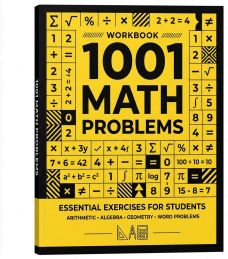
1. Essential Math for Data Science



- **Author**: Thomas Nield
- **Publisher**: O'Reilly Media, 2022 (ISBN: 9781098102920)
- **Book Page**: O'Reilly Book Link
- **Overview**: This book is an exceptional, hands-on guide designed to bridge the gap between mathematical theory and data science practice. It covers linear algebra, calculus, probability, and statistics using intuitive, coordinate-free explanations and practical Python code samples.

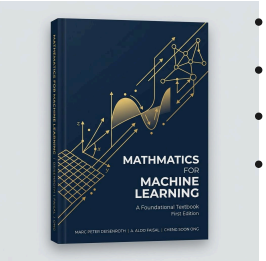
- **Why Read It?**: It is ideal for beginners and practitioners who want to understand *why* machine learning algorithms work behind the scenes, focusing on conceptual understanding before formula memorization.

2. 1001 Math Problems (2nd Edition)



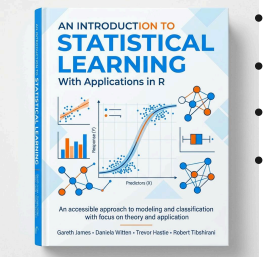
- **Publisher**: LearningExpress, LLC
- **Book Page**: LearningExpress Link
- **Overview**: A comprehensive, practical workbook containing foundational mathematics problems. It guides readers through step-by-step solutions for arithmetic, basic algebra, geometry, and numerical reasoning.
- **Why Read It?**: It is a perfect resource for building speed, confidence, and basic mathematical reasoning skills through structured practice.

3. Mathematics for Machine Learning



- **Authors**: Marc Peter Deisenroth, A. Aldo Faisal, and Cheng Soon Ong
- **Publisher**: Cambridge University Press, 2020 (ISBN: 9781108455145)
- **Official Website**: mml-book.com
- **Overview**: A highly popular textbook that provides a self-contained introduction to the mathematical tools necessary for machine learning, including linear algebra, analytic geometry, matrix decompositions, vector calculus, optimization, probability, and statistics.
- **Why Read It?**: It connects mathematical concepts directly to machine learning algorithms like linear regression, PCA, mixture models, and SVMs.

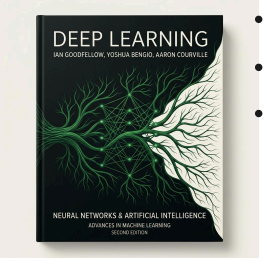
4. An Introduction to Statistical Learning (ISL)



- **Authors**: Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani
- **Publisher**: Springer, 2nd Edition, 2021 (ISBN: 9781071614175)
- **Official Website**: statlearning.com
- **Overview**: This classic textbook (available in both R and Python versions) provides an accessible overview of the field of statistical learning, covering regression, classification, resampling, regularization, tree-based methods, SVMs, and unsupervised learning.
- **Why Read It?**: It is the gold standard for understanding statistical machine learning concepts without getting bogged down in overly advanced math.

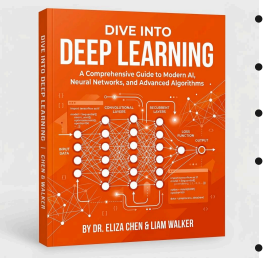
5. Deep Learning

- **Authors**: Ian Goodfellow, Yoshua Bengio, and Aaron Courville



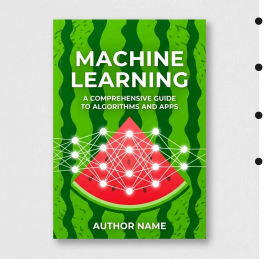
- **Publisher**: MIT Press, 2016 (ISBN: 9780262035613)
- **Official Website**: deeplearningbook.org
- **Overview**: Widely regarded as the definitive textbook on deep learning theory, this book covers the mathematical foundations (linear algebra, probability, numerical computation), classical machine learning concepts, and deep network architectures (feedforward networks, CNNs, RNNs, and optimization).
- **Why Read It?**: It provides a rigorous theoretical foundation for anyone wishing to understand the mathematics behind deep learning architectures.

6. Dive into Deep Learning (D2L)



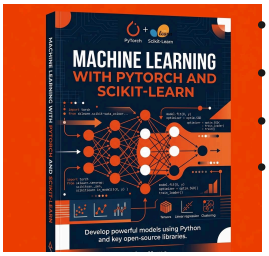
- **Authors**: Aston Zhang, Zachary C. Lipton, Mu Li, and Alexander J. Smola
- **Publisher**: Cambridge University Press, 2023 (ISBN: 9781009382656)
- **Official Website**: d2l.ai
- **Overview**: An interactive, code-first textbook that covers deep learning theory alongside fully functional code implementations in PyTorch, TensorFlow, and MXNet.
- **Why Read It?**: Perfect for hands-on learners who want to see mathematical concepts directly translated into executable Python code and Jupyter notebooks.

7. Machine Learning (commonly known as the “Watermelon Book”)



- **Author**: Zhou Zhihua
- **Publisher**: Tsinghua University Press, 2016 (ISBN: 9787302423287)
- **Official Page**: Tsinghua University Press Book Link
- **Overview**: Written by Nanjing University Professor Zhou Zhihua, this highly popular 425-page textbook provides a comprehensive introduction to machine learning. It won the first prize of the inaugural National Excellent Teaching Materials Award (Higher Education Category), has been adopted by over 500 academic institutions, and has seen dozens of print runs.
- **Why Read It?**: It is an incredibly well-structured, clear, and comprehensive text covering classical machine learning, ensemble methods, clustering, neural networks, and evaluation metrics.

8. Machine Learning with PyTorch and Scikit-Learn

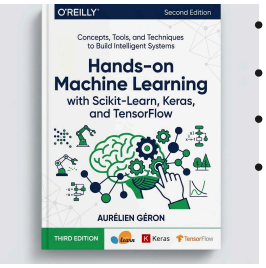


- **Authors**: Sebastian Raschka, Yuxi (Hayden) Liu, and Vahid Mirjalili
- **Publisher**: Packt Publishing, 2022 (ISBN: 9781801819312)
- **GitHub Repository**: Code and Notebooks
- **Overview**: A comprehensive guide to building machine learning and deep learning models using PyTorch and Scikit-Learn. It bridges theory with practical coding, covering

classification, regression, model evaluation, CNNs, RNNs, and transformers.

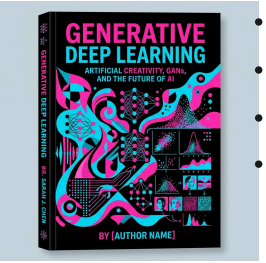
- **Why Read It?**: Ideal for engineers seeking to transition from traditional statistical learning to deep neural networks using PyTorch.

9. Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow (3rd Edition)



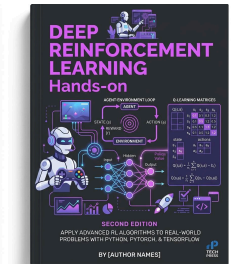
- **Author**: Aurélien Géron
- **Publisher**: O'Reilly Media, 2022 (ISBN: 978109825608)
- **GitHub Repository**: Code and Notebooks
- **Overview**: A practical, project-based guide to building machine learning systems. It starts with Scikit-Learn to cover classical models, then moves into Keras and TensorFlow to build, train, and deploy deep neural architectures.
- **Why Read It?**: Famously clear, accessible, and packed with practical tips on hyperparameter tuning, data preprocessing, and model deployment.

10. Generative Deep Learning (2nd Edition)



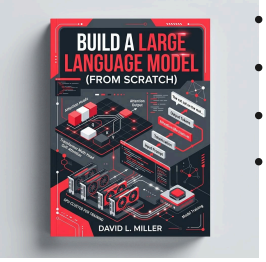
- **Author**: David Foster
- **Publisher**: O'Reilly Media, 2023 (ISBN: 9781098134181)
- **GitHub Repository**: Code and Notebooks
- **Overview**: A complete guide to modern generative models. It teaches you how to design and build Variational Autoencoders (VAEs), Generative Adversarial Networks (GANs), Transformers (GPT-3), Diffusion Models, and deep reinforcement learning agents.
- **Why Read It?**: Essential for understanding how AI systems create text, images, and music, with intuitive step-by-step PyTorch code.

11. Deep Reinforcement Learning Hands-on (2nd Edition)



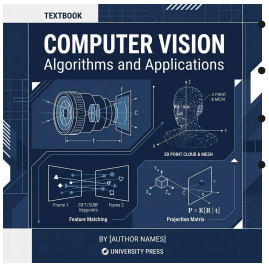
- **Author**: Maxim Lapan
- **Publisher**: Packt Publishing, 2020 (ISBN: 9781838985103)
- **GitHub Repository**: Code and Notebooks
- **Overview**: A comprehensive, practical guide to deep reinforcement learning (DRL) algorithms. It covers Q-learning, deep Q-networks (DQNs), policy gradients, actor-critic methods, and applying DRL to robotics, gaming agents, and trading.
- **Why Read It?**: The go-to reference for practical implementations of complex RL algorithms in PyTorch.

12. Build a Large Language Model (From Scratch)



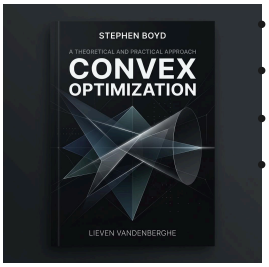
- **Author**: Sebastian Raschka
- **Publisher**: Manning Publications, 2024 (ISBN: 9781633437166)
- **GitHub Repository**: Code and Notebooks
- **Overview**: A brilliant, step-by-step guide to coding and training a Generative Pre-trained Transformer (GPT) style LLM from scratch in PyTorch. It explains tokenization, self-attention mechanisms, transformer architecture, pre-training, and instruction fine-tuning.
- **Why Read It?**: It demystifies the black box of LLMs, showing that you can write the entire architecture in standard PyTorch code.

13. Computer Vision: Algorithms and Applications (2nd Edition)



- **Author**: Richard Szeliski
- **Publisher**: Springer, 2022 (ISBN: 9783030349363)
- **Official Website**: szeliski.org/Book
- **Overview**: The definitive reference for computer vision algorithms. It covers image formation, processing, feature detection, camera geometry, 3D reconstruction, and modern deep learning-based vision systems.
- **Why Read It?**: An essential read for understanding spatial geometry, coordinate transforms, and modern visual perception algorithms in robotics and image processing.

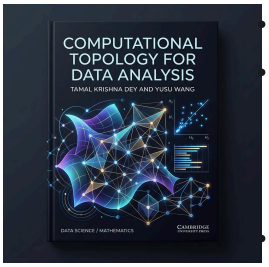
14. Convex Optimization



- **Author**: Stephen Boyd and Lieven Vandenberghe
- **Publisher**: Cambridge University Press, 2004 (ISBN: 9780521833783)
- **Official Website**: EE364a Stanford Course Page
- **Overview**: The absolute gold standard textbook for convex optimization theory, algorithms, and applications. It provides a detailed, rigorous treatment of convex sets, convex functions, standard problem classes (LP, QP, SOCP, SDP, GP), duality, and numerical methods like Newton's method and interior-point methods.
- **Why Read It?**: The definitive theoretical reference for understanding optimal decision-making, robust engineering design, and modern machine learning formulations.

15. Computational Topology for Data Analysis

- **Author**: Tamal Krishna Dey and Yusu Wang
- **Publisher**: Cambridge University Press, 2022 (ISBN: 9781108422741)



- **Official Website**: Cambridge Book Page
- **Overview**: A comprehensive and modern introduction to the computational and algorithmic aspects of topology, focusing specifically on topological data analysis (TDA). It covers simplicial complexes, homology, persistent homology, barcodes, and their applications in analyzing shape and structure in high-dimensional datasets.
- **Why Read It?**: Essential for understanding how abstract topological structures are computed algorithmically and applied to extract robust, noise-resistant features from data.

Download PDF Version

You can download the complete **Math for Data Science** book as a single offline PDF file.

📥 Download Link

Click the link below to download the complete book:

👉 📥 **Download Complete Book PDF (A4 Format)**

📖 PDF Features

- Single-page layout containing all chapters of this blog.
- Ready for offline reading and printing.
- Standard A4 format with margins optimized for tablet or paper reading.